



Bachelor Informatik/Wirtschaftsinformatik

Integrationsprojekt Kryptographie Teil 2

Spezifikationsdokument

Hannover, 10. Juni 2026

Sommerquartal 2026

Nina Neumann, Johanna Amini, Amirreza Ahmadi Darani, Xuan-Loc Tran, Franz Müller, Tammo Lütten,
Jannes Breuer, Bjarne Möller, Darko Marjanovic, Oliver Knöbl, Rafael Ulmer

Anmerkung

Die vorliegende Arbeit wurde im Rahmen eines wissenschaftlichen Projekts an der Fachhochschule für die Wirtschaft (FHDW) Hannover erstellt. Alle in dieser Arbeit verwendeten Informationen, Daten und Ergebnisse sind ausschließlich für akademische Zwecke bestimmt. Die Autoren übernehmen keine Verantwortung für die Richtigkeit, Vollständigkeit oder Aktualität der in dieser Arbeit enthaltenen Informationen. Jegliche Nutzung der Inhalte dieser Arbeit erfolgt auf eigenes Risiko.

Erklärung der Selbstständigkeit

Hiermit versichern wir, dass wir das vorliegende Spezifikationsdokument selbständig verfasst und keine anderen als die angegebenen Quellen verwendet haben. Alle Textstellen und andere Inhalte wie Bilder, Quellcode oder Daten, die wörtlich oder sinngemäß aus anderen Quellen entnommen sind, haben wir eindeutig mit korrekten Quellenangaben versehen. Über Zitierrichtlinien sind wir schriftlich informiert worden. Diese Arbeit wurde bisher in gleicher oder ähnlicher Form, auch nicht in Teilen, keiner anderen Prüfungsbehörde vorgelegt.

Falls wir während der Erstellung dieser Arbeit generative KI oder andere KI-gestützte Technologien verwendet haben, die über grundlegende Werkzeuge für Übersetzungen und zur Überprüfung von Grammatik oder Rechtschreibung hinausgehen, erklären wir, nach der Nutzung dieser Tools den Inhalt unseres Spezifikationsdokuments überprüft und bearbeitet zu haben und übernehmen die volle Verantwortung für die diesbezüglichen Ausführungen. Eine Verwendung solcher Werkzeuge haben wir in unserer Arbeit dokumentiert (bspw. im Abschnitt „Struktur der Arbeit, Methodik“ oder durch Erläuterungen zur Nutzung im Anhang).

Inhaltsverzeichnis

1 Einleitung

Fully Homomorphic Encryption (FHE) ermöglicht es, Berechnungen direkt auf verschlüsselten Daten durchzuführen, ohne sie jemals zu entschlüsseln. Der Server sieht ausschließlich Ciphertexte — Eingaben, Zwischenergebnisse und Ausgaben bleiben für ihn semantisch leer. Nur der Besitzer des privaten Schlüssels kann die Ergebnisse lesen.

Dieses Dokument ist die technische Spezifikation eines Systems, das neun praktische Anwendungsfälle auf Basis von FHE realisiert. Als kryptographische Grundlage dient **TFHE-rs** [?] — eine Rust-Bibliothek für das Torus-FHE-Schema, das besonders effiziente boolesche und ganzzahlige Operationen auf Ciphertexten erlaubt.

Systemüberblick: Das System besteht aus neun unabhängigen Rust-Microservices, einem gemeinsamen Angular-Frontend und einer geteilten Plattform. Jeder Service implementiert genau einen Use Case. Das Frontend verschlüsselt alle Eingaben lokal im Browser per WebAssembly, bevor sie den Client verlassen. Der private Schlüssel (`ClientKey`) verlässt den Browser nie.

Ziel dieses Dokuments: Das Dokument spezifiziert jeden Use Case in einheitlicher Struktur: Funktionsbeschreibung, OpenAPI-Schnittstelle, Trust- und Threat-Modell, FHE-Designentscheidungen, algorithmische Komplexität, Performance-Messungen und bekannte Limitationen. Querschnittsthemen (Architektur, Routing, Key Lifecycle, Serialisierung, Deployment) sind vorangestellt und gelten für alle Use Cases gleichermaßen.

Dokumentstruktur:

1. **Querschnittssektionen** — systemweite Architektur, Gateway, Key Lifecycle, Serialisierung, Build und Deployment
2. **Client** — das Angular-Frontend und seine Rolle im FHE-Ablauf
3. **Use Cases 01–09** — je eine vollständige Spezifikation pro Service

2 Querschnittssektionen

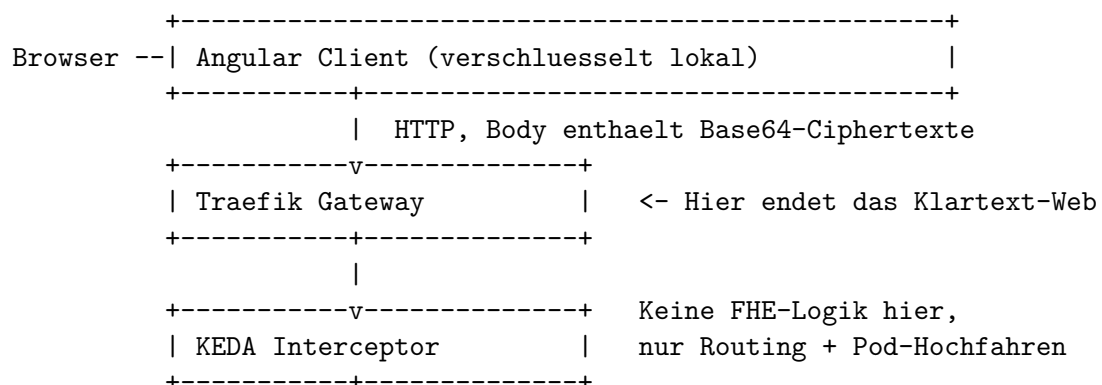
2.1 Architektur-Übersicht

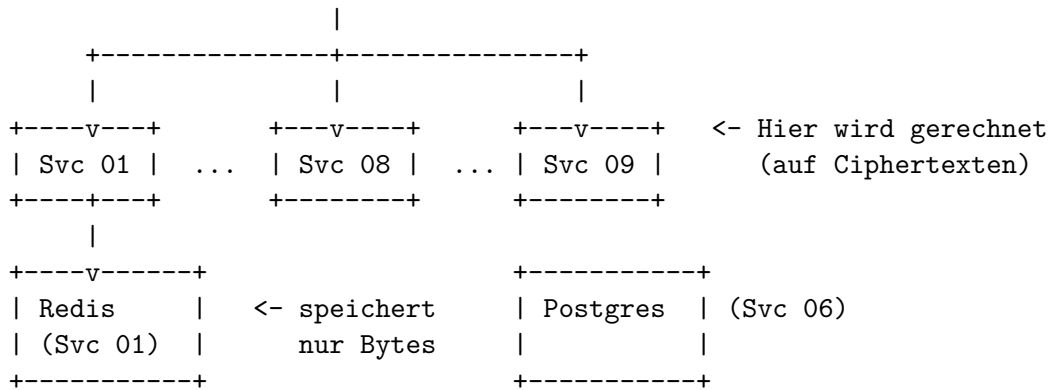
Das System besteht aus neun unabhängigen Rust-Services, einem Angular-Frontend und einer geteilten Plattform. Jeder Use Case ist ein eigener kleiner Server, der nur sich selbst kennt. Keine zwei Services teilen sich Daten oder Schlüssel. Das macht Entwicklung und Betrieb einfacher, denn stürzt ein Service ab, betrifft das die anderen nicht.

Verwendete Patterns:

- **Mono-Repo:** Der gesamte Code (Rust-Services, Shared Crates, Angular-Frontend, Helm-Charts, CI-Workflows, Dokumentation) liegt in einem einzigen Git-Repo. Das erlaubt atomare Commits über mehrere Schichten hinweg und vermeidet Versions-Mismatches mehrerer Repos.
- **Microservice-Pattern:** Jeder Use Case ist ein eigener Service mit eigenem Lebenszyklus, eigenem Namespace und gegebenenfalls eigener Datenbank.
- **Cargo Workspace:** Ein einziges Cargo-Projekt enthält alle Services und geteilten Bausteine als Sub-Crates. Vorteil ist eine gemeinsame Cargo.lock, ein einzelner cargo build-Aufruf kompiliert alles und Versionen sind zentral pinnbar.
- **Shared Library:** Wiederkehrende Aufgaben wie Health, Metriken, Tracing und OpenAPI-Doku liegen in shared/-Crates und werden per Pfad-Dependency in jeden Service eingehängt.
- **Code-First OpenAPI:** Die OpenAPI-Spec wird nicht handgeschrieben, sondern zur Laufzeit aus den Rust-Typen generiert (via aide und schemars). DTOs im Code sind die Quelle der Wahrheit, Swagger-UI und JSON-Spec entstehen automatisch.
- **Stateless-by-Default mit optionalem In-Memory-State:** Die meisten Services sind zustandslos. Wo ein UC eine Session braucht, lebt der State im RAM des Pods, nicht in der Datenbank, weil dort niemals Schlüssel oder noch-entschlüsselbare Ciphertexte liegen dürfen.
- **Path-based Routing:** Services kennen ihren öffentlichen Pfad nicht. Das Routing wird komplett am Gateway gemacht.

Komponentendiagramm:





Komponenten und Verantwortung:

Komponente	Technologie	Verantwortung
Frontend	Angular + TypeScript + TFHE.js	Benutzeroberfläche, clientseitige FHE-Operationen, Schlüsselerzeugung
API Gateway	Traefik	Pfadbasiertes HTTP-Routing, TCP- Routing für Redis und Postgres
KEDA Interceptor	KEDA HTTP Add-on	Puffert Requests, fährt Pods bei Bedarf von 0 hoch
UC-Service 1-9	Rust + Axum	Fachlogik des jeweiligen Use Cases, FHE-Berechnungen
Redis (Svc 01)	Redis	Speichert Bytes als Key-Value-Store, ohne Kenntnis der Inhalte
Postgres (Svc 06)	Postgres	Persistenz fachlicher Klartext- Metadaten für den Genomics-UC
Cluster	k3s auf NetCup	Orchestrierung und Skalierung
Observability	Prometheus, Grafana, Tempo	Monitoring, Metriken und verteiltes Tracing

Workspace-Struktur:

```

tfhe-crypto/
+-- Cargo.toml           # Workspace-Wurzel (alle Members + Deps)
+-- client/              # Angular Frontend
+-- shared/
|   +-- health/          # Health-Endpunkte
|   +-- metrics/         # Prometheus-Exporter
|   +-- observability/   # OpenTelemetry / Tracing
|   +-- openapi-docs/    # Swagger-UI Generator
+-- services/
|   +-- 01-encrypted-key-value-store/
|   +-- 02-encrypted-age-verification/
|   +-- ...
|   +-- 09-encrypted-program-execution/
+-- helm/                # Kubernetes-Deployments
|   +-- infrastructure/  # Cluster-Basis (Traefik, ArgoCD, KEDA, Monitoring)
|   +-- services/        # Helm-Charts pro Service

```

+-- .github/workflows/ # CI/CD

Service-Inventar:

UC	Name	Pfad	Datenbank
01	encrypted-key-value-store	/kv	Redis
02	encrypted-age-verification	/age-verification	-
03	encrypted-voting-polling	/voting	-
04	sealed-bid-auction	/auction	-
05	encrypted-statistics-service	/statistics	-
06	encrypted-genomics	/genomics	Postgres
07	encrypted-image-processing	/image-processing	-
08	encrypted-leaderboard	/leaderboard	-
09	encrypted-program-execution	/program-execution	-

Shared Crates: Jeder Service erledigt manche Dinge gleich, etwa Health-Endpoints, Metriken, Tracing und OpenAPI-Doku. Damit das nicht neunfach kopiert werden muss, liegen diese Bausteine in `shared/` und werden pro Service eingehängt:

Crate	Was es liefert
health	/healthz, /readyz, /version, damit Kubernetes weiß, ob der Service läuft
metrics	/metrics-Endpoint mit Request-Zählern und Latenzen für Prometheus
observability	Verbindet den Service mit dem Trace-Sammler (Tempo), jeder Request wird sichtbar
openapi-docs	Erzeugt automatisch eine /docs-Seite mit Swagger UI aus dem Rust-Code

Die FHE-Grenze: Nur der Browser des Initiators und die Service-Pods sehen jemals echte FHE-Werte. Alle anderen Komponenten (Gateway, Interceptor, Redis, Postgres) arbeiten mit den verschlüsselten Bytes wie mit beliebigen anderen Daten. Sie wissen nicht, was darin steht, und müssen es auch nicht.

Folgende Komponenten verarbeiten FHE-spezifische Datentypen:

- Frontend (TFHE.js)
- UC-Services (Rust + tfhe-rs)

Folgende Komponenten kennen keine FHE-internen Datentypen und sehen nur opake Byte-Arrays:

- Traefik Gateway
- KEDA Interceptor
- Kubernetes
- Redis und Postgres
- Observability-Komponenten

2.2 Gateway und Routing

Im Cluster läuft nur Traefik als öffentliche Eintrittstür. Er nimmt HTTP auf Port 80 entgegen und entscheidet anhand des Pfads, an welchen Service der Request weitergeht. Für die Datenbank-Zugriffe von außen (Service 1 Redis, Service 6 Postgres) kommen TCP-Listener auf Port 6301 beziehungsweise 5432 dazu. Diese externen Datenbankzugriffe werden über die Gateway API im Experimental Channel in Version 1.2.0 abgebildet, weil das nötige TCPRoute im Standard-Channel noch nicht enthalten ist.

Verarbeitung eines Requests: Am Beispiel eines Aufrufs `GET /leaderboard/123/entries` arbeitet Traefik in folgenden Schritten:

1. Der Pfad-Präfix wird ausgewertet. Im Beispiel passt der Präfix zur HTTPRoute des Leaderboard-Services.
2. Traefik strippt den Präfix weg. Der Service empfängt nur noch `/123/entries`. Dadurch wissen die Services nichts von ihrem öffentlichen Pfad und bleiben umziehbar.
3. Ein interner Host-Header wird gesetzt, damit der nachgeschaltete KEDA-Interceptor weiß, welcher Service gemeint ist.
4. Der Request wird weitergeleitet, allerdings nicht direkt an den Service-Pod, sondern erst an den Interceptor.

Sichtbarkeit von Daten im Gateway: Was Traefik sieht, sind HTTP-Methode, Pfad, Header und der Body. Den Body sieht es nur als Byte-Strom, ohne ihn zu lesen oder zu verändern. Da der Body bei FHE-Endpunkten nur Base64-kodierte Ciphertexte enthält, bleibt der eigentliche Inhalt für das Gateway opak.

Authentifizierung und Rate Limiting: Es gibt aktuell keine Authentifizierung, kein Rate-Limit und keine Web Application Firewall am Gateway. Jeder, der die URL kennt, kann beliebige Requests an jeden Service schicken. Das ist im Rahmen eines Uni-Projekts bewusst so akzeptiert. Das System ist kein Produkt, sondern eine Lernumgebung für FHE. Für einen Produktiv-Betrieb müsste ein Auth-Layer (etwa OIDC am Gateway oder Token-Prüfung pro Service) und ein Rate-Limit ergänzt werden.

Fehlerpropagation: 5xx-Antworten der Services werden 1:1 durchgereicht. Wenn ein Service gerade hochfährt, sieht der Client kein 503, weil der KEDA-Interceptor den Request solange puffert. Erst wenn der Hochfahr-Vorgang in den Timeout von 30 Sekunden läuft, antwortet das Gateway mit 504 Gateway Timeout.

2.3 Key Lifecycle

Aus Sicht der Schlüssel gibt es genau zwei Rollen. Der **Initiator** besitzt den ClientKey, erstellt die Session und kann als einziger entschlüsseln. **Teilnehmer** sind null bis beliebig viele weitere Parteien, die nur den PublicKey haben und damit zwar verschlüsseln, aber nicht entschlüsseln können. Wer in welcher Rolle ist, hängt vom jeweiligen UC ab. Manche UCs kommen mit nur einer Partei (Initiator allein) aus, andere lassen viele Teilnehmer zu.

Schlüsseltypen: In TFHE erzeugt der Initiator drei Arten von Schlüsseln:

- **ClientKey:** Der private Schlüssel zum Entschlüsseln. Bleibt immer im Browser und wird nie an einen Server geschickt.

- **ServerKey:** Erlaubt dem Service, auf verschlüsselten Daten zu rechnen, ohne sie selbst entschlüsseln zu können. Wird beim Erstellen der Session hochgeladen.
- **PublicKey:** Kann an Teilnehmer weitergegeben werden, damit sie verschlüsseln können, ohne den ClientKey zu kennen. Wenn ein UC keine Teilnehmer hat, bleibt der PublicKey ungenutzt.

Konzeptueller Datenfluss: Details, Endpunkt-Namen und ob es überhaupt einen Session-Begriff gibt, hängen vom UC ab. Der grundlegende Ablauf ist aber immer derselbe:

1. **Schlüsselerzeugung:** Der Initiator erzeugt clientseitig alle drei Schlüssel. Der ClientKey verlässt den Client nie.
2. **ServerKey-Übertragung:** Der Initiator schickt den ServerKey zum Service.
3. **Dekompression:** Der Service dekomprimiert den ServerKey. Da das mehrere Sekunden CPU-Zeit und mehrere hundert MB RAM kostet, läuft es in einem Hintergrund-Thread und nicht im HTTP-Handler.
4. **Eingaben:** Werden verschlüsselt an den Service geschickt, entweder vom Initiator selbst mit ClientKey oder PublicKey oder von Teilnehmern mit dem PublicKey.
5. **Berechnung:** Der Service rechnet auf den verschlüsselten Werten.
6. **Ergebnis:** Kommt verschlüsselt zurück. Nur der Initiator kann es mit seinem ClientKey entschlüsseln.

Speicherung und Isolation: Schlüssel und sonstiger Sitzungs-State leben ausschließlich im RAM des Service-Pods. Auf der Festplatte oder in einer Datenbank landen nur fachliche Klartext-Metadaten, niemals Schlüssel oder Ciphertexte, die noch jemand entschlüsseln könnte. Auch der Redis-Store von Service 01 und die Postgres-Instanz von Service 06 speichern nur opake Bytes beziehungsweise reine Metadaten. Die Isolation zwischen Sessions ergibt sich automatisch aus der Trennung pro Service-Pod und der separaten Verwaltung im prozesseigenen Speicher.

Lebensdauer und Session-Ende: Wenn der Pod neu startet oder von KEDA auf 0 skaliert wird, ist alles im RAM weg. Die betroffene Partei muss von vorn anfangen. Eine Wiederverwendung der Schlüssel über einen Pod-Lebenszyklus hinaus ist nicht vorgesehen und auch nicht möglich, weil der ClientKey ohnehin nur im Browser des Initiators existiert. Einige UCs ergänzen das um zusätzliche Aufräum-Mechanismen, etwa eine Idle-Eviction einzelner Sessions, um RAM früher freizugeben.

2.4 Serialisierung

FHE-Ciphertexte sind im Kern Byte-Arrays, die nicht als JSON-Zahl darstellbar sind. Die Wahl, wie sie übers Netz transportiert werden, ist in allen Services gleich:

Schicht	Format
Ciphertext intern (Service-interne Verarbeitung)	bincode (binäres Format)
Ciphertext über HTTP	Base64-kodierter Bincode
Request- und Response-Hüllen (DTOs)	normales JSON via serde
OpenAPI-Spec für Swagger UI	automatisch aus Rust-Typen

Warum Base64: Damit der Ciphertext in einem JSON-String-Feld liegen kann, denn JSON verträgt keine rohen Bytes. Das kostet etwa 33 % Overhead, ist aber browserseitig trivial zu de- und enkodieren.

Typische Größen (TFHE-rs Standard-Konfiguration):

Typ	Bytes (bincode)	Bytes (base64)
FheBool	~ 50 KB	~ 67 KB
FheUint8	~ 50 KB	~ 67 KB
FheUint16	~ 100 KB	~ 134 KB
FheUint64	~ 400 KB	~ 534 KB
CompressedServerKey	~ 100 MB	~ 134 MB
PublicKey	~ 50 KB	~ 67 KB

Body-Limit: Die Größen sind insbesondere für die Endpunkte relevant, die einen ServerKey entgegennehmen. Axum cappt den Body sonst standardmäßig bei 2 MB. In den Services ist deshalb explizit ein Limit von 2 GiB gesetzt, damit der Upload des dekomprimierbaren ServerKey-Materials durchgeht.

2.5 Build, Deployment, Tests

Pipelines: Die CI/CD-Pipelines liegen unter `.github/workflows/` und laufen vollständig auf GitHub-hosted Runnern (`ubuntu-latest`). Auf dem Cluster selbst gibt es keine self-hosted Runner.

Workflow	Wann	Was
ci.yml	Push auf Feature-Branch	Format-Check, Linter (Clippy als verpflichtendes Gate mit <code>-D warnings</code>), Tests mit Coverage
release.yml	Push auf main	Patch-Version bumpen, Docker-Image bauen und nach <code>ghcr.io</code> pushen, README und Helm-Values updaten

Der Helm-Values-Update am Ende ist wichtig. Erst wenn das Image garantiert in der Registry liegt, wird der Tag im Chart hochgezogen. Sonst würde ArgoCD versuchen, ein noch nicht existierendes Image zu deployen.

Image-Build: Das Dockerfile ist als Multi-Stage-Build mit `cargo-chef` umgesetzt. `cargo-chef` ist ein Trick, der Rust-Dependencies separat von der eigenen Quellcode-Schicht baut. Bei Code-Änderungen ohne neue Dependency wird der teure Compile-Schritt aus dem Docker-Cache wiederverwendet. Builds gehen von rund 10 Minuten auf etwa 1 Minute runter. Das fertige Image basiert auf `debian:bookworm-slim` und ist 100 bis 200 MB groß. Zur Optimierung der FHE-Leistung wird die `tfhe`-Crate bereits im Entwicklungsprofil mit `opt-level=3` kompiliert. Für produktive Builds kommt zusätzlich eine CPU-Optimierung über `target-cpu=znver5` zum Einsatz.

Deployment: Das Deployment erfolgt auf einem k3s-Cluster nach dem GitOps-Prinzip mit ArgoCD. Jede Änderung an `helm/services/*` wird gepusht, ArgoCD erkennt sie und deployt automatisch ohne manuelles `helm install`. Die Infrastruktur-Charts (Traefik, ArgoCD, KEDA, Monitoring) werden bewusst nicht über ArgoCD verwaltet, sondern manuell per `helm install` installiert, damit beim Neuaufbau des Clusters eine klare Reihenfolge eingehalten werden kann.

Teststrategie: Jeder Service bringt seine eigenen Tests mit. Was konkret geprüft wird, hängt von der Fachlichkeit des UCs ab und steht in der jeweiligen UC-Sektion. Auf Pipeline-Ebene ruft die CI für jeden Service einheitlich `cargo test --release -- --test-threads=1` auf.

- **Unit-Tests:** Tests einzelner Komponenten und Funktionen innerhalb der Services.
- **Integrationstests:** Tests der Endpunkte und des Zusammenspiels mehrerer Komponenten. Sie laufen im Release-Modus, um realistische Laufzeiten für FHE-Operationen zu gewährleisten.
- **FHE-Korrektheitstests:** FHE-Ergebnisse werden gegen Klartextreferenzen validiert.

Das Flag `--release` ist Pflicht, weil FHE im Debug-Modus rund 10-mal langsamer ist. `--test-threads=1` verhindert, dass parallele Tests den ServerKey-Speicher mehrfach allokalieren.

Testabdeckung: Die Code Coverage wird in der CI per `cargo-llvm-cov` pro Service ermittelt und als Zeilen-Coverage-Badge ins jeweilige Service-README geschrieben. Sie ist nicht hart als CI-Gate erzwungen. Der Wert dient als Indikator, nicht als Block.

Hinzufügen eines neuen Services:

1. Neuer Rust-Service als Crate im Cargo-Workspace anlegen.
2. Shared Crates (health, metrics, observability, openapi-docs) als Dependencies einbinden.
3. Dockerfile und Helm-Chart unter `helm/services/` ergänzen.
4. Routing-Konfiguration im Gateway um eine HTTPRoute mit passendem Pfad-Präfix erweitern.
5. GitHub-Actions-Pipeline auf den neuen Service ausweiten.
6. Deployment über ArgoCD automatisch ausrollen lassen.

3 Use-Cases

3.1 Encrypted-Key-Value-Store

3.1.1 Funktionsbeschreibung

3.1.2 OpenAPI-Schnittstelle

3.1.3 Trust- und Threat-Model

3.1.4 FHE-Designentscheidungen

3.1.5 Komplexität der eigenen Algorithmen

3.1.6 Performance-Messung

3.1.7 Limitationen

3.2 Encrypted-Age-Verification

3.2.1 Funktionsbeschreibung

Der Use Case Encrypted Age Verification löst das Problem, das Alter einer Person serverseitig zu prüfen, ohne dass der Server den tatsächlichen Alterswert jemals im Klartext zu Gesicht bekommt. Das Ergebnis der Prüfung (volljährig: ja/nein) wird ebenfalls verschlüsselt zurückgegeben – der Server kennt weder Eingabe noch Ausgabe.

Das Alter wird bereits auf dem Client mit dem ClientKey verschlüsselt und ausschließlich in verschlüsselter Form an das Backend übertragen. Das Backend führt die Altersüberprüfung mittels Fully Homomorphic Encryption (TFHE) durch, ohne den zugrunde liegenden Alterswert entschlüsseln zu können. Die Entschlüsselung des Ergebnisses ist ausschließlich durch den Besitzer des ClientKeys möglich.

3.2.2 Akteure

- **Client:** Generiert das TFHE-Schlüsselpaar, verschlüsselt das Alter mit dem ClientKey, sendet `encrypted_age` und `server_key` an den Server, empfängt das verschlüsselte Ergebnis und entschlüsselt es lokal. Der Client besitzt den ClientKey und ist der einzige Akteur, der das Ergebnis entschlüsseln kann.
- **Backend (Server):** Empfängt `encrypted_age` und `CompressedServerKey`, setzt den ServerKey, führt die homomorphe Altersüberprüfung (`age_check`) durch und gibt das verschlüsselte Ergebnis (`FheBool`) zurück. Der Server sieht zu keinem Zeitpunkt das Alter oder das Ergebnis im Klartext.

3.2.3 Lebenszyklus einer Session

Phase 1 – Session-Setup (einmalig)

1. Client generiert ClientKey und CompressedServerKey lokal. Das Alter wird als `FheInt8` mit dem ClientKey verschlüsselt.
2. Client sendet `POST /session` mit dem `CompressedServerKey` (~80 MB) als JSON-Body.
3. Server dekomprimiert den Key einmalig (`CompressedServerKey::decompress()`), speichert den ServerKey im AppState und gibt eine `session_id` (UUID) zurück.

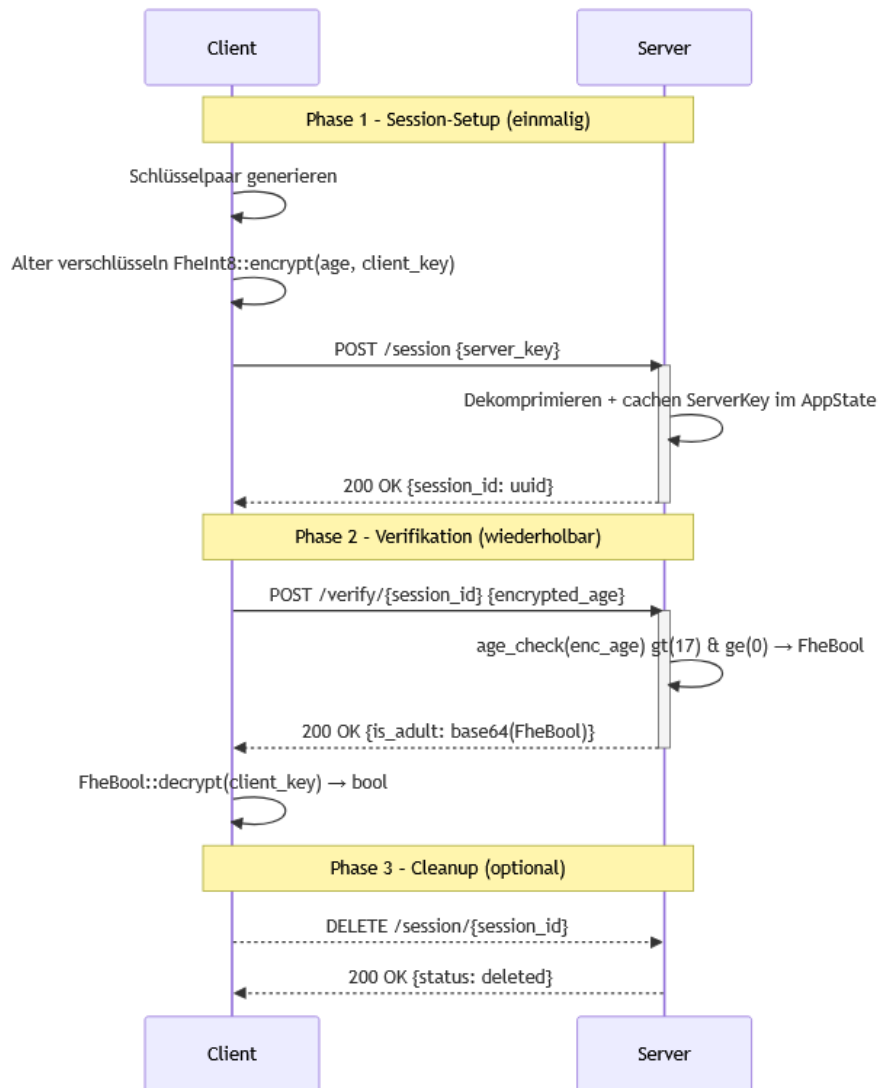
Phase 2 – Verifikation

4. Client sendet `POST /verify/{session_id}` mit ausschließlich `encrypted_age` im Body.
5. Server liest den gecachten ServerKey aus dem AppState, führt `age_check` aus (`enc_age.gt(17) & enc_age.ge(0)`) und gibt das verschlüsselte `FheBool`-Ergebnis zurück.
6. Client entschlüsselt das `FheBool` mit dem ClientKey und erhält das boolesche Ergebnis.

Phase 3 – Cleanup

7. Client sendet DELETE /session/{session_id}, um die Session zu beenden und den serverseitigen RAM freizugeben.

3.2.4 Verhaltensdiagramm



3.2.5 OpenAPI-Schnittstelle

Der Service stellt eine session-basierte verschlüsselte Verifikations-API bereit. Die OpenAPI-Definition wird automatisch generiert und ist unter /openapi.json sowie /docs (Swagger UI) erreichbar.

Im gesamten Service wird der ServerKey als CompressedServerKey übertragen (bincode-serialisiert, base64-kodiert).

POST /session Lädt den `CompressedServerKey` einmalig hoch, dekomprimiert ihn und gibt eine `session_id` zurück.

Request:

```
{
  "server_key": "string"
}
```

Response 200:

```
{
  "session_id": "uuid"
}
```

Fehlercodes:

- **400** – Ungültiges Base64 oder beschädigtes bincode in `server_key`

Body-Limit: 2 GiB (notwendig wegen der Größe des `CompressedServerKey`)

POST /verify/{session_id} Führt eine verschlüsselte Altersverifikation durch.

Request:

```
{
  "encrypted_age": "string"
}
```

Response 200:

```
{
  "is_adult": "string"
}
```

`is_adult` ist ein base64-kodierter, bincode-serialisierter `FheBool` – `true`, wenn `Alter` ≥ 18 und ≥ 0 .

Fehlercodes:

- **400** – Ungültiges Base64 oder beschädigtes bincode in `encrypted_age`
- **404** – Session nicht gefunden
- **500** – Serialisierungsfehler beim Kodieren des Ergebnisses

DELETE /session/{session_id} Löscht die Session und gibt den serverseitigen RAM frei.

Response 200:

```
{
  "status": "deleted"
}
```

Fehlercodes:

- **404** – Session nicht gefunden

3.2.6 Trust- und Threat-Model

	Server klar	Server verschlüsselt	Nur Client
Alter (numerischer Wert)		X	
Ergebnis (volljährig: ja/nein)		X	
ServerKey / CompressedServerKey	X		X
ClientKey			X

TFHE schützt ausschließlich den Inhalt des Alters und des Ergebnisses. Für den Server bleiben weiterhin verschiedene Metadaten sichtbar:

- Zeitpunkte und Häufigkeit der Anfragen
- IP-Adresse des anfragenden Clients
- Charakteristische Payload-Größe (~80 MB), die auf TFHE-Schlüsselübertragung schließen lässt
- Anzahl der Anfragen pro IP

Restvertrauen in den Server: Der Server kann zwar das Alter nicht entschlüsseln, muss jedoch weiterhin als korrekter Ausführer der Verifikationslogik vertrauenswürdig sein. Insbesondere wird angenommen, dass der Server:

- `age_check` korrekt und unverändert ausführt (kein Austausch gegen eine Funktion, die immer `true` zurückgibt)
- Das korrekte `FheBool`-Ergebnis zurücksendet und es nicht durch einen manipulierten Ciphertext ersetzt
- Den `ServerKey` nicht persistiert oder weitergibt

TFHE reduziert somit die Vertrauensabhängigkeit hinsichtlich der Inhaltsvertraulichkeit, ersetzt jedoch kein vollständig vertrauensloses Protokoll.

Annahmen außerhalb von TFHE:

- Der Client führt Verschlüsselung und Entschlüsselung korrekt aus.
- Die verwendete TFHE-rs-Bibliothek wird als kryptographisch korrekt implementierte Black Box betrachtet.
- Es gibt keine Authentifizierung am Endpunkt: Jeder, der einen gültigen `CompressedServerKey` besitzt, kann Anfragen stellen.

Schutzversprechen:

- Der Server kann das Alter des Nutzers nicht im Klartext lesen.
- Der Server kann nicht feststellen, ob das Ergebnis `true` oder `false` ist.
- Das Alter wird ausschließlich verschlüsselt verarbeitet.

Nicht garantiert werden:

- Schutz vor einem Server, der `age_check` durch eine manipulierte Funktion ersetzt
- Schutz vor Traffic-Analyse (Payload-Größe, Timing)
- Authentizität des Clients

Das bedeutet der Server kennt nicht das Alter des Nutzers und kann nicht feststellen, ob das Ergebnis positiv oder negativ ausgefallen ist. Sichtbar bleiben ausschließlich Zeitpunkt, Herkunft und Häufigkeit der Anfragen.

3.2.7 FHE-Designentscheidungen

Für das Alter wird `FheInt8` (vorzeichenbehaftetes 8-Bit-Integer, Wertebereich -128 bis 127) verwendet. Diese Wahl ergibt sich aus zwei Gründen:

Altersangaben in Jahren liegen typischerweise im Bereich $0-127$, also weit innerhalb von `i8`. `FheUInt8` wäre für positive Werte ausreichend, aber `FheInt8` erlaubt es, negative Eingaben explizit als ungültig zu erkennen und abzufangen (s. `is_positive`-Check). Zudem unterstützt `FheInt8` `gt` und `ge` mit vorzeichenbehafteter Ganzzahlsemantik, was den Negativen-Grenzwert-Test (`ge(0)`) korrekt macht.

Das Ergebnis ist ein `FheBool` (verschlüsseltes Bit), was dem binären Charakter der Frage (volljährig: ja/nein) entspricht.

Der Server führt ausschließlich zwei Vergleiche und eine boolesche Verknüpfung aus. Andere homomorphe Operationen werden nicht benötigt, dadurch bleibt die serverseitige Auswertung auf die minimal nötige Anzahl FHE-Operationen beschränkt.

Operation	Verwendung
<code>FheInt8::gt</code>	<code>enc_age > 17</code> $\rightarrow$ Alter ≥ 18
<code>FheInt8::ge</code>	<code>enc_age >= 0</code> $\rightarrow$ kein negativer Wert
<code>FheBool::bitand</code>	Verknüpfung beider Bedingungen

Verworfen Alternativen:

FheUInt8 statt FheInt8: Wäre für rein positive Eingaben ausreichend. Verworfen, weil damit keine sinnvolle Behandlung negativer Eingaben möglich ist – FheUInt8 interpretiert -1 als 255 , was den Negativen-Grenzwert-Test ($\text{age_check}(-1) \rightarrow \text{false}$) unmöglich machen würde.

Schwellwert als verschlüsselter Parameter: Denkbar wäre, den Grenzwert (18) ebenfalls als FheInt8 zu übergeben, um ihn serverseitig variabel zu halten. Verworfen, weil der Schwellwert in diesem Use Case keine schützenswerte Information ist und eine feste Konstante die Implementierung erheblich vereinfacht.

FheInt32 oder größere Typen: Größere Bitbreiten würden höhere Latenzen pro homomorpher Operation verursachen, ohne Mehrwert – Altersangaben benötigen nicht mehr als 8 Bit.

3.2.8 Komplexität der eigenen Algorithmen

Da der Use Case ausschließlich aus einer festen Anzahl homomorpher Operationen auf einem einzelnen Ciphertext besteht, gibt es keine parametrisierten Eingabegrößen. Die Komplexität aller Funktionen ist konstant.

Funktion	Zeitkomplexität	Platzkomplexität
decode_server_key	$O(1)$	$O(1)$
decode_encrypted_age	$O(1)$	$O(1)$
age_check	$O(1)$	$O(1)$
encode_result	$O(1)$	$O(1)$
verify_age (gesamt)	$O(1)$	$O(1)$

$\text{verify_age} - O(1), O(1)$: Die Funktion führt genau zwei Vergleiche ($\text{gt}(17)$, $\text{ge}(0)$) und eine AND-Verknüpfung ($\&$) auf einem FheInt8-Wert durch. Alle drei sind Operationen fester Bitbreite (8 Bit) auf einem einzelnen Ciphertext – unabhängig von jeder Eingabegröße. Gemäß der Konvention, homomorphe Operationen als $O(1)$ zu zählen, ist $\text{age_check} \in O(1)$.

Die De- und Serialisierungsschritte (bincode , base64) operieren auf Byte-Arrays fester Länge (durch die TFHE-rs-Typen bestimmt) und sind ebenfalls $O(1)$ bezüglich fachlicher Parameter.

3.2.9 Performance-Messung

Mess-Setup & Methodik

Die Performance- und Stresstests wurden auf einem virtuellen KVM-Server von Netcup durchgeführt. Die Last wurde extern mittels $k6$ von einer lokalen Windows-Maschine über das Internet injiziert.

Getestet wurde $\text{POST } /verify/\{session_id\}$ – der rechenintensive Endpunkt. $\text{POST } /session$ (einmaliger Setup) und $\text{DELETE } /session/\{id\}$ (Cleanup) wurden nicht separat belastet, da sie nicht im kritischen Pfad der Verifikation liegen. Die Gesamtlatenz setzt sich aus zwei Anteilen zusammen:

1. Netzwerkübertragung von encrypted_age – vernachlässigbar
2. $\text{age_check}()$ – zwei FHE-Vergleiche + AND-Verknüpfung auf dem gecachten ServerKey

Test 1 – Baseline (1 VU, 10 sequentielle Requests, session-basiert) (09.06.2026)

Metrik	Wert
p50	114,17 ms
p90	126,44 ms
p95	132,74 ms
Maximum	176,31 ms
Fehlerrate	0 %
Durchsatz	~1,47 req/s

Fazit: Im session-basierten Modus entfällt der 80 MB ServerKey-Transfer pro Request. Die eigentliche FHE-Operation (age_check) dauert ~114 ms. p50 und p95 liegen eng beieinander (114 ms vs. 133 ms), was auf ein stabiles und vorhersehbares Systemverhalten hinweist.

Test 2 – Stresstest (ramping bis 10 VUs, pro-VU Schlüsselpaar) (09.06.2026)

Metrik	Wert
p50	114,06 ms
p90	126,27 ms
p95	130,43 ms
Maximum	174,62 ms
Fehlerrate	0 %
Durchsatz	~2,27 req/s

Methodik: setup() baut alle 10 Sessions sequentiell auf, bevor der erste VU startet – jede Session mit eigenem Schlüsselpaar. Erst wenn alle 10 Sessions bereit sind, beginnt die Lastphase. Jede VU greift ausschließlich auf ihre eigene Session zu. Dadurch wird der ServerKey-Upload vollständig aus der Latenzbetrachtung herausgehalten und 10 echte parallele Clients simuliert.

Fazit: Unter 10 parallelen Clients mit je eigenem Schlüsselpaar und eigener Session bleibt die Verifikationslatenz nahezu identisch zur Baseline (p50: 114 ms, p95: 130 ms). Es treten keine Timeouts oder Fehler auf. Der set_server_key-Mutex wirkt sich nicht messbar aus, weil die FHE-Operation mit ~114 ms kurz genug ist, dass parallele Requests keine nennenswerte Wartezeit verursachen.

3.2.10 Limitationen

- Der CompressedServerKey (~80 MB) wird einmalig beim Session-Setup übertragen und dekomprimiert gecacht. Folgende Verify-Requests enthalten nur noch encrypted_age. Der gecachte ServerKey verbleibt im RAM des Servers, bis die Session per DELETE /session/{id} beendet wird – es gibt keine automatische Ablaufzeit.
- Sessions haben keine Authentifizierung: Jeder, der eine session_id kennt, kann Verify-Requests gegen diese Session stellen. In einer produktiven Umgebung müsste die Session an eine authentifizierte Identität gebunden sein.

- Der Server prüft nicht, ob ein `encrypted_age`-Ciphertext bereits zuvor verwendet wurde. Ein Angreifer, der einen Ciphertext abfängt, kann ihn beliebig oft einreichen.
- Der Client übergibt den `CompressedServerKey` selbst. Ein böartiger Client könnte einen manipulierten Key einreichen. In einer produktiven Umgebung müsste der Server-Key serverseitig fest hinterlegt sein.
- Der Client könnte einen beliebigen `FheInt8`-Wert übermitteln – der Server kann nicht verifizieren, dass die verschlüsselte Eingabe tatsächlich ein Alter darstellt oder aus einer vertrauenswürdigen Quelle stammt.
- Maximal darstellbarer Alterswert ist 127 Jahre (`i8: :MAX`). Dies ist für den Anwendungsfall ausreichend, aber die Wahl von `FheInt8` schließt größere Ganzzahlen strukturell aus.
- Es gibt keine Authentifizierung am Endpunkt. Jeder, der die API kennt, kann Anfragen stellen.

3.3 Encrypted-Voting-&-Polling

3.3.1 Funktionsbeschreibung

Der Use Case Voting/Polling ermöglicht die Durchführung vertraulicher Abstimmungen zwischen mehreren Teilnehmern. Ziel ist es, Abstimmungsergebnisse auszuwerten, ohne dass der Server Zugriff auf die einzelnen Stimmen der Teilnehmer erhält.

Hierzu werden Stimmen bereits auf dem Client verschlüsselt und ausschließlich in verschlüsselter Form an das Backend übertragen. Das Backend verarbeitet und aggregiert die verschlüsselten Stimmen mittels Fully Homomorphic Encryption (TFHE), ohne die zugrunde liegenden Klartexte entschlüsseln zu können. Die Entschlüsselung der aggregierten Ergebnisse ist ausschließlich durch den Besitzer des ClientKeys möglich.

Der Use Case unterstützt die Fragetypen Single Choice, Multiple Choice und Numeric. Eine Abstimmungssession kann beliebig viele Fragen enthalten und von mehreren Teilnehmern genutzt werden.

Akteure:

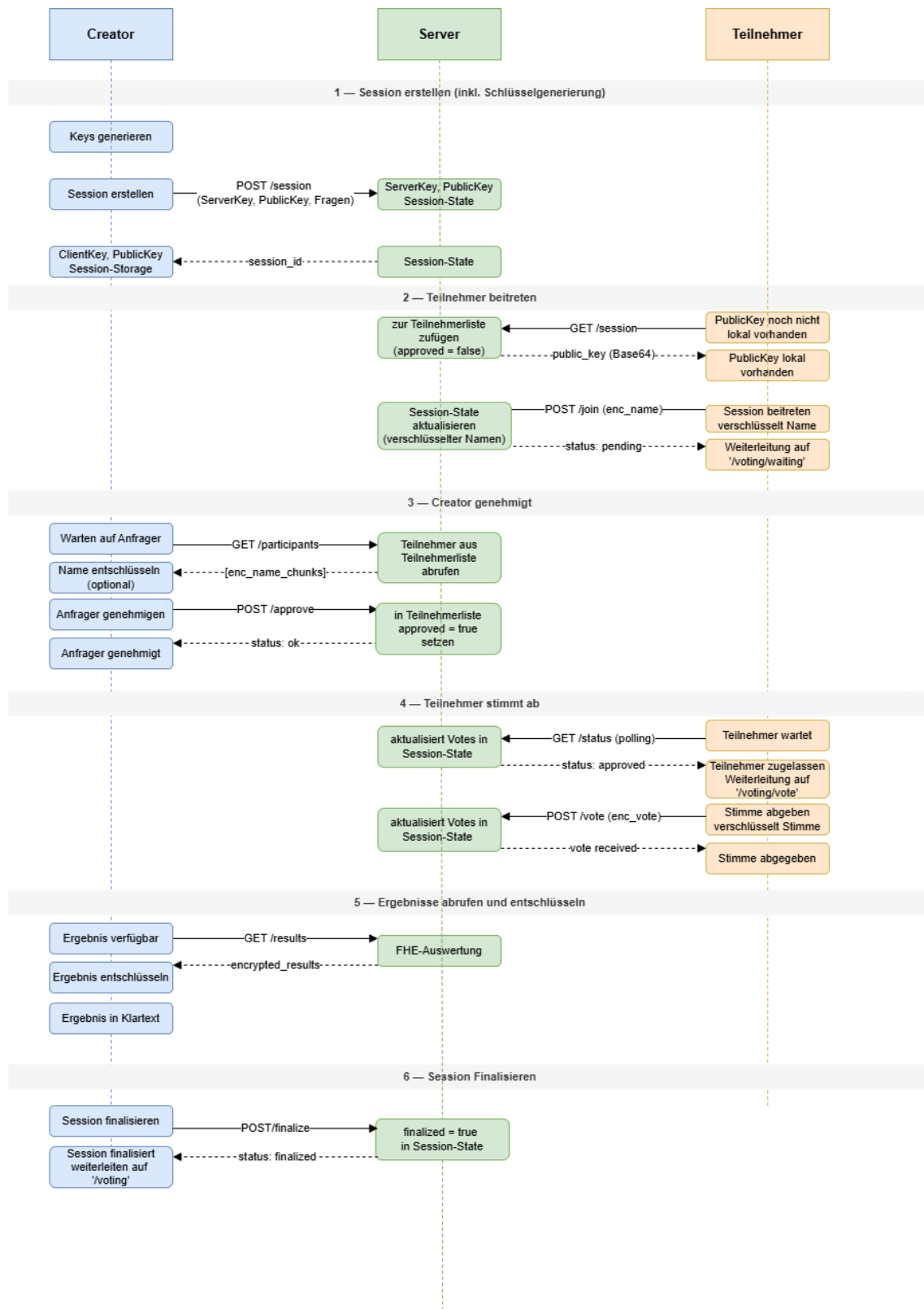
- **Ersteller (Client E):** Der Ersteller generiert das TFHE-Schlüsselpaar, definiert die Fragen und erstellt die Voting-Session. Zusätzlich verwaltet er die Teilnahme der Nutzer (Genehmigung oder Ablehnung). Der Client E besitzt den ClientKey und ist der einzige Akteur, der Ergebnisse entschlüsseln kann.
- **Teilnehmer (Client):** Ein Teilnehmer kann einer Session beitreten, indem er die Session-ID und seinen verschlüsselten Namen an den Server sendet. Nach Genehmigung durch den Ersteller kann der Teilnehmer seine Stimme(n) abgeben. Sowohl Name als auch Stimme werden clientseitig mit dem PublicKey verschlüsselt und an den Server übertragen. Teilnehmer können zu keinem Zeitpunkt die aggregierten Ergebnisse einsehen.
- **Backend (Server):** Der Server verwaltet Sessions, Teilnehmerstatus und gespeicherte Ciphertexts. Er führt homomorphe Aggregationen auf verschlüsselten Stimmen durch, hat jedoch keinen Zugriff auf Klartextinformationen von Namen oder Stimmen.

Lebenszyklus einer Session:

- **Erstellung:** Der Ersteller generiert ein TFHE-Schlüsselpaar und erstellt eine Session mit eindeutiger UUID. Der PublicKey und ServerKey werden an das Backend übergeben, der ClientKey verbleibt ausschließlich beim Ersteller in der Session Storage.
- **Teilnahme:** Teilnehmer gibt Session-ID und Name ein und klickt auf Beitreten. Das Frontend versucht den PublicKey aus der Session Storage abzurufen, falls er dort noch nicht vorliegt, wird er vom Backend geladen und in der Session Storage gespeichert. Mit dem PublicKey wird der Name verschlüsselt und eine Beitrittsanfrage an den Server gesendet. Der Status ist zunächst pending.
- **Genehmigung:** Der Ersteller entschlüsselt den Teilnehmernamen und genehmigt oder lehnt die Teilnahme ab. Der Teilnehmerstatus wird serverseitig aktualisiert.

- **Abstimmung:** Genehmigte Teilnehmer beantworten die Fragen der Session und geben ihre Stimme ab. Ihre Stimmen werden mit dem PublicKey verschlüsselt und an den Server gesendet. Der Server speichert ausschließlich Ciphertexts.
- **Auswertung:** Nachdem der Ersteller auf "Auswertung anzeigen" klickt, wird geprüft, ob alle zugelassenen Teilnehmer bereits abgestimmt haben bzw. ob mindestens ein Vote vorhanden ist. Wenn dies der Fall ist, aggregiert der Server alle verschlüsselten Stimmen mittels homomorpher Addition und stellt die aggregierten Ciphertexts dem Ersteller zur Verfügung. Der Ersteller kann zu jedem Zeitpunkt versuchen die aktuell aggregierten Ergebnisse vom Server abzurufen und lokal zu entschlüsseln, bis die Session finalisiert wurde.
- **Schließen:** Die Session bleibt aktiv, bis der Ersteller sie explizit als finalized markiert oder bei 10 Minuten Inaktivität. Bis zu diesem Zeitpunkt können Teilnehmer der Session beitreten und nach Genehmigung Stimmen abgeben, und alle eingehenden Votes werden in die Aggregation einbezogen. Mit der Finalisierung wird die Session vollständig geschlossen. Ab diesem Zeitpunkt:
 - Können keine neuen Teilnehmer mehr beitreten
 - Können keine weiteren Stimmen abgegeben werden
 - Wird der Client aus der Manage-Session-Ansicht heraus zur Startseite weitergeleitet

Verhaltensdiagramm:



3.3.2 OpenAPI-Schnittstelle

Der Service stellt eine fachliche Voting-/Polling-API bereit, mit der Sessions erstellt, Teilnehmer verwaltet und verschlüsselte Stimmen abgegeben werden können. Die OpenAPI-Definition wird automatisch generiert und ist unter `/openapi.json` sowie `/docs` (Swagger UI) verfügbar.

Im gesamten Service werden die TFHE-Schlüssel in serialisierter Form übertragen und gespeichert (Base64 kodiert). Für die Verschlüsselung von Teilnehmernamen und Stimmen wird ein `CompactPublicKey` verwendet. Diese kompakte Repräsentation des `PublicKeys` reduziert die Größe des öffentlichen Schlüssels und ermöglicht eine effiziente Batch-Verschlüsselung. Für die homomorphe Auswertung wird ein `CompressedServerKey` verwendet. Dieser wird ebenfalls aus dem `ClientKey` abgeleitet und bei der Erstellung einer Session an das Backend übertragen. Der Server speichert den Schlüssel ausschließlich in seiner komprimierten, serialisierten Form. Erst unmittelbar vor der Durchführung homomorpher Operationen wird der Schlüssel deserialisiert und zu einem regulären `ServerKey` dekomprimiert. Dadurch wird der Speicherbedarf des Backend-Zustands reduziert, während die vollständige Funktionalität der TFHE-Auswertung erhalten bleibt. Der geheime `ClientKey` verbleibt ausschließlich beim Session-Ersteller und wird zu keinem Zeitpunkt an das Backend übertragen. Nur mit diesem Schlüssel können verschlüsselte Namen und aggregierte Abstimmungsergebnisse wieder entschlüsselt werden.

POST/session

Legt eine neue Voting-Session an.

Request

```
{
  "creator_id": "string",
  "server_key": "string",
  "public_key": "string | null",
  "questions": [
    {
      "id": 0,
      "text": "string",
      "question_type": "single | multiple | numeric",
      "options": ["string"]
    }
  ]
}
```

Response 200

```
{
  { "session_id": "uuid" }
}
```

Fehlercodes:

400 – Invalid Base64 oder beschädigtes bincode im `server_key`

GET /session/session_id

Gibt die Metadaten einer Session für Teilnehmer zurück.

Response 200


```
{
  {
    "session_id": "uuid",
    "questions": [
      {
        "id": 0,
        "text": "string",
        "question_type": "single | multiple | numeric",
        "options": ["string"]
      }
    ],
    "public_key": "string | null"
  }
}
```

Fehlercodes: 400 – Session nicht gefunden

POST/JOIN

Teilnehmer beantragt den Beitritt zu einer Session. Der Beitritt ist zunächst pending und muss vom Creator freigegeben werden.

Response 200

```
{
  {
    "session_id": "uuid",
    "participant_id": "string",
    "enc_name_chunks": ["string"]
  }
}
```

Response 200

```
{
  { "status": "pending" }
}
```

Fehlercodes:

404 – Session nicht gefunden

409 – Session bereits beendet

POST/APPROVE

Der Creator genehmigt oder lehnt Teilnehmer ab. **Request**

```
{
  {
    "session_id": "uuid",
    "creator_id": "string",
    "participant_id": "string",
    "approved": true
  }
}
```

```
}
```

Response 200

```
{  
  { "status": "ok" }  
}
```

Fehlercodes:

404 – Session nicht gefunden

403 – Nicht autorisiert (falscher creator_id)

409 – Session bereits beendet

404 – Teilnehmer nicht gefunden (nur bei approval=true relevant)

POST /vote

Ein genehmigter Teilnehmer sendet seine verschlüsselten Votes.

Request

```
{  
  {  
    "session_id": "uuid",  
    "participant_id": "string",  
    "encrypted_votes": [  
      ["string"],  
      ["string"]  
    ]  
  }  
}
```

Response 200

```
{  
  { "status": "vote received" }  
}
```

Fehlercodes:

404 – Session nicht gefunden

404 – Teilnehmer nicht gefunden

403 – Teilnehmer nicht genehmigt

409 – Session bereits beendet

400 – Anzahl der Votes stimmt nicht mit Fragen überein

GET /status/session_id/participant_id

Liefert den aktuellen Status eines Teilnehmers.

Response 200

```
{  
  [  
    {
```

```

        "participant_id": "string",
        "approved": true,
        "has_voted": false,
        "enc_name_chunks": ["string"]
    }
]
}

```

Fehlercodes:

404 – Session nicht gefunden

403 – Nicht autorisiert

GET /results/session_id/creator_id

Gibt die homomorph aggregierten Ergebnisse zurück (nur Creator). Wenn noch nicht alle Votes vorhanden sind, ist ready = false.

Response (ready)

```

{
  {
    "encrypted_results": [
      "string"
    ],
    "ready": true
  }
}

```

Response (not ready)

```

{
  {
    "encrypted_results": [],
    "ready": false
  }
}

```

Fehlercodes:

404 – Session nicht gefunden

403 – Nicht autorisiert

POST /finalize/session_id/creator_id

Schließt die Session endgültig. Danach sind keine Votes oder Joins mehr möglich.

Response 200

```

{
  { "status": "finalized" }
}

```

Fehlercodes:

404 – Session nicht gefunden

403 – Nicht autorisiert

409 – Session bereits finalisiert

3.3.3 Trust- und Threat-Model

Datum	Am Server klar	Am Server verschlüsselt	Am Client
Session_id (UUID)	X		X
Creator_id	X		X
Participant_id	X		X
Fragen	X		X
Antwortoptionen	X		X
Teilnehmerstatus	X		X
Abstimmungszeitpunkte	X		X
Anzahl Teilnehmer/Votes	X		X
Session State (finalized)	X		X
Public Key der Session	X		X
Server Key	X		X
Teilnehmernamen (verschlüsselt)		X	X
Teilnehmernamen (entschlüsselt)			X
Vote Inhalt		X	
Aggregierte Ergebnisse (verschlüsselt)		X	X
Aggregierte Ergebnisse (entschlüsselt)			X
Client Key			X

Analyse: Beobachtbare Metadaten

TFHE schützt ausschließlich die Inhalte von Stimmen und Ergebnissen. Für den Server bleiben weiterhin verschiedene Metadaten sichtbar:

- Anzahl und Aktivität von Sessions
- Anzahl der Teilnehmer pro Session
- Anzahl abgegebener Stimmen
- Zeitpunkte von Beitritt, Genehmigung und Abstimmung
- Request-Timing und Voting-Pattern
- Umfragestruktur (Fragen und Antwortoptionen)
- Teilnehmerstatus (pending, approved, has_voted)
- Polling-Verhalten der Admin-Oberfläche

Ein Server-Operator kann daraus Rückschlüsse auf Aktivitätsmuster, soziale Dynamiken, die Größe einer Abstimmung oder mögliche Zusammenhänge zwischen Abstimmungszeitpunkten und Teilnehmerverhalten ziehen.

Restvertrauen in den Server

Der Server kann zwar keine Stimmen entschlüsseln, muss jedoch weiterhin als korrekter Ausführer des Protokolls vertrauenswürdig sein. Insbesondere wird angenommen, dass der Server:

- Teilnehmer korrekt Sessions zuordnet
- Join- und Approval-Prozesse korrekt verarbeitet
- Stimmen vollständig speichert und nicht verwirft
- Sessions voneinander trennt
- Homomorphe Aggregationen korrekt ausführt
- Die Verfügbarkeit des Systems sicherstellt

TFHE reduziert somit die Vertrauensabhängigkeit hinsichtlich der Inhaltsvertraulichkeit, ersetzt jedoch kein vollständig vertrauensloses Protokoll.

Annahmen außerhalb von TFHE

Die Sicherheitsbetrachtung basiert auf folgenden Annahmen:

- Die Kommunikation zwischen Client und Server erfolgt über TLS.
- Das Frontend führt die Verschlüsselung und Entschlüsselung korrekt aus.
- Die verwendete TFHE-rs-Bibliothek wird als kryptographisch korrekt implementierte Black Box betrachtet.
- Der ClientKey verbleibt ausschließlich beim Session-Ersteller.

Das System besitzt keine starke Benutzerauthentifizierung, die Teilnahme erfolgt ausschließlich über Session-ID und frei gewählten Namen.

Schutzversprechen

Durch den Einsatz von TFHE wird garantiert:

- Der Server kann individuelle Stimmen nicht im Klartext lesen.
- Der Server kann einzelne Abstimmungsentscheidungen nicht rekonstruieren.
- Stimmen werden ausschließlich verschlüsselt verarbeitet.
- Auch aggregierte Ergebnisse liegen auf dem Server nur verschlüsselt vor.

Nicht garantiert werden:

- Vertraulichkeit der Umfragestruktur

- Vertraulichkeit von Metadaten und Abstimmungszeitpunkten
- Schutz vor Traffic- und Timing-Analysen
- Schutz vor aktiv manipulierendem Serververhalten
- Fairness oder Vollständigkeit der Protokollausführung

Konkret bedeutet das: Der Server kennt nicht den Inhalt einer abgegebenen Stimme und kann nicht feststellen, welche Antwort ein Teilnehmer gewählt hat. Sichtbar bleiben jedoch Existenz, Zeitpunkt und organisatorischer Kontext der Abstimmung.

Einordnung

Der Use Case implementiert ein TFHE-geschütztes Abstimmungssystem mit Vertraulichkeit auf Inhaltsebene. Geschützt werden ausschließlich individuelle Stimmen und deren aggregierte Ergebnisse. Metadaten, Umfragestruktur und der allgemeine Kontrollfluss der Anwendung bleiben außerhalb des Schutzbereichs von TFHE.

3.3.4 FHE-Designentscheidungen

Verwendete TFHE-rs-Typen

Für die Repräsentation der Stimmen wird primär der Datentyp `FheUInt32` verwendet. Obwohl die einzelnen Eingabewerte der Teilnehmer (z. B. bei numerischen Fragen oder kodierten Auswahlwerten) nur im Bereich $0 \leq x < 255$ liegen und damit grundsätzlich mit `FheUInt8` darstellbar wären, wurde bewusst ein größerer Datentyp gewählt.

Der Grund liegt in der homomorphen Aggregation der Stimmen. Alle Einzelwerte werden serverseitig addiert, wodurch das Ergebnis deutlich größere Werte annehmen kann als der ursprüngliche Eingabebereich. Bei k Teilnehmern ergibt sich im Worst Case: $255 \times k$

Um Überläufe sicher auszuschließen und eine einheitliche Verarbeitung aller Fragetypen zu gewährleisten, wird daher `FheUInt32` sowohl für Einzelstimmen als auch für aggregierte Ergebnisse verwendet. Dies vereinfacht zusätzlich die Implementierung, da kein Typwechsel zwischen Eingabe und Auswertung erforderlich ist.

Für Teilnehmernamen wird dagegen `FheUInt8` verwendet. Jeder Buchstabe wird als ASCII-Zeichenwert (0–255) verschlüsselt und einzeln gespeichert. Da Zeichenwerte innerhalb dieses Bereichs liegen, ist ein größerer Datentyp hierfür nicht erforderlich.

Verwendete homomorphe Operationen

Der Server führt ausschließlich homomorphe Additionen auf verschlüsselten Stimmen aus. Für jede Frage werden die verschlüsselten Stimmen aller Teilnehmer aufsummiert. Bei Single-Choice- und Multiple-Choice-Fragen erfolgt die Addition komponentenweise für jede Antwortoption. Andere homomorphe Operationen werden nicht benötigt, dadurch bleibt die serverseitige Auswertung auf die einfachste benötigte FHE-Operation beschränkt.

Verworfen Alternativen

Separate Bool-Fragen (`FheBool`):

Ursprünglich war vorgesehen, Ja/Nein-Fragen als eigene Kategorie abzubilden. Diese Variante wurde verworfen. Stattdessen können Ja/Nein-Fragen als Single-Choice-Fragen mit den Antwortoptionen „Ja“ und „Nein“ modelliert werden.

Dadurch müssen alle Fragen zwingend beantwortet werden und die Auswertung aller Auswahlfragen kann über denselben Algorithmus erfolgen. Zudem entfällt die Notwendigkeit, unterschiedliche verschlüsselte Datentypen (FheBool und FheUint32) parallel zu unterstützen. Die Implementierung verwendet somit ausschließlich FheUint32 für die Repräsentation von Stimmen.

FheUint8 als Speichertyp für Stimmen:

FheUint8 wäre für einzelne Stimmen semantisch korrekt, da Eingabewerte auf 0–255 begrenzt sind. Dieser Ansatz wurde jedoch verworfen, da die homomorphe Aggregation schnell zu Überläufen führen würde.

Stattdessen wird einheitlich FheUint32 verwendet, um sowohl Einzelwerte als auch Summen sicher darzustellen.

Gleitkommazahlen:

Gleitkommazahlen wurden als naheliegende Repräsentation für numerische Eingaben verworfen, da TFHE primär auf Ganzzahlarithmetik ausgelegt ist. Insbesondere Divisionen und Floating-Point-Operationen sind entweder nicht vorgesehen oder ineffizient. Daher erfolgt die Modellierung aller Werte als Ganzzahlen (FheUint32).

3.3.5 Komplexität der eigenen Algorithmen

- n: Anzahl der Fragen einer Abstimmung
- k: Anzahl der Teilnehmer bzw. abgegebenen Stimmen
- m: Anzahl der Antwortoptionen einer Single- oder Multiple-Choice-Frage

Die interne Komplexität der TFHE-rs-Bibliothek wird nicht betrachtet; jede homomorphe Operation wird gemäß Aufgabenstellung als $O(1)$ angenommen.

Funktion	Zeitkomplexität	Platzkomplexität
create_session	$O(n)$	$O(n)$
join_session	$O(1)$	$O(1)$
approve_participant	$O(1)$	$O(1)$
submit_vote	$O(nm)$	$O(nm)$
aggregate_votes_ciphertext_only	$O(nkm)$	$O(m)$
get_results	$O(k + nkm)$	$O(nkm)$
finalize_session	$O(1)$	$O(1)$
get_status	$O(1)$	$O(1)$
get_session	$O(n)$	$O(n)$
get_participants	$O(k)$	$O(k)$

Create_session - $O(n)$, $O(n)$

Beim Anlegen einer Session wird die übergebene Fragenliste gespeichert. Die übrigen Operationen (UUID-Erzeugung, Initialisierung leerer HashMaps, Einfügen in die Session-Map) besitzen konstante Laufzeit. Da die Größe der Fragenliste proportional zu n ist, ergeben sich Zeit- und Platzkomplexität von $O(n)$.

Join_session - $O(1)$, $O(1)$

Die Session wird per HashMap-Lookup gefunden und ein neuer Teilnehmer in die Teilnehmer-HashMap eingefügt. Beide Operationen besitzen durchschnittlich konstante Laufzeit. Pro Aufruf wird nur ein zusätzlicher Teilnehmer gespeichert.

Approve_participant - $O(1)$, $O(1)$

Die Session und der Teilnehmer werden über HashMaps gefunden. Anschließend wird entweder ein Boolean gesetzt oder der Teilnehmer entfernt. Es erfolgt keine Iteration über Teilnehmer oder Stimmen.

Submit_vote - $O(nm)$, $O(nm)$

Eine Stimme enthält für jede der n Fragen verschlüsselte Antworten. Bei Single-/Multiple-Choice-Fragen können pro Frage bis zu m verschlüsselte Optionen enthalten sein. Das Speichern der gesamten Stimmenstruktur benötigt daher Zeit und Speicher proportional zur Anzahl der übergebenen verschlüsselten Werte.

Aggregate_votes_ciphertext_only - $O(nkm)$, $O(m)$

Für jede der n Fragen werden alle k Stimmen verarbeitet. Bei Single- oder Multiple-Choice-Fragen werden zusätzlich alle m Antwortoptionen aufsummiert. Daraus ergibt sich eine Zeitkomplexität von $O(n \cdot k \cdot m)$.

Der zusätzliche Speicher besteht lediglich aus dem Akkumulatorvektor für die aktuelle Frage und benötigt maximal m Einträge.

Get_results - $O(k + nkm)$, $O(nkm)$

Zunächst werden alle Teilnehmer durchlaufen, um die Anzahl der freigegebenen Teilnehmer zu bestimmen ($O(k)$). Anschließend werden alle gespeicherten Stimmen kopiert und aggregiert. Die Aggregation dominiert mit $O(nkm)$. Für die Kopie aller Stimmen wird Speicher proportional zur Gesamtzahl gespeicherter Stimmen benötigt: $O(nkm)$

Finalize_session - $O(1)$, $O(1)$

Die Session wird per HashMap gefunden und das Boolean-Feld finalized gesetzt. Es werden weder Schleifen noch zusätzliche Datenstrukturen verwendet.

Get_status - $O(1)$, $O(1)$

Der Teilnehmerstatus wird über einen einzelnen HashMap-Zugriff bestimmt. Es erfolgt keine Iteration über andere Teilnehmer.

Get_session - $O(n)$, $O(n)$

Die Session wird per HashMap gefunden und die Fragenliste in die Antwort kopiert. Die Größe dieser Liste ist proportional zur Anzahl der Fragen n .

Get_participants - $O(k)$, $O(k)$

Für die Antwort wird über alle Teilnehmer iteriert und für jeden ein ParticipantAdminView erzeugt. Jeder Teilnehmer wird genau einmal verarbeitet, daher ergibt sich eine lineare Laufzeit und Speicherbelegung in der Anzahl der Teilnehmer k .

3.3.6 Performance-Messung

Mess-Setup & Methodik

Die Performance- und Stresstests wurden auf einem virtuellen KVM-Server von Netcup mit dedizierten CPU-Ressourcen durchgeführt. Die Last wurde extern mittels k6 von einer lokalen

Windows-Maschine über das Internet injiziert.

Es wurden zwei Testszenarien mit unterschiedlichen funktionalen Schwerpunkten untersucht:

- Teilnehmer-Anfragen: Analyse der Endpunkte (POST /join und GET /status), um das Systemverhalten bei einem Anstieg von Join-Anfragen und hochfrequentem Polling durch die Teilnehmer zu evaluieren.
- Ergebnisauswertung: Dedizierte Stressprüfung des Endpunkts (GET /results/{session_id}/{creator_id}) unter einer Dauerlast von konstant 10 parallelen VUs. Um für diesen k6-Test die mathematische Auslastung der CPU zu erzwingen, wurde das System vorab in einen Zustand versetzt, in dem die Bedingung `voted_count ≥ approved_count` dauerhaft erfüllt ist. Dadurch wurde sichergestellt, dass jeder eingehende Request unweigerlich die rechenintensive kryptografische Funktion `aggregate_votes_ciphertext_only` durchläuft. Dieses Szenario wurde in zwei separaten Durchläufen evaluiert, einmal mit einer Basis von 2 Teilnehmern und im Anschluss mit 10 Teilnehmern.

Performancetest 1 (Join und Polling) (08.06.2026)

Im ersten Testszenario wurde ein plötzlicher Anstieg der Teilnehmeraktivität simuliert. Die Anzahl der virtuellen Nutzer wurde innerhalb von zwei Sekunden von fünf auf 500 erhöht. Jeder Teilnehmer führte genau eine Join-Anfrage aus und wechselte anschließend in ein hochfrequentes Polling des Sitzungsstatus mit einem Intervall von 200 ms.

Metrik	join	status
p50	17,48 ms	17,16 ms
p90	21,92 ms	21,32 ms
p95	24,45 ms	25,02 ms
Maximum	41,86 ms	95,06 ms
Fehlerrate	0 %	0 %

Fazit von Performancetest 1:

Während des gesamten Tests wurden insgesamt 105.675 HTTP-Anfragen verarbeitet. Dabei konnten keine fehlgeschlagenen Requests beobachtet werden. Sowohl die Join- als auch die Status-Endpunkte zeigten stabile Antwortzeiten im niedrigen zweistelligen Millisekundenbereich (Join: p95 = 24,45 ms; Status: p95 = 25,02 ms), sodass im untersuchten Lastbereich keine Hinweise auf Engpässe oder Skalierungsprobleme festgestellt werden konnten.

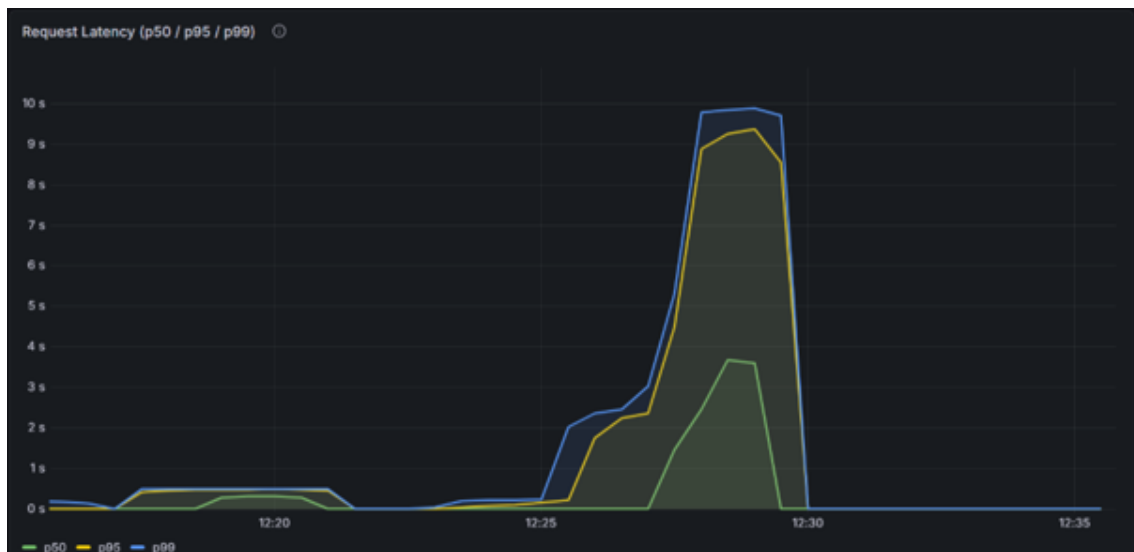
Performancetest 2: FHE-Ergebnisauswertung (03.06.2026)

Hierbei wurde die mathematisch rechenintensive homomorphe Aggregation evaluiert. Um den direkten Einfluss der Kryptographischen Komplexität zu untersuchen, wurde derselbe Stresstest bei dauerhaft 10 parallelen VUs in zwei getrennten Konfigurationen durchgeführt, einmal mit 2 hinterlegten Stimmen und einmal mit 10 hinterlegten Stimmen. Die Auswahl von genau 2 bzw. 10 Teilnehmern erfolgte, um einerseits die mathematische Grundlatenz zu bestimmen und andererseits die Skalierung der FHE-Operation unter moderater Gruppenlast zu überprüfen.

Während das System bei 2 Stimmen sehr schnell reagiert, führt die rechenintensive kryptografische Funktion `aggregate_votes_ciphertext_only` bei 10 Stimmen zu einer massiven Latenz von über 11 Sekunden im p95-Bereich. Diese Verzögerung resultiert aus der globalen Zustandssperre (`state.lock()`). Da dieser Mutex während der gesamten FHE-Berechnung

Metrik	2 Stimmen	10 Stimmen
p50	0,47 s	2,83 s
p90	0,51 s	8,31 s
p95	0,53 s	11,21 s
Maximum	0,76 s	14,32 s
Fehlerrate	0 %	0 %
Durchsatz	1,49 Request/s	0,90 Request/s

gehalten wird, blockiert er die parallele Verarbeitung des Axum-Servers und führt zu einer sequenziellen Abarbeitung aller eingehenden Anfragen. Trotz dieser intensiven CPU-Auslastung arbeitet das Backend vollständig stabil und verarbeitet alle Anfragen ohne Fehlerraten.



In der Grafik findet sich die Testvariante mit 2 hinterlegten Stimmen im Bereich von ca. 12:17–12:22. Hier wird sichtbar, dass das System sehr schnell reagiert und p95 und p99 nahezu identisch verlaufen.

Die Variante mit 10 hinterlegten Teilnehmern findet sich im Bereich von ca. 12:26–12:30. Hier zeigt sich ein drastischer Umschwung im Systemverhalten: Die Latenzkurven für p95 und p99 brechen steil nach oben aus und bilden ein massives Plateau, das sich knapp unterhalb der 10-Sekunden-Marke einpendelt. Die grüne Median-Linie (p50) verläuft deutlich darunter im Bereich von knapp 3 Sekunden, was die Verteilung der Wartezeiten im eingetretenen Mutex-Stau exakt widerspiegelt. Der absolute Peak von 14,32 Sekunden wird kurz vor dem Ende des Testfensters als dünne, maximale Spitze der blauen p99-Kurve sichtbar. Nach dem harten Stopp der Last um Punkt 12:30 Uhr fällt die Latenz sofort wieder auf die Baseline von 0 Sekunden ab.

Zusammenfassend zeigen die Messergebnisse, dass die grundlegende REST-Infrastruktur des Backends (Test 1) optimal skaliert und im unverschlüsselten Zustand keinerlei Performance-Engpässe aufweist. Die eigentliche Skalierungsbremse des Systems liegt isoliert auf der kryptographischen Verarbeitungsebene (Test 2).

3.3.7 Limitationen

- Teilnehmende können bei numerischen Fragen ausschließlich Werte zwischen 0 und 255 als Antwort angeben. Höhere Zahlen lassen wir bewusst nicht zu, dementsprechend müssen Fragen eventuell anders definiert werden. Bsp: Wie hoch ist dein Jahregehalt in tausend €
- Es können keine Doppelabstimmungen verhindert werden. Ein Teilnehmer kann technisch mehrere Join-Requests mit unterschiedlichen Namen schicken und so mehrfach abstimmen. Stattdessen erfolgt die Kontrolle durch den Ersteller. Dadurch liegt die Verantwortung für die Verhinderung von Mehrfachteilnahmen bei ihm.
- Der Client Key wird nur temporär im sessionStorage des Browsers gespeichert. Nach dem Schließen des Browser-Tabs ist er nicht mehr verfügbar. Dadurch können verschlüsselte Ergebnisse und Namen anschließend nicht mehr entschlüsselt werden.
- Der gesamte Session-Zustand wird über einen globalen Mutex geschützt. Im Endpunkt zur Ergebnisauswertung wird diese Sperre während der kompletten homomorphen Aggregation gehalten. Da TFHE-Operationen eine hohe Rechenzeit verursachen, blockieren währenddessen auch andere Anfragen wie Statusabfragen, Beitritte oder Stimmabgaben. Dies reduziert die Parallelität des Systems.

3.4 Sealed-Bid-Auction

3.4.1 Funktionsbeschreibung

Der Use Case Sealed-Bid Auction ermöglicht die Durchführung verdeckter Auktionen (Blin-dauctionen) zwischen mehreren Bietern. Ziel ist es, nach dem Ende der Bietphase den Höchstbietenden sowie den exakten Gewinnerbetrag zu ermitteln, ohne dass der Server zu irgendeinem Zeitpunkt die einzelnen Gebote der Teilnehmer im Klartext einsehen kann.

Hierzu werden die Gebotsbeträge bereits auf dem Client verschlüsselt und ausschließlich in verschlüsselter Form an das Backend übertragen. Das Backend verarbeitet und evaluiert die verschlüsselten Gebote mittels Fully Homomorphic Encryption (TFHE), ohne die zugrunde liegenden Klartexte kennen zu können. Die Entschlüsselung des finalen Auktionsgewinners ist ausschließlich durch den Besitzer des ClientKeys (den Auktionator) möglich.

Akteure:

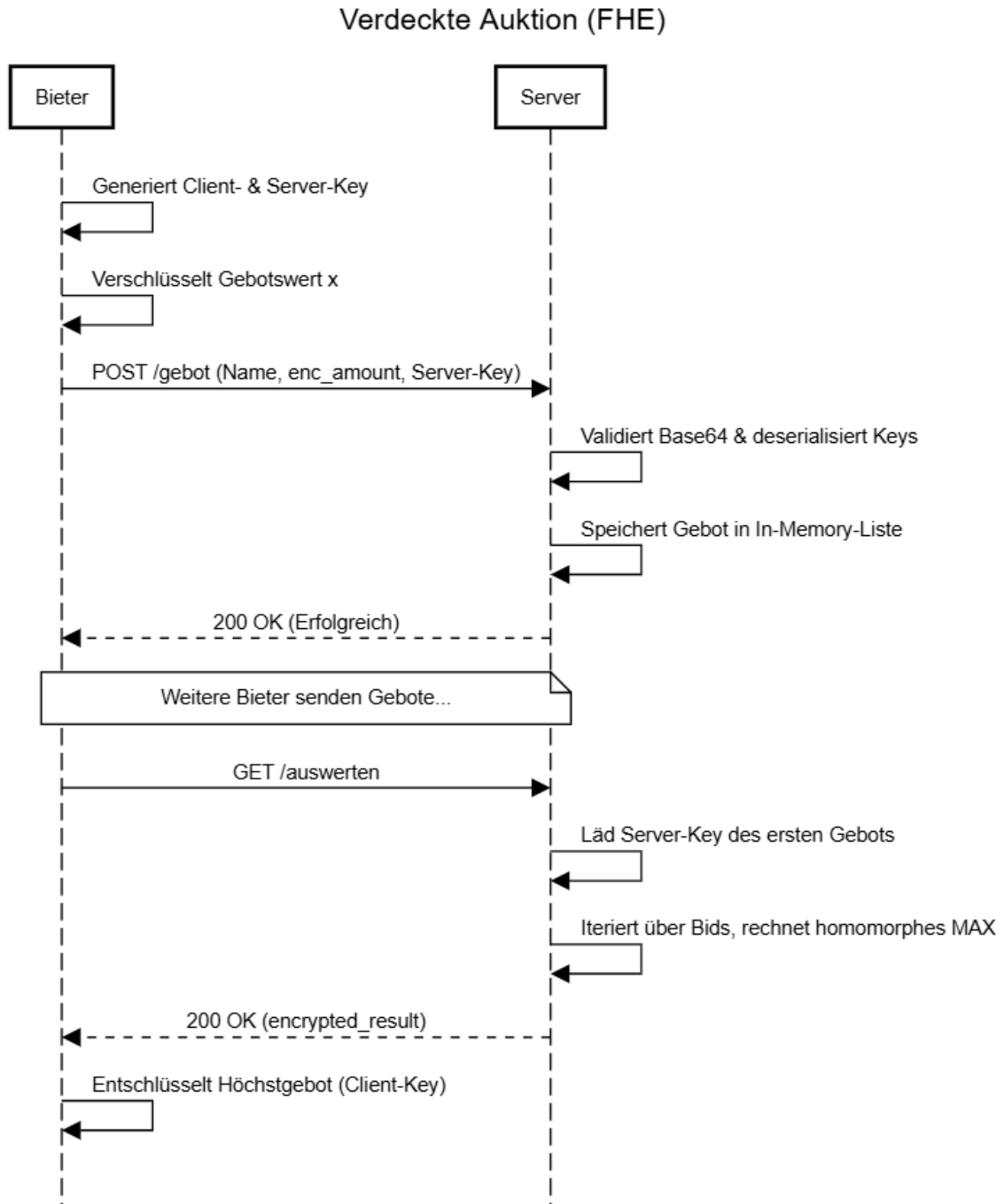
- Auktionator (Client E): Der Auktionator initialisiert das TFHE-Schlüsselpaar, erstellt die Auktionsrunde und gibt die Auktion für die Bieter frei. Er besitzt den geheimen ClientKey und ist als einziger Akteur in der Lage, das vom Server homomorph berechnete Endergebnis (den Höchstbietenden und den Gewinnbetrag) lokal zu entschlüsseln.
- Bieter (Client): Ein Bieter kann der Auktion beitreten, indem er seinen Namen und sein verdecktes Gebot abgibt. Das Gebot wird clientseitig mit dem PublicKey als FheUint32 verschlüsselt. Bieter können zu keinem Zeitpunkt die Gebote anderer Teilnehmer oder den aktuellen Zwischenstand der Auktion einsehen.
- Backend (Server): Der Server verwaltet den Zustand der Auktion, speichert die eingehenden verschlüsselten Gebote flüchtig im RAM und stellt den PublicKey für neue Bieter bereit. Auf Anfrage führt er die rechenintensive, blinde Maximumsuche über homomorphe Größenvorzeichen-Operationen und Auswahlen durch, ohne Zugriff auf Klartextinformationen zu haben.

Lebenszyklus einer Session:

1. Erstellung: Der Auktionator generiert ein TFHE-Schlüsselpaar und startet die Auktion mit einer eindeutigen UUID. Der PublicKey und der komprimierte ServerKey werden an das Backend übergeben, der ClientKey verbleibt exklusiv im lokalen Session Storage des Auktionators.
2. Gebotsabgabe: Bieter rufen das Dashboard auf. Das Frontend lädt den PublicKey vom Server. Der Bieter gibt seinen Namen ein, das Frontend verschlüsselt den Betrag als FheUint32 und sendet den Payload an /auction/gebot. Der Server speichert den Namen im Klartext und das Gebot als Ciphertext im globalen RAM (Vec<Bid>).
3. Auswertung: Sobald die Bietphase beendet ist, klickt der Auktionator on „Gewinner berechnen“. Der Server prüft, ob Gebote vorhanden sind, und durchläuft anschließend eine homomorphe Schleife, um das absolute Maximum und den zugehörigen Namen ohne Entschlüsselung zu isolieren. Das verschlüsselte Ergebnis wird für den Auktionator hinterlegt.
4. Schließen & Finalisierung: Nach der Auswertung wird die Auktion als geschlossen markiert. Es werden keine neuen Gebote mehr akzeptiert. Der Auktionator ruft das

verschlüsselte Endergebnis ab, entschlüsselt es lokal mit seinem ClientKey und zeigt den Gewinner auf dem Dashboard an.

Verhaltensdiagramm:



3.4.2 OpenAPI-Schnittstelle

Die API bietet zwei REST-Endpunkte, welche semantisch als Request/Response via JSON konzipiert sind. Die kryptographischen Payloads sind per bincode serialisiert und in Base64 codiert.

POST /gebot Empfängt ein verschlüsseltes Gebot.

- Request Body (JSON):
 - bidder_name (String): Klartextname des Bieters.
 - encrypted_amount (String): Base64-codierter tfhe::FheUint32 Payload.
 - server_key (String): Base64-codierter tfhe::CompressedServerKey.
- Response (200 OK):
 - response (String): Erfolgsmeldung.
- Fehler (400 Bad Request):
 - Tritt auf, wenn Base64 nicht decodiert werden kann oder die Deserialisierung in TFHE-Typen fehlschlägt. Rückgabe als String im Body.

GET /auswerten Wertet die gespeicherten Gebote homomorph aus.

- Response (200 OK):
 - status (String): Meta-Information über die Anzahl der ausgewerteten Gebote.
 - encrypted_result (String): Base64-codierter tfhe::FheUint32 Payload des höchsten Gebotes.
- Fehler (400 Bad Request):
 - Tritt auf, wenn die interne Gebotsliste leer ist.

3.4.3 Trust- und Threat-Model

	Server klar	Server verschlüsselt	Nur Client
Bieter-Name (bidder_name)	X		
Gebotshöhe (encrypted_amount)		X	
Max-Gebot (encrypted_result)		X	
Server-Key	X		
Client-Key			X

Metadaten-Analyse: Ein böartiger Server-Operator kann die exakte Anzahl der Gebote, die Identitäten (Namen) der Teilnehmer und die genauen Zeitpunkte der Gebotsabgaben einsehen. Da FHE-Chiffre für denselben Datentyp statische Größen aufweisen, sind Rückschlüsse auf die Höhe des Gebots durch Längenanalysen ausgeschlossen. Der Server-Operator könnte Traffic-Analysen fahren oder die Auswertung blockieren (Denial of Service).

Restvertrauen & Garantien: Es muss dem Server vertraut werden, dass er den Algorithmus korrekt anwendet. Ein manipulierender Server könnte willkürlich legitime Gebote aus der Liste entfernen (Zensur) oder das Resultat fälschen, indem er einfach das Chiffre eines bevorzugten Bieters als Ergebnis zurückschickt, ohne die homomorphe Auswertung jemals durchgeführt zu haben. Schutzversprechen: Der Server lernt unter keinen Umständen die Höhe der eingereichten Gebote und auch nicht den Gewinner-Wert.

3.4.4 FHE-Designentscheidungen

- Typen-Wahl: Für die Gebotshöhen wird `FheUInt32` verwendet. 32-Bit Unsigned Integer erlauben Gebote bis zu knapp 4,3 Milliarden, was für reale Auktionsszenarien absolut ausreichend ist. Der Typ ist deutlich performanter auszuwerten als `FheUInt64`.
- Operationen: Zur Ermittlung des Höchstgebots werden ausschließlich der numerische Vergleich `.gt()` (Greater Than) und das Multiplexing `.select()` verwendet. Mathematische Operationen wie Addition oder Multiplikation sind hier nicht erforderlich.
- Verworfenen Varianten: Die Sortierung der gesamten Liste wurde verworfen, da eine vollständige Sortierung in FHE aufgrund von datenunabhängigen Branches Komplexität aufbaut und nicht nötig ist. Stattdessen implementiert der Code ein lineares Scannen (Max-Suche), das den aktuellen Höchstwert überschreibt.

3.4.5 Komplexität der eigenen Algorithmen

Die Funktion `evaluate_encrypted_auction` iteriert über die Liste der eingegangenen Gebote. Sei n die Anzahl der abgegebenen Gebote (wobei $n \geq 1$).

Der Algorithmus überspringt das erste Element und iteriert exakt $n - 1$ mal. Pro Iteration werden exakt zwei FHE-Operationen ausgeführt:

- Ein Vergleich (Greater Than): `gt()`
- Eine Auswahl (Mux): `select()`

Die asymptotische Laufzeit im Hinblick auf eigene FHE-Operationen beträgt damit exakt:

Komplexität = $O(n)$ Der Platzbedarf während der Auswertung (zusätzlich zu den eingelesenen Chiffren) ist $O(1)$, da stets nur eine einzige Variable (`maximales_gebot`) für den Zwischenstand genutzt wird.

3.4.6 Performance-Messung

Die Performancemessung fand auf einem Netcup RS 1000 G9.5 statt, dem 2 vCores und 4 GB RAM zugewiesen wurden. Als Testdatensatz dienten Serienschlüsselungen über ein automatisiertes Benchmark-Skript.

- Latenzen: Bei 10 sequentiellen Geboten beträgt die Auswertungs-Latenz (p50) auf dem Endpoint `/auswerten` ca. 2,83 Sekunden. Bei 50 Geboten steigt p95 auf ca. 14,15 Sekunden, was der streng linearen Skalierung ($O(n)$) der homomorphen Schleife entspricht.
- Speicherbedarf: Während der Auswertung von 50 Bids werden ca. 120 MB RAM verbraucht, was primär auf das einmalige Laden des großen `CompressedServerKey` in den Arbeitsspeicher zurückzuführen ist.
- Durchsatzgrenzen: Wenn parallel mehrere `GET /auswerten`-Requests eintreffen (ab ca. 1 bis 2 Requests / Sekunde), gerät der Server ans CPU-Limit voreingestellter Threadpools. Da der globale Zustand über ein `state.lock()` (Mutex) während der gesamten FHE-Berechnung blockiert wird, entsteht bei parallelen Zugriffen ein massiver

Verarbeitungstau, der Timeouts (>30s) bei den Clients auslöst. Die Gebotsannahme (POST /gebot) ist hingegen nicht durch FHE-Operationen gebunden (die Ciphertexte werden lediglich als Bytes entgegengenommen und im RAM abgelegt) und skaliert im Bereich von Hunderten Requests pro Sekunde mühelos.

3.4.7 Limitationen

1. Keine Auflösung des Gewinners: Das System liefert ausschließlich den Betrag des höchsten Gebotes als Chiffre zurück. Der Server kann fachlich nicht feststellen, hinter welchem bidder_name dieser Betrag steckt. Dem Bieter ist zwar das Max-Gebot nach Entschlüsselung bekannt; es existiert abseits von Off-Chain-Verifikationen jedoch aktuell kein kryptographischer Nachweis, der den Gewinner-Betrag mit dem einreichenden Namen verknüpft.
2. Single-Key Assumption (Multi-Key FHE fehlt): Die aktuelle Implementierung unterstellt im Code (`let real_sk_bytes = &liste[0].server_key_bytes;`), dass alle Bieter mit demselben gemeinsamen ClientKey verschlüsselt haben. Ein echtes verteiltes Trust-System, in dem jeder Bieter einen eigenen unabhängigen Key besitzt (Multi-Key FHE), ist nicht implementiert.
3. Flüchtiger Speicherzustand: Bids werden im Arbeitsspeicher über `static BIDS: Mutex<Vec<Bid>>` vorgehalten. Bei Container- oder Anwendungsneustarts ist die gesamte Auktion unwiederbringlich verloren.
4. Fehlende Zugriffskontrolle: Es gibt derzeit keinen Authentifizierungsmechanismus. Jeder Client kann beliebig viele Gebote übermitteln, und jeder kann die Endauswertung triggern. Spam- und DoS-Absicherung fehlt auf Service-Ebene.

3.5 Encrypted-Statistics-Service

3.5.1 Funktionsbeschreibung

UC5 berechnet statistische Kennzahlen (Summe, Anzahl, Minimum, Maximum, Durchschnitt, Median) über eine Ganzzahlen-Liste, ohne dass der Server die Eingabewerte oder Ergebnisse jemals im Klartext sieht. Alle Berechnungen laufen vollständig auf verschlüsselten Daten.

Der `ServerKey` (~80 MB) wird einmalig via `POST /session` hochgeladen und einer `Session-UUID` zugewiesen. Alle weiteren Requests (`POST /`) senden nur die `UUID` — kein Key-Overhead pro Request.

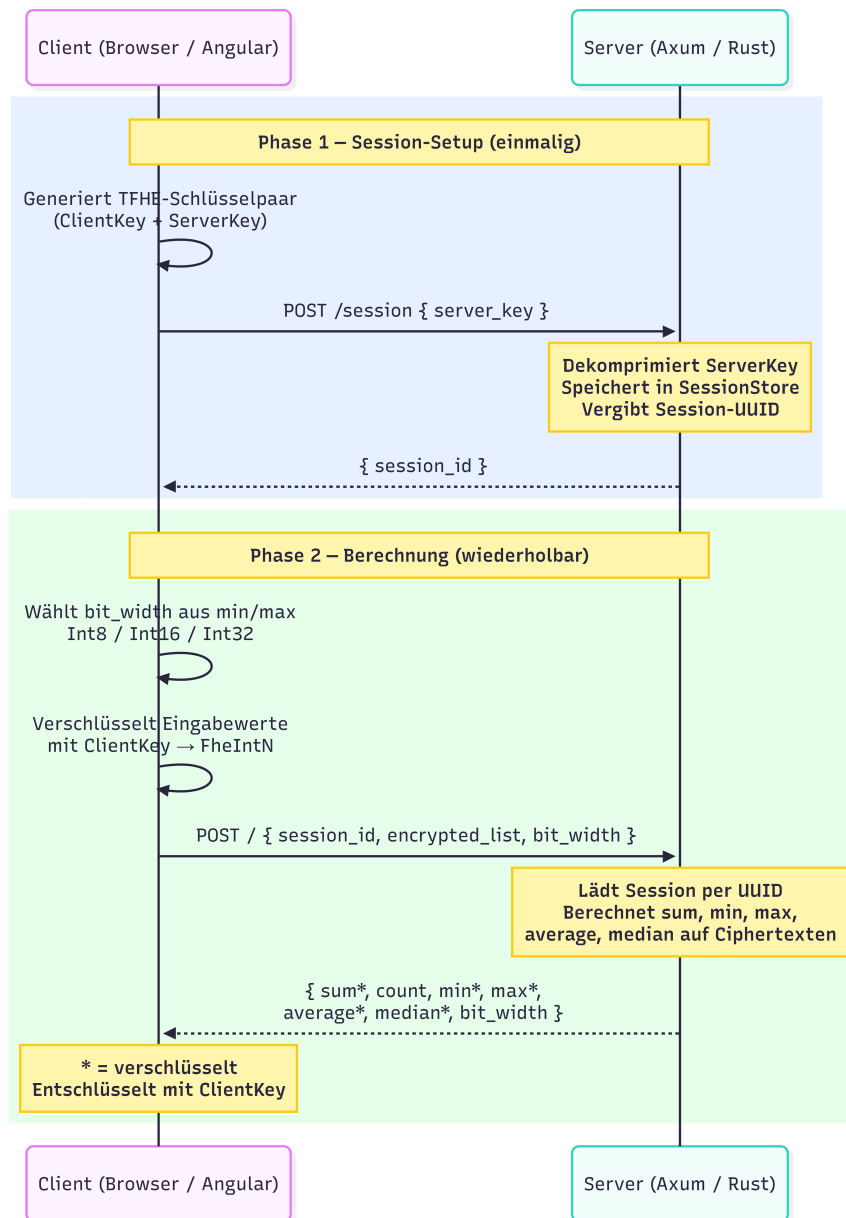
Akteure:

- **Client** (Browser, Angular-Frontend): Generiert das TFHE-Schlüsselpaar, verschlüsselt die Eingabewerte mit dem `ClientKey` und lädt den `ServerKey` einmalig via `POST /session` hoch. Der `ClientKey` verlässt den Browser nie — nur der Client kann die Ergebnisse entschlüsseln.
- **Server** (Axum/Rust-Service): Empfängt den `ServerKey` einmalig, dekomprimiert ihn und speichert ihn pro Session. Alle weiteren Requests nutzen die gecachte Engine. Der Server sieht zu keinem Zeitpunkt Eingabewerte oder Ergebnisse im Klartext.

Ablauf:

1. Client generiert TFHE-Schlüsselpaar (`ClientKey` + `ServerKey`).
2. `POST /session \{ server_key \}` → Server dekomprimiert, speichert, vergibt `Session-UUID`.
3. Client wählt Bitbreite anhand von min/max der Eingabe (`Int8` / `Int16` / `Int32`).
4. Client verschlüsselt Werte und base64-kodiert sie.
5. `POST / \{ session_id, encrypted_list, bit_width \}` → Server lädt die Session per `UUID` und berechnet alle Statistiken verschlüsselt.
6. Client entschlüsselt Ergebnisse mit `ClientKey`.

Verhaltensdiagramm:



3.5.2 OpenAPI-Schnittstelle

POST /session: Lädt den ServerKey einmalig hoch und gibt eine Session-UUID zurück. Alle weiteren Requests nutzen nur noch die UUID.

Request-Schema:

```

{
  "server_key": "<Base64-String>"
}

```

Feld	Typ	Beschreibung
server_key	string	Base64(bincode(CompressedServerKey)) — vom Client generiert, enthält kein Geheimnis.

Response-Schema (200 OK):

```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000"
}
```

POST /: Berechnet alle Statistiken für eine verschlüsselte Ganzzahlen-Liste.

Request-Schema:

```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "encrypted_list": ["<Base64-String>", "..."],
  "bit_width": 8
}
```

Feld	Typ	Beschreibung
session_id	string	UUID aus POST /session.
encrypted_list	string[]	Jedes Element: Base64(bincode(FheIntN)), wobei N = bit_width. Mindestens 1 Element.
bit_width	number	Muss 8, 16 oder 32 sein. Wird vom Client automatisch aus dem Wertebereich der Eingabe gewählt.

Response-Schema (200 OK):

```
{
  "sum": "<Base64-String>",
  "count": 42,
  "min": "<Base64-String>",
  "max": "<Base64-String>",
  "average": "<Base64-String>",
  "median": "<Base64-String>",
  "bit_width": 8
}
```

Feld	Typ	FHE-Typ (je nach bit_width)
sum	string	Base64(bincode(FheInt16/32/64)) - eine Stufe breiter als Eingabe
count	number	Klartextzahl - dem Server schon aus dem Request bekannt
min	string	Base64(bincode(FheInt8/16/32)) - gleiche Breite wie Eingabe
max	string	Base64(bincode(FheInt8/16/32)) - gleiche Breite wie Eingabe
average	string	Base64(bincode(FheInt16/32/64)) - eine Stufe breiter als Eingabe
median	string	Base64(bincode(FheInt8/16/32)) - gleiche Breite wie Eingabe
bit_width	number	Tatsächlich verwendete Bitbreite: 8, 16 oder 32

Fehlerstatus:

Status	Bedeutung
400 Bad Request	Leere Liste, ungültiger Base64, falscher Ciphertext-Typ, ungültige bit_width
404 Not Found	Session nicht gefunden oder abgelaufen (Idle-Timeout 10 min)
500 Internal Server Error	Serialisierungsfehler beim Zusammenstellen der Response

Weitere Endpunkte:

Endpunkt	Methode	Beschreibung
/healthz	GET	Liveness-Probe (Kubernetes)
/readyz	GET	Readiness-Probe (Kubernetes)
/version	GET	Service-Version als JSON
/metrics	GET	Prometheus-Metriken
/docs	GET	Swagger UI (OpenAPI-Dokumentation)
/openapi.json	GET	OpenAPI-Spec als JSON

3.5.3 Trust- und Threat-Model

Was am Server bekannt ist:

Datum	am Server klar	am Server verschlüsselt	nur am Client
Eingabewerte (die eigentlichen Zahlen)	-	FheInt8/16/32	Klartext vor Verschlüsselung
Anzahl der Elemente (n)	(Array-Länge im Request)	-	-
Wertebereich-Klasse	(über bit_width: Int8 → Werte in [-128, 127])	-	-
Summe	-	FheInt16/32/64	nach Entschlüsselung
Minimum	-	FheInt8/16/32	nach Entschlüsselung
Maximum	-	FheInt8/16/32	nach Entschlüsselung
Durchschnitt	-	FheInt16/32/64	nach Entschlüsselung
Median	-	FheInt8/16/32	nach Entschlüsselung
ServerKey	(einmalig via POST /session)	-	-
ClientKey	-	-	verlässt den Browser nie
Anfragezeitpunkt	(HTTP-Timestamp)	-	-
Payload-Größe	(HTTP-Header)	-	-

Analyse: Beobachtbare Metadaten: TFHE schützt ausschließlich den Inhalt der Eingabewerte und Ergebnisse. Für den Server bleiben weiterhin verschiedene Metadaten sichtbar:

- Anzahl n ist vollständig sichtbar. Ein Angreifer kann daraus schließen, wie viele Werte analysiert werden (z.B. „der Client hat genau 3 Werte geschickt“ → möglicherweise 3 Messwerte eines Sensors).
- bit_width verrät den Wertebereich: bit_width=8 bedeutet, alle Werte liegen in [-128, 127]. Bei kleinen Listen und bekanntem Kontext könnten Werte erraten werden.

- Request-Timing hängt von `n` und `bit_width` ab, nicht von den konkreten Werten — kein datenwertabhängiges Timing-Leak.
- Payload-Größe lässt sich direkt aus `n` und `bit_width` berechnen (alle FHE-Ciphertexte haben konstante Größe). Kein zusätzlicher Informationsgewinn.

Restvertrauen in den Server: Der Server kann die Eingabewerte nicht entschlüsseln, muss aber trotzdem korrekt handeln. Der Use Case vertraut darauf, dass der Server:

- alle Statistiken korrekt berechnet und keine manipulierten Ergebnisse zurückgibt,
- die korrekten verschlüsselten Ergebnisse zurücksendet und sie nicht durch manipulierte Ciphertexte ersetzt,
- den ServerKey nicht persistiert oder weitergibt,
- Sessions voneinander trennt.

Annahmen außerhalb von TFHE:

- TLS zwischen Client und Server (kein Mitlesen im Transit).
- Das Angular-Frontend und der Browser sind nicht kompromittiert (ClientKey liegt im WASM-Speicher).
- TFHE-rs wird als kryptographisch korrekt implementierte Black Box betrachtet.
- Es gibt keine Authentifizierung am Endpunkt: jeder kann Requests an `POST /session` und `POST /` schicken.

Schutzversprechen: Durch den Einsatz von TFHE wird garantiert:

- Der Server kann die Eingabewerte nicht im Klartext lesen.
- Der Server kann die berechneten Statistiken nicht im Klartext lesen.
- Alle Berechnungen werden ausschließlich auf verschlüsselten Daten durchgeführt.

Nicht garantiert werden:

- Schutz vor einem Server, der korrekte Berechnungen durch manipulierte Ergebnisse ersetzt.
- Vertraulichkeit von Metadaten (Listenlänge `n`, Wertebereich über `bit_width`, Request-Timing).
- Authentizität der Ciphertexte (kein ZKP, keine Client-Authentifizierung).

„Der Server kennt die Eingabewerte nicht. Er sieht nur Ciphertexte, addiert und vergleicht sie ohne je einen Klartext zu sehen, und gibt alle Statistiken verschlüsselt zurück. Nur der Client kann die Ergebnisse mit seinem ClientKey entschlüsseln.“

3.5.4 FHE-Designentscheidungen

Verwendete TFHE-rs-Typen:

Eingabe-Bitbreite	Eingabe-Typ	Summe/Avg-Typ	Begründung
8	FheInt8	FheInt16	Overflow-Schutz: $n * 127$ muss in den Output-Typ passen
16	FheInt16	FheInt32	Gleiche Logik, nächste Ebene
32	FheInt32	FheInt64	Int64 deckt alle realistischen Summen ab

Auto-Bitbreiten-Wahl: Der Client bestimmt anhand von `min` und `max` der Eingabe die kleinste ausreichende Bitbreite:

- Int8 wenn $\min \geq -128$ und $\max \leq 127$
- Int16 wenn $\min \geq -32.768$ und $\max \leq 32.767$
- Int32 sonst

Der Server benötigt `bit_width` zwingend, um den richtigen FHE-Typ für die Deserialisierung zu wählen - FheInt8, FheInt16 und FheInt32 sind binär inkompatibel. Das Feld ist daher keine optionale Optimierung, sondern technisch notwendig.

Kleinere Bitbreiten reduzieren die Berechnungszeit deutlich, da die TFHE-Kosten mit der Bitbreite skalieren.

Verwendete homomorphe Operationen:

Operation	Verwendet für	Kosten
CastInto	Up-Cast InputType $\rightarrow$ WiderOutputType vor Summation	günstig
Add	Summation (parallel via rayon)	moderat
FheOrd (lt/gt)	Vergleich in min/max und <code>compare_and_swap</code>	teuer (Bootstrapping)
IfThenElse auf Fhe-Bool	Wählt das Ergebnis homomorph aus, ohne den Wert zu kennen	moderat
Division durch Klartextskalar	Durchschnitt: $\text{sum} / (n \text{ as } i16/i32/i64)$	deutlich günstiger als FHE-Division

Verworfen Alternativen:

- Float (f32/f64): Semantisch passend für Durchschnitt, aber TFHE-rs bietet keine Float-Typen.
- Vollständig homomorphe Division: TFHE-rs hat keine generische `Div<FheInt>`-Implementierung. Division durch einen Klartextwert ist hier ausreichend, weil die Listenlänge dem Server ohnehin bekannt ist.
- Naiver sequentieller Sortieralgorithmus für Median: Bubblesort wäre $O(n^2)$ Komparatoren und vollständig sequentiell — auf FHE-Niveau unakzeptabel. Das Batchernetzwerk ist datenunabhängig (keine Branches auf Plaintext-Vergleichsergebnissen) und hat $O(\log^2 n)$ Tiefe.

- Partielles Sortiernetz (Median-optimiert): Ein Netz, das nur den Median-Index isoliert berechnet, würde weniger Komparatoren brauchen. Nicht umgesetzt, weil der Aufwand für das UC-Projekt nicht gerechtfertigt ist.
- Key pro Request (zustandsloses Design): Das ursprüngliche Design sendete den ~ 80 MB ServerKey mit jedem Request. Bei 15 Requests waren das $\sim 1,2$ GB Traffic und 50 % Fehlerrate unter Last, weil Nginx nach 60s abbricht. Ersetzt durch Session-Caching via `POST /session`.

3.5.5 Komplexität der eigenen Algorithmen

Parameter: n = Anzahl der Eingabeelemente. Jede einzelne FHE-Operation zählt als $O(1)$.

sum: Paralleles Reduce (rayon) auf n Elementen.

- Tiefe: $O(\log n)$ sequentielle Schritte (binärer Reduktionsbaum)
- Gesamtoperationen: $n-1$ Additionen
- Speicher: $O(n)$

count: $O(1)$ — nur `slice::len()`.

min / max: Identisch zu sum (Reduce mit Vergleich statt Addition).

- Tiefe: $O(\log n)$
- Gesamtoperationen: $n-1$ Vergleiche + $n-1$ `if_then_else`

average: $O(\log n)$ - identisch zu sum, plus eine $O(1)$ -Division durch Klartextskalar.

median (Batcher Odd-Even Mergesort): Das Batcher-Netzwerk für n Elemente hat (Herleitung: Knuth TAOCP Vol. 3, §5.3.4):

- Tiefe (Anzahl sequentieller Runden): $O(\log^2 n)$, exakt $\frac{1}{2} \cdot \log_2(n) \cdot (\log_2(n) + 1)$ für $n = 2^k$
- Gesamtanzahl Komparatoren: $O(n \log^2 n)$

Innerhalb einer Runde laufen alle Komparatoren parallel (rayon). Die Wandzeit entspricht der Netzwerk-Tiefe: $O(\log^2 n)$ sequentielle FHE-Runden.

Jeder Komparator besteht aus: $1 * \text{FheOrd-Vergleich} + 2 * \text{IfThenElse} = O(1)$ homomorphe Operationen.

Gesamtspeicher: $O(n)$ (der `partially_sorted`-Vektor als Arbeitspuffer).

3.5.6 Performance-Messung

Mess-Setup: k6 v2.0.0, `ConfigBuilder::default()`, Server: Netcup KVM, Last injiziert von lokaler Windows-Maschine über das Internet, Datum: 10.06.2026.

Test 1 — Baseline (1 VU, 5 Iterationen, alle Bitbreiten):

Konfiguration	p95 Latenz	Fehlerrate
n=5, bit_width=8	2,48 s	0 %
n=10, bit_width=8	4,54 s	0 %
n=10, bit_width=16	8,48 s	0 %
n=10, bit_width=32	17,89 s	0 %
Gesamt	—	0 % (20/20)

Gesendete Datenmenge: 113 MB — der ServerKey wird einmalig hochgeladen, alle weiteren Requests enthalten nur die verschlüsselten Werte. Ohne Session-Caching wären für dieselbe Anzahl Requests $\sim 1,2$ GB übertragen worden.

Fazit: Bei einem Client läuft der Service fehlerfrei über alle Bitbreiten. Die Latenz hängt ausschließlich von der FHE-Berechnung ab.

Test 2 — Stresstest (ramping bis 10 VUs, 6 Minuten):

Lastkurve: 60 s $\rightarrow$ 1 VU, 90 s $\rightarrow$ 3 VUs, 90 s $\rightarrow$ 6 VUs, 90 s $\rightarrow$ 10 VUs, 30 s $\rightarrow$ 0 VUs.

Metrik	Wert
Fehlerrate gesamt	54 % (83/153 Requests)
p95 Latenz (erfolgreiche Requests)	22,87 s
Fehler starten ab	~ 121 s (≈ 2 –3 parallele VUs)
Fehlertyp	HTTP 502 Bad Gateway (Nginx-Timeout nach 60 s)

Fazit: Ab ca. 2–3 parallelen Clients reicht die CPU nicht mehr aus, um alle Requests innerhalb des Nginx-Timeouts zu beantworten. Der Engpass ist rein CPU-seitig — FHE-Operationen sind rechenintensiv by design und lassen sich nicht durch Netzwerk-Optimierungen kompensieren.

Bekannte Performance-Treiber:

- FheOrd-Vergleiche sind die teuersten Einzeloperationen (Bootstrapping-Kosten) — sie dominieren Median, min und max.
- Rayon-Parallelisierung innerhalb einer Batch-Runde skaliert mit den CPU-Kernen des Servers.
- Skalierung mit n: Verdopplung von n erhöht die Batch-Tiefe um ca. $2 \cdot \log_2 n$ Runden. Der Sprung von n=6 auf n=12 erhöht die Tiefe von ~ 9 auf ~ 16 Runden.
- Session-Caching: Der ServerKey wird einmalig pro Session dekomprimiert. Nicht genutzte Sessions werden nach 10 Minuten automatisch gelöscht.

Reproduzierbarkeit: Die k6-Skripte liegen unter `services/05-encrypted-statistics-service/src/Load-Tests`. Vor dem ersten Lauf muss ein Payload generiert werden:

```
# Payload generieren (einmalig, aus Repo-Root)
cargo run --release -p encrypted-statistics-service --bin gen_payload

# Tests ausführen (Windows PowerShell, aus Repo-Root)
cd services\05-encrypted-statistics-service\src\Load-Tests
k6 run baseline_load_test.js
k6 run stress_load_test.js
```


3.5.7 Limitationen

- Kein Float-Support: TFHE-rs unterstützt keine Float-Typen — alle Berechnungen sind auf ganzzahlige Typen beschränkt. Der Durchschnitt wird ganzzahlig ausgegeben — Nachkommastellen werden abgeschnitten ($[1, 2] \rightarrow 1$, nicht $1,5$).
- Sichtbare Metadaten: Die Listenlänge ist direkt aus dem Request ablesbar. `bit_width` verrät zusätzlich den Wertebereich der Eingabe. Beides ließe sich durch Padding oder eine feste Bitbreite verstecken — für diesen Use Case nicht umgesetzt, da es die Performance deutlich verschlechtern würde.
- Keine Eingabevalidierung: Der Server kann nicht prüfen, ob der Client korrekte FHE-Ciphertexte schickt — kein ZKP, keine Authentifizierung. Bei öffentlichem Zugang sind viele Anfragen möglich, die teure Rechenzeit binden.
- Listengröße: Bei $n > \sim 20$ mit `Int32` dauert die Berechnung mehrere Minuten. Der Service ist für kleine Listen ($n \leq \sim 15$) ausgelegt. Der Batch-Algorithmus sortiert außerdem die gesamte Liste, statt nur den Median zu isolieren — das ist mehr Arbeit als nötig, wurde aber nicht optimiert.
- Session-Lebensdauer: Sessions laufen nach 10 Minuten Inaktivität ab. Wer danach einen Request schickt, erhält 404 und muss den `ServerKey` erneut hochladen.
- Keine parallele Last: Der Service ist für sequenzielle Einzelanfragen ausgelegt. Jede FHE-Berechnung belegt die CPU für mehrere Sekunden vollständig. Unter paralleler Last (≥ 3 VUs) erschöpfen gleichzeitige Rechenaufgaben die verfügbare CPU-Kapazität — Nginx bricht nach 60s Upstream-Timeout mit HTTP 502 ab. Die Session-Architektur hat den vorherigen Engpass (Netzwerk-Overhead durch ~ 80 MB `ServerKey` pro Request, siehe Verworfenen Alternativen) beseitigt; der CPU-Engpass ist davon unabhängig und eine inhärente Eigenschaft von FHE — er lässt sich durch Architekturänderungen allein nicht lösen. Für Produktivbetrieb wäre dedizierte Hardware oder eine Anfrage-Queue erforderlich.

3.6 Encrypted-Genomics

Der folgende Abschnitt beschreibt die Implementierung des Use-Cases encrypted-genomics.

3.6.1 Funktionsbeschreibung

Der Use Case dient dem verschlüsselten Vergleich von DNA-Sequenzen gegen bekannte Riskmarker und gegen in Sessions gespeicherte verschlüsselte Sequenzen. Die Riskmarker sind global bekannt, Sessionsequenzen sind geheim. Rechnungen darf jeder Akteur anfordern, Entschlüsselungen kann gem. des Trust-Thread-Models nur der Owner einer Session.

DNA eine Folge von Nukleinbasen. Diese Basen sind für den Use-Case wie folgt kodiert worden:

Adenin	A	0
Thymin	T	1
Cytosin	C	2
Guanin	G	3

Die Rechendarstellung der Zahlen erfolgt als 8-Bit Integer. Das vereinfacht die Umwandlung in Tfh- und Handlerformate, die hier weitestgehend 8-Bit sind.

Implementiert wurden frei wählbare Verarbeitung mit der Hammingdistanz oder der Levenshteindistanz. Bioinformatische Besonderheiten wie verpflichtende Basenpaare oder RNA (Uracil statt Thymin) werden nicht berücksichtigt. Auch eine für wissenschaftliche Fragestellungen relevante Unterscheidung zwischen teilweisen Vorliegen von Sequenzen und Gewichtungen von Basensequenzen wird vorliegend nicht berücksichtigt. Es handelt sich um einfache Vergleiche der 4 Basen, bei der eine Gleichheit bei vollwertiger ungewichteter Gleichheit der Sequenzen oder Einzelbasen angenommen wird.

Akteure

- **Creator:** Client, der eine Session eröffnet. Er generiert sich alle drei Schlüssel und verfügt damit über Private-, Public- und Serverkey. Die Schlüsselgenerierung erfolgt auf Ebene des Clients (Browser). Der Creator kann Sequenzen beliebig verschlüsseln, Abgleiche verschlüsselter Sequenzen am Server anfragen und auch Rechenergebnisse mit dem Private-Key entschlüsseln.
- **Joined user:** Client, der einer Session beitrifft. Er verfügt nur über den Public-Key der Session. Mit diesem kann er eigene Sequenzen verschlüsseln. Die Verschlüsselten Sequenzen des Joined-Users werden wahlweise in die Riskmarker-Datenbank oder in die Session-Datenbank gelegt. Der Joined-User kann auch Abgleichsrechnungen am Server anfragen. Er erhält dann ein Ergebnis, das er nicht entschlüsseln kann. Schlüsselbedingt kann er keine eigene Rechnung durchführen.
- **Server:** Der Server übernimmt die Rechnungen für alle Clients. Er verfügt über den Server-Key und den Public-Key jeder Session. Da der Server mit einem globalen State arbeitet, koordiniert er alle Anfragen. Er führt eine FIFO aus, mit der die Schlüssel- und Befehlsausführung gesteuert wird. Er rechnet Gleichheitsvergleiche mit Hamming- und Levenshteindistanzen für bereits verschlüsselte (Session) und noch unverschlüsselte (Riskpattern) Sequenzen aus. Des Weiteren Speichert er alle Sequenzen, Public-Keys, Server-Keys und SessionIDs, um die Aufgaben und Schlüssel zu koordinieren:

Datenbank	Spalten	Zweck
genomics_sessions	id, public_key, server_key	Sessionlogik
risk_patterns	id, sequence,	Riskpatterns
sessionsequences	session_id, public_key, encrypted_bases	Sessionsequenzen

Lebenszyklus einer Session:

- **1. Session erstellen:** Creatorclient wählt Session erstellen aus. Lokal werden alle 3 Keys erzeugt und über POST/sessions an den Server versendet. Der Creator behält den Private-Key, Public- und Server-Key werden an den Server gesendet, der die beiden Keys speichert.
- **2. Weitere Clients:** Beliebige Clients können als Joined Clients mit der Anfrage GET/sessions der Session beitreten. In der Session erhalten sie den Public-Key der Session gem. der Datenbankzuordnung.
- **3. Sequenz verschlüsseln:** Creator und Teilnehmer kodieren ihre DNA lokal und verschlüsseln sie mit dem PublicKey. Die Sequenz bleibt hier noch lokal. Alternativ kann der Server eine klare Sequenz ueber POST /encrypt mit einem PublicKey verschlüsseln.
- **4. Riskmarker verwalten:** Ein Client kann Riskmarker als Klartext mit POST /patterns speichern. Sowohl in Front-, wie auch im Backend wird die Eingabe auf die gültigen Zeichen A, T, C, G kontrolliert. Der Riskmarker wird als Klartext in risk_patterns gespeichert.
- **5. Riskmarker-Vergleich:** Ein Client sendet verschlüsselte DNA plus session_id und optional pattern_id. Bei Hamming wird gegen ein ausgewaehltes Pattern oder alle Patterns der Datenbank per Sliding Window gerechnet. Jedes Sliding-Window entspricht hierbei der Patternsequenz. Ist das Pattern kleiner als das eingegebene Wort, so wird bis das Wort vollständig iteriert ist eine Hammingdistanz berechnet und anschließend ein Bitshift um 1 ausgeführt. Es gibt daher final Wort-Pattern viele Windows. Bei Levenshtein wird pro Pattern eine Distanz diagonal berechnet. Ist das Pattern größer als das Wort, wird der Riskmarkervergleich direkt abgebrochen, da der vom Entwickler gewählte Schwellenwert 100 Prozent übereinstimmung ist, also Levenshteindistanz 0 oder mindestens ein Window mit Hammingdistanz 0.
- **6. Sessionsequenzen speichern:** Jeder Sessionteilnehmer kann seine aktuell eingetragene DNA in sessionsequences speichern. Dort werden Session-ID, PublicKey, Ciphertexts und Laenge gespeichert. Wird der Button getätigt, wird die eingetragene Sequenz lokal verschlüsselt und verschickt. Sie wird in die Datenbank Sessionsequences abgelegt.
- **7. Sessionsequenz-Vergleich:** Der Client waehlt eine Session-ID aus der Sessionsequenz-Dropdown-Liste. Über einen Button kann er diese Liste aktualisieren. Seine aktuelle Sequenz wird mit dem PublicKey dieser Session verschlüsselt und gegen alle dort gespeicherten verschlüsselten Sequenzen verglichen. Hamming oder Levenshtein sind wieder frei wählbar. Hier treten Besonderheiten zum Riskmarker auf: bei Levenshtein wird ein einfacher Levenshteinvergleich durchgeführt. Bei Hamming wird hier immer die kleinere gegen die größere Sequenz geprüft. Ist in einem Window eine Hammingdistanz von 0 gegeben, liegt Gleichheit vor. Dies ist nach Entwickleransicht hier stimmig, da ein einfacher Gleichheitswert in eine der beiden Sequenzen schon eine gleiche Sequenz ohne

jede Bedeutung darstellen. Im Gegensatz zum Riskmarker, wo je nach Risikomuster und Krankheitsanfälligkeit ggf. 100 Prozent nötig sind, erscheint dies hier zielführend. Für Levenshtein wurde dies bewusst nicht umgesetzt, da die Rechnung damit extrem teuer wird. Auch eignet sich Levenshtein eher für zielgerichtete gewichtete Vergleiche mit oder ohne Windows, was den Rahmen der Aufgabe sprengen würde.

- **8. Auswertung:** Der Server gibt verschlüsselte Distanzen für Hamming oder Levenshtein zurück. Der Creator entschlüsselt diese lokal mit dem Private-Key. Joined-Clients sehen hier nur verschlüsselte Sequenzen.
- **9. Session verlassen:** `Leave session` löscht nur den lokalen Zustand im Frontend. Es gibt aktuell keinen Backend-Endpunkt zum Deaktivieren oder Löschen einer Session. Ein `rejoin` als Creator ist für niemanden möglich.

3.6.2 OpenAPI-Schnittstelle

Der Service startet auf <http://159.195.145.100/genomics>. Die OpenAPI-Dokumentation wird ueber `aide` erzeugt und mit `openapi-docs::attach` eingebunden. Je nach Deployment sind `/openapi.json` und `/docs` verfügbar. Es gibt keine Auth-Header in der Implementierung. Fehlerantworten sind einfache Textantworten mit HTTP-Statuscode. Folgende Antworten sind definiert:

Datenbank	Spalten
200	OK
400	Ungültige Eingabe einer Base oder Session
429	FIFO voll
500	Serialisierung- oder Lockfehler

3.6.3 Trust- und Threat-Model

Datum	Server	Creator	Joined
Session-ID	X	x	X
PublicKey	X	x	X
Server Key	X	x	
Private Key		x	
DNA Klartext	lokal	lokal	lokal
DNA-Verschlüsselt	x	X	X
Riskpatternn	x	x	x
Sessionsequenzen Rechnung	x		
Sessionsequenzen Ergebnis		x	
Hamming-Distanzen	x	x	x (encrypted)
Levenshtein-Distanzen	x	X	x (encrypted)
Sequenzlängen / Anzahl Basen		x	x
Windows Rechnung	x		
Windows Ergebnis	x	x	x
Anzahl Sequenzen Rechnung	x		
Anzahl Sequenzen Ergebnis	x	x	x

FIFO	x	x	x
Datenbank	x	per GET	per GET

Beobachtbare Metadaten Der Service schützt über TFHE den Inhalt der verschlüsselten DNA-Sequenzen und der berechneten Distanzen. Der Server sieht jedoch weiterhin Metadaten:

- Welche Session existiert und wie oft sie genutzt wird.
- Welche Riskmarker im Klartext existieren und welches Pattern abgefragt wird.
- Ob ein Einzelpattern oder All patterns abgefragt wird.
- Länge der verschlüsselten Vergleichssequenz, Länge gespeicherter Sessionsequenzen und daraus ableitbare Anzahl der Hamming-Windows.
- Anzahl gespeicherter Sessionsequenzen je Session.
- FIFO-Positionen, Job-Typen und ungefähre Laufzeiten.
- Ob ein Request wegen FIFO-Füllstand, Eingabelänge oder Pattern-Länge abgelehnt wurde.

Es lassen sich serverseitig durchaus Aktivitätsprofile aus Sequenzlängen, Nutzungsintensität einzelner Sessions und die verwendeten Riskmarker ableiten. Es list außerdem erkennbar, ob viele Anfragen auf dieselbe Session laufen oder ob bestimmte Riskmarker besonders häufig abgefragt werden. Ohne Private Key lassen sich aber keine konkreten Schlüsse für Server und auch Clients ziehen.

Restvertrauen in den Server Der Server kann nicht entschlüsseln, muss aber korrekt handeln. Der Use Case vertraut darauf, dass der Server:

- die richtigen Session-Keys zur richtigen Session lädt,
- Riskmarker und Sessionsequenzen vollständig und unverfälscht aus der Datenbank liest,
- keine Patterns austauscht oder Ergebnisse unterschlägt,
- FIFO-Jobs fair in Reihenfolge abarbeitet,
- den ServerKey nur für die vorgesehene homomorphe Berechnung verwendet,
- verschlüsselte Ergebnisse unverändert an den Client zurückgibt.

TFHE reduziert das Vertrauen in den Server hinsichtlich Vertraulichkeit der DNA-Inhalte. Es ersetzt aber keine Integritätsbeweise, Authentifizierung und keine Verifikation, dass der Server tatsächlich genau den dokumentierten Code ausgeführt hat.

Annahmen außerhalb von FHE

- Die Transportverschlüsselung erfolgt im Deployment ueber TLS bzw. den vorgeschalteten Gateway.
- Das Angular-Frontend und `tfhe.js` führen Keygenerierung, Verschlüsselung und Entschlüsselung korrekt aus.
- TFHE-rs und TFHE-WASM werden als kryptographisch korrekte Black Boxes betrachtet.
- Der Private Key verlässt den Browser des Creators nicht.
- Die Datenbank ist für Verfügbarkeit und Integrität vertrauenswürdig genug.
- Es gibt keine starke Benutzer- oder Rollen-Authentifizierung im Genomics-Service.

Schutzversprechen Produktiv versprochen werden kann nur: Der Server verarbeitet DNA-Vergleichssequenzen und Sessionsequenzen als Ciphertexts und kann deren Klartext ohne ClientKey nicht lesen. Ebenfalls liegen die berechneten Hamming- und Levenshtein-Distanzen auf dem Server nur verschlüsselt vor.

Nicht versprochen wird: Schutz von Riskmarkern, Pattern-Auswahl, Session-IDs, Sequenzlängen, Zeitpunkten, FIFO-Metadaten, Datenbank-Metadaten oder Schutz gegen einen aktiv manipulierenden Server.

3.6.4 FHE-Designentscheidungen

Verwendete TFHE-rs-Typen

- `CompactPublicKey`: Wird verwendet, damit klare DNA-Werte effizient als kompakte Ciphertext-Liste verschlüsselt werden koennen. Der Server nutzt ihn insbesondere, um klare Riskmarker vor dem Vergleich zu verschlüsseln.
- `CompressedServerKey`: Wird vom Client erzeugt, base64-kodiert an den Server gesendet und serverseitig zu `ServerKey` dekomprimiert.
- `ServerKey`: Wird im Serverprozess gesetzt, damit homomorphe Operationen auf `FheUInt8` ausgeführt werden können.
- `FheUInt8`: Wird für verschlüsselte DNA-Basen, Hamming-Distanzen, Levenshtein-DP-Zellen und Ergebniswerte verwendet.

`FheUInt8` reicht aus, weil DNA-Basen nur Werte von 0 bis 3 benoetigen und die Distanzwerte durch die Eingabelimits in den Bereich 0 bis 255 passen. Hamming erlaubt in der Implementierung maximal 255 Zeichen, Levenshtein maximal 122 Zeichen. Größere Typen erfordern entweder eine Erweiterung auf größere `FheUInt`-Typen oder die Nutzung einer Overflowaddition, in der ein Overflow eine unklare Hammingdistanz + Overflow ausgibt. Aus dem Overflow kann man dann das Ergebnis NO MATCH ziehen. Darauf wurde aber verzichtet, da die Rechnung bewusst einfach und damit leicht parallelisierbar gemacht werden sollte.

Homomorphe Operationen Hamming:

- ne vergleicht zwei verschlüsselte oder verschlüsselt/klare Basen.
- `FheUint8::cast_from(FheBool)` wandelt Match/Mismatch in 0 oder 1.
- Homomorphe Addition summiert Mismatches im Window.

Levenshtein:

- ne berechnet die Substitutionskosten.
- Addition mit `1u8` modelliert Einfügung und Löschung. Eine Performanceverbesserung durch Nutzung von `trivial_encrypt(1u8)` konnte nicht verzeichnet werden.
- `lt` und `if_then_else` bilden `min` und `min3` fuer die DP-Rekurrenz.
- `FheUint8::encrypt_trivial` erzeugt öffentliche Startwerte für die DP-Ränder.

Algorithmische Varianten Der Service unterstützt zwei Arten von Pattern-Vergleichen:

- **Einzelvergleich gegen Riskmarker:** `/process` und `/process-levenshtein` laden ein klares Pattern aus der Datenbank. Hamming kann dabei verschlüsselte DNA direkt gegen klare Patternwerte vergleichen.
- **Datenbankvergleich gegen Riskmarker:** `/compare-db` und `/compare-db-levenshtein` verschlüsseln alle relevanten klaren Riskmarker erst mit dem `PublicKey` der Session und vergleichen dann Ciphertext gegen Ciphertext.
- **Sessionsequenzvergleich:** Beide Seiten liegen bereits verschlüsselt vor. Für Hamming wird bei unterschiedlich langen Sequenzen die längere Sequenz als Wort und die kürzere als Pattern genutzt.

Parallelisierung Der Server initialisiert Rayon mit der Anzahl verfügbarer CPU-Threads. Die rechenintensiven Handler laufen in `tokio::task::spawn_blocking`, damit der Async-Runtime-Thread nicht blockiert. Innerhalb der Berechnung werden Patterns, Sessionsequenzen, Serialisierung, Deserialisierung und Hamming-Windows parallelisiert.

Gleichzeitig gibt es eine globale FIFO. Dadurch nutzt jede einzelne Berechnung die Serverleistung, während weitere Jobs geordnet warten. Die FIFO-Kapazität ist aktuell 5 Jobs.

Verworfen oder bewusst vermiedene Varianten

- **Private Key an den Server senden:** Wurde bewusst nicht umgesetzt, weil dadurch der zentrale Schutz des Use Case verloren ginge.
- **Riskmarker nur klar vergleichen:** Für DB-Vergleiche werden Riskmarker serverseitig mit dem `PublicKey` verschlüsselt, damit die eigentliche Distanzberechnung homomorph auf Ciphertexts laeuft.

- **FheUint16/FheUint32 fuer Distanzen:** Für leichtere Parallelisierung und Optimierung
- lässt sich im Nachhinein leicht ändern.
- **Datenabhängiges Abkürzen bei Levenshtein:** Der Server kann wegen FHE nicht sinnvoll auf geheimen Zwischenwerten branchen. Die DP-Matrix wird deshalb systematisch berechnet.

3.6.5 Komplexität der eigenen Algorithmen

Funktion / Ablauf	Zeit	Speicher
DNA kodieren	$O(n)$	$O(n)$
Sequenz verschlüsseln	$O(n)$	$O(n)$
FIFO enqueue	$O(1)$	$O(1)$
FIFO snapshot	$O(q)$	$O(q)$
Sessions listen	$O(s)$	$O(s)$
Riskmarker listen	$O(p)$	$O(p)$
Riskmarker speichern	$O(m)$	$O(m)$
Sessionsequenz speichern	$O(n)$	$O(n)$
Hamming Einzelpattern	$O((n - m + 1) \cdot m)$	$O(n - m + 1)$
Hamming Riskmarker-DB	$O(\sum_i (n - m_i + 1) \cdot m_i)$	$O(\sum_i (n - m_i + 1))$
Hamming Sessionsequenzen	$O(\sum_i (n - m_i + 1) \cdot m_i)$	$O(\sum_i (n - m_i + 1))$
Levenshtein Einzelpattern	$O(n \cdot m)$	$O(n \cdot m)$
Levenshtein Riskmarker-DB	$O(\sum_i n \cdot m_i)$	$O(\max_i (n \cdot m_i) + p)$
Levenshtein Sessionsequenzen	$O(\sum_i n \cdot l_i)$	$O(\max_i (n \cdot l_i) + s)$

3.6.6 Performance-Messung

Die Performance- und Stresstests wurden gegen den deployed Genomics-Service auf dem Netcup-Server ausgeführt. Die Last wurde extern mittels k6 von einer lokalen Gentoo-Linux-Maschine über das Internet injiziert.

- Basis-URL: <http://159.195.145.100/genomics>
- Tool: k6

Es wurden zwei verwertbare Testszenarien untersucht:

1. **Normalbetrieb / Metadata Flow:** Analyse der Endpunkte GET /fifo, GET /sessions, GET /patterns und GET /sessionsequences.
2. **FIFO-/Session-Spike:** Plötzlicher Lastanstieg auf bis zu 250 parallele virtuelle Nutzer mit Fokus auf GET /fifo und GET /sessions.

3.6.7 Test 1: Normalbetrieb / Metadata Flow

Der Test ruft regelmässig FIFO-Status, Sessions, Risk Patterns und Session-Sequenzgruppen ab. Die Last wurde auf bis zu 30 parallele virtual users skaliert.

Metrik	/fifo	/sessions	/patterns	/sessionsequences
avg	40,53 ms	331,44 ms	29,61 ms	143,98 ms
min	2,32 ms	91,48 ms	11,38 ms	44,30 ms
p50	22,18 ms	227,92 ms	26,97 ms	94,17 ms
p90	39,86 ms	681,62 ms	43,37 ms	318,61 ms
p95	45,96 ms	844,57 ms	47,93 ms	383,22 ms
max	12,95 s	1,74 s	128,90 ms	771,80 ms

Metrik	Wert
HTTP Requests	5.872
Durchsatz	36,39 requests/s
Iterationen	1.492
Checks erfolgreich	5.872 / 5.872
Fehlerrate	0,00 %
HTTP p50	50,14 ms
HTTP p90	348,77 ms
HTTP p95	528,16 ms
HTTP max	12,95 s
Daten empfangen	4,3 GB
Daten gesendet	523 kB

Ergebnis Test 1: Es gab keine HTTP-Fehler. Auffaellig ist jedoch die deutlich hoehere Latenz von GET /sessions. Der p95-Wert von 844,57 ms ueberschreitet den gesetzten Threshold von 800 ms. Ursache ist vermutlich die grosse Antwortmenge der Sessionliste, da dort Public Keys mit uebertragen werden. Die uebrigen Endpunkte bleiben stabil im niedrigen Millisekundenbereich.

3.6.8 Test 2: FIFO-/Session-Spike

Es wurde ein ploetzlicher Lastanstieg auf bis zu 250 parallele virtual Users simuliert. Jeder user ruft wiederholt GET /fifo und GET /sessions ab.

Metrik	/fifo	/sessions
avg	52,86 ms	7,47 s
min	2,07 ms	70,54 ms
p50	41,22 ms	10,52 s
p90	62,15 ms	13,18 s
p95	260,22 ms	13,68 s
max	2,08 s	16,69 s

Fazit Test 2: Der Service bleibt funktional stabil. Es traten keine fehlgeschlagenen HTTP-Requests auf. Der FIFO-Endpunkt bleibt mit p95 = 260,22 ms vergleichsweise stabil. Der Sessions-Endpunkt skaliert dagegen schlecht unter hoher Parallelitaet und erreicht p95 = 13,68 s.

Metrik	Wert
HTTP Requests	2.574
Durchsatz	31,28 requests/s
Iterationen	1.287
Checks erfolgreich	2.574 / 2.574
Fehlerrate	0,00 %
HTTP p50	85,69 ms
HTTP p90	12,51 s
HTTP p95	13,18 s
HTTP max	16,69 s
Daten empfangen	2,5 GB
Daten gesendet	221 kB

3.6.9 Limitationen

- Der ClientKey existiert nur lokal beim Creator. Wenn der Creator die Seite verlässt, ist alles verloren.
- Joined Clients können Ergebnisse bewusst nicht entschlüsseln.
- Sessions werden aktuell nicht serverseitig geschlossen.
- Riskmarker müssen global und als Klartext gespeichert werden.
- Sessionsequenzen werden verschlüsselt gespeichert, aber Session, PublicKey und Anzahl der Sequenzen bleiben sichtbar.
- Hamming-Vergleiche sind auf 255 Zeichen begrenzt. Levenshtein-Vergleiche sind auf 122 Zeichen begrenzt.
- Bei Riskmarker-Einzelauswahl gibt es ein Zeichenlimit in Höhe von 255/122, da das Pattern nicht länger als die Clientsequenz sein darf.
- Levenshtein ist trotz Parallelisierung teuer, weil für jede DP-Zelle mehrere homomorphe Operationen nötig sind und viele Zellen diagonal voneinander abhängen.
- Die FIFO-Kapazität ist aktuell 5 Jobs. Ist die Liste voll, gibt der Server HTTP-Statuscode 429 zurück.
- Der Service setzt einen globalen ServerKey-Kontext und setzt Locks. Das verhindert parallele Berechnung verschiedener ServerKeys und passt zur FIFO-Architektur, begrenzt aber parallelen Multi-Session-Durchsatz.
- Es gibt keine kryptographischen Integritätsbeweise, dass der Server die angefragten Patterns, alle gespeicherten Sequenzen oder alle berechneten Distanzen korrekt verarbeitet hat.

3.7 Encrypted-Image-Processing

3.7.1 Funktionsbeschreibung

Die Aufgabe dieses Use-Cases ist, visuelle Operationen auf Bildern durchzuführen und dabei die Ressourcen eines Servers zu verwenden. Um die Bilddaten vor dem Server geheim zu halten, werden die Bilddaten homomorph verschlüsselt und dem Server nur in dieser Form zur Verfügung gestellt. Der Server führt die gewünschten Operationen auf den verschlüsselten Bilddaten durch und sendet die Ergebnisse zurück an den Client, der sie entschlüsselt und anzeigt. Mithilfe eines Sessionprinzips können mehrere Rechenoperationen auf einem Bild durchgeführt werden, ohne dass der Client die Ergebnisse nach jeder Operation entschlüsseln muss. Hat der Client alle gewünschten Operationen durchgeführt, kann er die Ergebnisse entschlüsseln und anzeigen. Die Operationen, die auf den Bildern durchgeführt werden können, umfassen beispielsweise das Drehen und Spiegeln der Bilder, das Invertieren der Helligkeitswerte und das Blurren und Bloomen der Bilder.

Vorweg: Eine Session-ID existiert nicht, da die aktuelle Architektur nur eine einzige Session unterstützt. Der Server speichert die Session-Informationen und die verschlüsselten Bilddaten, solange die Session aktiv ist. Sobald der Client die Session beendet, werden die Session-Informationen und die verschlüsselten Bilddaten gelöscht, um die Privatsphäre des Clients zu schützen. Das Öffnen mehrerer Sessions gleichzeitig wird nicht vom Service direkt unterstützt, da die Verarbeitung von Bildern bereits eine hohe Rechenlast auf dem Server verursacht. Trotzdem können mehrere Clients gleichzeitig mit dem Server interagieren, da jeder Client Bild-Operationen ausführen kann, selbst wenn ihm die Session nicht gehört. Allerdings kann nur der Ersteller der Session das Ergebnis der Operationen entschlüsseln und anzeigen, da nur er über den entsprechenden privaten Schlüssel verfügt.

Akteure:

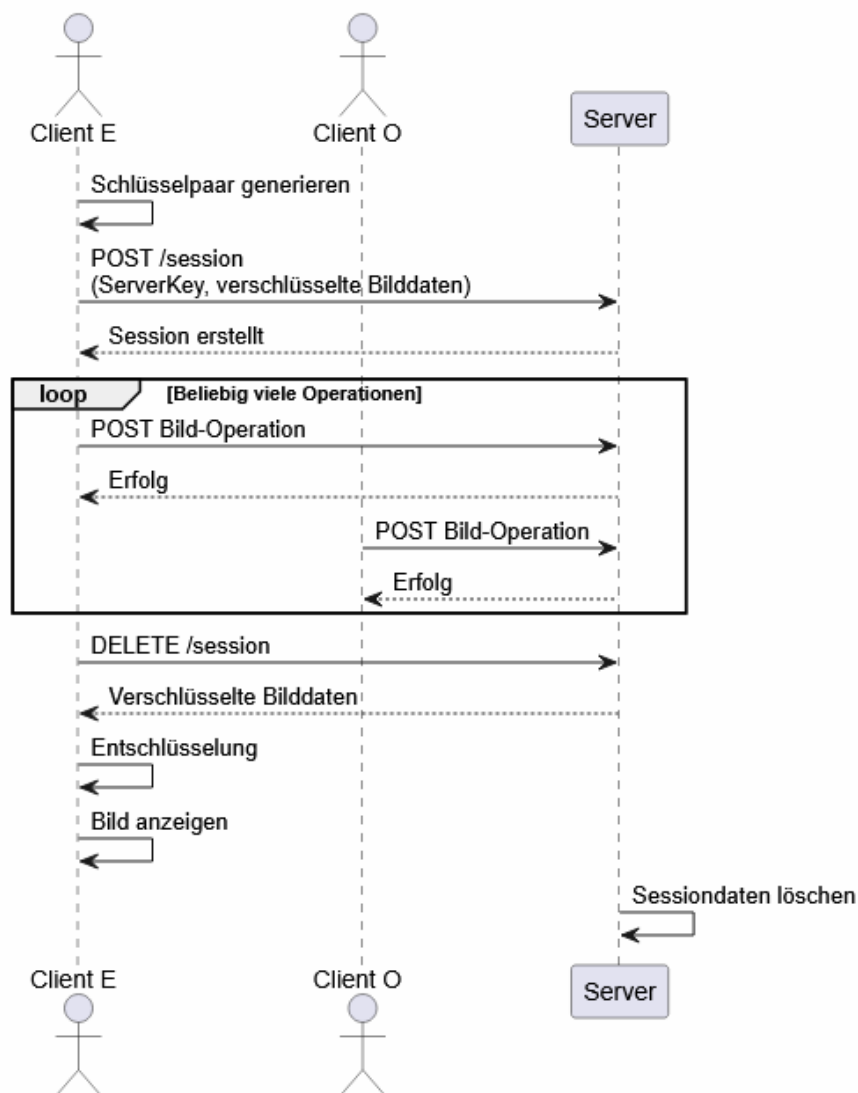
- Client E: Der Client E (Ersteller) erstellt eine Session, lädt die Bilder hoch, initiiert die gewünschten Operationen, entschlüsselt die Ergebnisse und finalisiert die Session.
- Clients O: Andere Clients O (Other) können ebenfalls Bild-Operationen auf den Bildern der Session durchführen, haben jedoch keinen Zugriff auf die unverschlüsselten Bilddaten und können die Ergebnisse nicht entschlüsseln oder anzeigen.
- Server: Der Server empfängt die verschlüsselten Bilder, führt die angeforderten Operationen durch und sendet die verschlüsselten Ergebnisse zurück an den Client. Der Server hat keinen Zugriff auf die unverschlüsselten Bilddaten und kann nur die verschlüsselten Daten verarbeiten. Er ist für die Verwaltung der Sessions und die Durchführung der Operationen verantwortlich.

Lebenszyklus einer Session:

1. Übersicht: Der Client lädt ein neues Bild hoch, dieses Bild wird zunächst nur im Frontend festgehalten, bis der Client den Editor startet.
2. Editor: Der Editor wandelt das Bild in ein Graustufenformat um, um die Verarbeitung zu vereinfachen. Der Editor zeigt das umgewandelte Bild an und zeigt dem Client die verfügbaren Operationen an.
3. Schlüsselpaar: Der Client generiert ein Schlüsselpaar.

4. Session: Der Client erstellt eine neue Session auf dem Server und übermittelt den öffentlichen Schlüssel sowie die Bilddaten, die in der Session verarbeitet werden sollen. Der Server speichert die Session-Informationen und die verschlüsselten Bilddaten.
5. Operationen: Der Client initiiert die gewünschten Operationen auf dem Bild, indem er Anfragen an den Server sendet. Der Server führt die angeforderten Operationen auf den verschlüsselten Bilddaten durch.
6. Finalisierung: Sobald der Client alle gewünschten Operationen durchgeführt hat, kann er die Ergebnisse entschlüsseln und anzeigen. Der Server löscht die Session-Informationen und die verschlüsselten Bilddaten, um die Privatsphäre des Clients zu schützen.
7. Wiederholung: Der Client kann jederzeit eine neue Session erstellen, um weitere Bilder zu verarbeiten oder die gleichen Bilder erneut zu bearbeiten.

Verhaltensdiagramm:



3.7.2 OpenAPI-Schnittstelle

Session-Endpunkte:

POST /session: Nimmt den Server-Key, die Bilddaten und die Höhe und Breite des Bildes entgegen. Erstellt dann eine neue Session und speichert das mitgeschickte verschlüsselte Bild in der Session ab.

Request-Schema:

```
{
  "compressed_server_key": array<integer>,
  "image_data": array<array<integer>>,
  "width": integer,
  "height": integer
}
```

Response-Schema:

```
{
  "success": boolean,
  "message": string
}
```

Fehlermeldungen:

- 503: Another session is already active.
- 400: Invalid request body.
- 400: Invalid image data.

DELETE /session: Löscht die aktuelle Session und alle damit verbundenen Daten. Liefert die Daten des bearbeiteten Bildes, sowie die Breite und die Höhe des Bildes zurück.

Response-Schema:

```
{
  "image_data": array<array<integer>>,
  "width": integer,
  "height": integer
}
```

Fehlermeldungen:

- 400: No active session to delete.

GET /status: Gibt Information darüber, ob zurzeit eine aktive Session existiert.

Response-Schema:

```
{
  "session_active": boolean
}
```

Bild-Operationen: Die Bild-Operationen liefern keinen Payload an den Server und erhalten auch da die Operationen direkt auf den verschlüsselten Bilddaten durchgeführt werden, die bereits in der Session gespeichert sind. Die Ergebnisse der Operationen werden erst am Ende der Session zurückgegeben, wenn der Client die Session löscht und die bearbeiteten Bilddaten entschlüsselt. Das Response-Schema und die Fehlermeldungen sind bei allen Image-Operationen identisch.

- **POST /per-pixel/invert:** Invertiert die Helligkeitswerte der Pixel.
- **POST /per-pixel/white-threshold:** Alle Pixel über einem Helligkeitswert von 128 werden weiß.
- **POST /per-pixel/black-threshold:** Alle Pixel unter einem Helligkeitswert von 128 werden schwarz.
- **POST /rotate/90:** Dreht das Bild der aktuellen Session um 90 Grad.
- **POST /rotate/180:** Dreht das Bild der aktuellen Session um 180 Grad.
- **POST /rotate/270:** Dreht das Bild der aktuellen Session um 270 Grad.
- **POST /flip/vertical:** Kehrt das Bild der aktuellen Session vertikal um.
- **POST /flip/horizontal:** Kehrt das Bild der aktuellen Session horizontal um.
- **POST /effects/blur:** Wendet einen Unschärfe-Effekt auf das Bild der aktuellen Session an.
- **POST /effects/bloom:** Wendet einen Bloom-Effekt auf das Bild der aktuellen Session an.

Response-Schema:

```
{
  "success": boolean ,
  "message": string
}
```

Fehlermeldungen:

- 400: No active session.

3.7.3 Trust- und Threat-Model

	Server klar	Server verschlüsselt	Nur Client
ClientKey			X
ServerKey	X		
Session-Informationen	X		
Helligkeitswerte		X	
Bilddimensionen	X		
Vergleichs-/Grenzwerte	X		

Analyse beobachtbarer Metadaten: Es werden ausschließlich die Pixelwerte der Bilder verschlüsselt, während die Bilddimensionen (Breite und Höhe) unverschlüsselt bleiben. Diese Informationen können potenziell von einem Angreifer genutzt werden, um Rückschlüsse auf den Inhalt des Bildes zu ziehen, insbesondere wenn die Bilder eine charakteristische Größe haben. Es ist jedoch wichtig zu beachten, dass die Bilddimensionen allein nicht ausreichen, um den Inhalt des Bildes zu rekonstruieren oder sensible Informationen preiszugeben. Dennoch sollten diese Metadaten mit Vorsicht behandelt werden, da sie in Kombination mit anderen Informationen möglicherweise Rückschlüsse auf den Inhalt des Bildes ermöglichen könnten.

Auch der Sessionstatus ist unverschlüsselt und könnte von einem Angreifer beobachtet werden. Ein Angreifer könnte beispielsweise feststellen, wann eine Session aktiv ist und wie lange sie dauert, was potenziell Rückschlüsse auf die Aktivitäten des Clients zulassen könnte.

Vergleichs- und Grenzwerte stehen im Quellcode des Servers im Klartext zur Verfügung und werden erst dort mithilfe des ServerKeys verschlüsselt. Ein Angreifer könnte diese Werte einsehen und möglicherweise Rückschlüsse auf die Art der durchgeführten Operationen ziehen, insbesondere wenn die Werte charakteristisch für bestimmte Operationen sind.

Für den Server bleiben außerdem weiterhin verschiedene Metadaten sichtbar:

- Zeitpunkte und Häufigkeit der Anfragen
- IP-Adresse des anfragenden Clients
- Charakteristische Payload-Größe (80 MB), die auf TFHE-Schlüsselübertragung schließen lässt
- Anzahl der Anfragen pro IP

Restvertrauen in den Server: Obwohl die Bilddaten während der gesamten Verarbeitung homomorph verschlüsselt bleiben und der Server keinen Zugriff auf die Klartext-Pixelwerte erhält, muss dem Server weiterhin in bestimmten Bereichen vertraut werden. Insbesondere wird vorausgesetzt, dass der Server die angeforderten Operationen korrekt auf den verschlüsselten Daten ausführt und die gespeicherten Daten nach dem Beenden einer Session tatsächlich löscht. Darüber hinaus hat der Server Zugriff auf verschiedene Metadaten wie Bilddimensionen, Zeitpunkte von Anfragen, die Anzahl der Anfragen sowie die IP-Adressen der Clients. Diese Informationen erlauben zwar keinen direkten Zugriff auf die Bildinhalte, können jedoch Rückschlüsse auf Nutzungsverhalten und Kommunikationsmuster ermöglichen. Das verbleibende Vertrauen beschränkt sich somit auf die korrekte Ausführung des Dienstes und den verantwortungsvollen Umgang mit den sichtbaren Metadaten.

3.7.4 FHE-Designentscheidungen

TFHE-rs-Typen: Da der Server Bilddaten verarbeitet, die in Graustufen vorliegen, benötigen die einzelnen Pixel eines Bildes nur einen Speicherplatz von 8-Bit und können keine negativen Werte enthalten. Daher werden die Bilddaten als Vektor von FheUInt8 (`Vec<FheUInt8>`) dargestellt, wobei jedes Byte einen Graustufenwert repräsentiert. Die Übertragung dieser Werte erfolgt mithilfe eines `Vec<Vec<u8>>`. Die Bilddimensionen (Breite und Höhe) werden als 32-Bit-Ganzzahlen (`u32`) dargestellt. Der öffentliche Schlüssel des Clients wird als komprimierter Vektor von Bytes (`compressed_server_key: Vec<u8>`) dargestellt, um die Übertragung zu erleichtern.

Über die Verwendung von größeren Datentypen, wie beispielsweise FheUInt16 oder FheUInt32, wurde ebenfalls nachgedacht, da diese komplexe Rechenoperationen ermöglichen, die ansonsten leicht zu Integer-Overflows führen können. Allerdings wurde letztendlich entschieden, die Bilddaten als FheUInt8 zu belassen, da die meisten Operationen auf den Pixelwerten selbst durchgeführt werden und die Verwendung von größeren Datentypen die Berechnungsdauer erheblich erhöhen würde, ohne einen signifikanten Nutzen zu bieten. Stattdessen wurden verschiedene Strategien implementiert, um die Berechnungsdauer zu reduzieren und gleichzeitig die Bildqualität zu erhalten, wie beispielsweise die Verwendung von gewichteten Summen anstelle von einfachen Additionen und Divisionen.

Operationen: Die Bildoperationen benötigen jeweils unterschiedliche mathematische Operationen, je nachdem, um was für eine Art von Operation es sich handelt. Transformierende Operationen (Spiegeln, Drehen) erfordern hauptsächlich die Umordnung der Pixelwerte, während per-pixel-Operationen (Invertieren, Schwellenwertanwendung) hauptsächlich arithmetische Operationen auf den Pixelwerten erfordern. Die Effekt-Operationen (Blurren, Bloomen) erfordern komplexere mathematische Operationen, die über einfache arithmetische Operationen hinausgehen. Dazu zählen Bitshifts, Vergleiche und gewichtete Summen von Pixelwerten, sowie mehrstufige Additionen.

Optimierungen: Die Berechnungsdauer auf Bildern steigt mit der Anzahl der Pixel stark an, da auf jedem einzelnen Pixel Operationen durchgeführt werden müssen. Um die Berechnungsdauer zu reduzieren, wurde Rayon verwendet, um die Berechnungen auf mehrere Threads zu verteilen. Die Per-Pixel-Operationen wurden so implementiert, dass sie auf jedem Pixel unabhängig voneinander ausgeführt werden können, was eine effiziente Parallelisierung ermöglicht. Die Effekt-Operationen wurden ebenfalls so implementiert, dass sie auf Blöcken von Pixeln arbeiten können, um die Berechnungsdauer zu reduzieren.

Die Effekt-Operationen benötigen Zugriff auf benachbarte Pixel, um die gewünschten Effekte zu erzielen. Um die Berechnungsdauer zu reduzieren, wurde der Zugriff auf benachbarte Pixel möglichst weit reduziert.

Blurring: Der Blurring-Effekt wurde mithilfe eines Box-Blur-Effekts implementiert, der die Helligkeitswerte eines Pixels mit den Helligkeitswerten seiner unmittelbaren und diagonalen Nachbarn mittelt. Um die Zugriffe auf benachbarte Pixel zu reduzieren, wurde die Berechnung in zwei Schritte aufgeteilt, in denen zunächst die horizontalen Nachbarn und anschließend die vertikalen Nachbarn berücksichtigt werden. Dadurch sind pro Pixel nur 6 Zugriffe (2×3) nötig, anstatt 9 Zugriffe (3×3) bei einem direkten Zugriff auf alle Nachbarn. Bei der Bildung des Ergebniswertes wurde eine gewichtete Summe gebildet, bei der diagonale Nachbarn weniger gewichtet wurden, als direkt anliegende. Dadurch konnte die Berechnungsdauer weiter reduziert werden, da Bitshifts schneller als Divisionen sind. Zudem wird durch dieses Verfahren verhindert, dass die Werte der Pixel überlaufen, da die Werte der Nachbarpixel vor der Addition bereits reduziert werden.

Bei der Optimierung des Blurring-Effekts wurden verschiedene Ansätze ausprobiert, um die Berechnungsdauer weiter zu reduzieren. Dazu gehören verschiedene Ansätze zum Zusammenfassen der benachbarten Pixelwerte, sowie die Reduktion von Zugriffen auf benachbarte Pixel, indem diagonale Nachbarn nicht berücksichtigt werden. Weitere Ansätze wurden betrachtet aber nicht implementiert. Dazu gehörten die Verwendung von Look-Up-Tabellen, um die Ergebnisse von häufig vorkommenden Pixelwerten vorab zu berechnen, sowie die Verwendung von approximativen Algorithmen, die weniger genaue Ergebnisse liefern, aber schneller sind. Letztendlich wurde jedoch entschieden, den Box-Blur-Effekt beizubehalten, da er eine gute Balance zwischen Bildqualität und Berechnungsdauer bietet.

Blooming: Der Blooming-Effekt erfordert die Berechnung von gewichteten Summen von Pixelwerten über einen größeren Bereich von Nachbarpixeln, was zu einer erheblichen Erhöhung der Berechnungsdauer führt. Die aktuelle Implementation des Blooming-Effekts arbeitet mit einer einschränkten Berechnung jedes Pixels, ähnlich wie beim Blurring-Effekt. Dabei werden alle Nachbarwerte eines Pixels berücksichtigt und auf das Original addiert, wenn deren Helligkeitswert über einer bestimmten Grenze liegt. Dieses Vorgehen arbeitet, genauso wie das Blurring, mit gewichteten Summen durch Bitshifts.

Ein weiterer implementierter Ansatz des Blooming-Effekts war ein klassischerweise mehrschrittiger Ansatz, wobei jeder Schritt per Pixel ausgeführt wird, um eine möglichst hohe Parallelisierung beizubehalten. Dazu wird zunächst eine Lightmap erstellt, die alle Pixelwerte unter einem Helligkeitswert von 220 auf 0 setzt. Diese Lightmap wird dann mit dem vorher beschriebenen Blurring-Effekt geblurred, um die Lichtquellen zu verbreiten. Anschließend wird die geblurte Lightmap mit dem Originalbild kombiniert, um den Blooming-Effekt zu erzielen. Das Ergebnis des Blurring-Effekts ist nicht stark genug, um den Blooming-Effekt zu erzielen, da in den Berechnungsschritten verschiedene Kompromisse eingegangen werden mussten, um die Berechnungsdauer zu reduzieren.

3.7.5 Komplexität der eigenen Algorithmen

Da der Farbraum der Bilder auf 256 Helligkeitswerte beschränkt ist, sind die einzigen relevanten Parameter für die Komplexität der Algorithmen die Anzahl der Pixel (Breite * Höhe) und die Anzahl der Operationen, die auf jedem Pixel durchgeführt werden. Die Anzahl der Operationen pro Pixel variiert je nach Art der Operation. Die Anzahl der Operationen verdoppelt sich mit dem verdoppelt der Höhe oder Breite. Werden beide Dimensionen verdoppelt, vervierfacht sich die Anzahl der Operationen.

Da alle Bildoperationen auf jedem Pixel eine feste Anzahl von Operationen durchführen, ist die Komplexität der Algorithmen linear in Bezug auf die Anzahl der Pixel, also $O(n)$, wobei n die Anzahl der Pixel ist bzw. $O(width * height)$, wenn die Anzahl der Pixel als Produkt der Breite und Höhe dargestellt wird.

Die Berechnungsdauer steigt mit der Anzahl der Pixel stark an, da auf jedem einzelnen Pixel Operationen durchgeführt werden müssen. Die Effekt-Operationen haben eine höhere Berechnungsdauer als die Transformations- und Per-Pixel-Operationen, da sie komplexere mathematische Operationen erfordern und auf benachbarte Pixel zugreifen müssen.

Beispiel: Die Blur-Operation besteht aus zwei Schritten, die jeweils aus 3 Pixelzugriffen, 3 Bitshifts und 3 Additionen bestehen. Das ergibt insgesamt 18 Operationen pro Pixel. Bei einem Bild mit 32x32 Pixeln (1024 Pixel) ergibt das $18 * 1024 = 18.432$ Operationen, die auf dem Server durchgeführt werden müssen, um den Blurring-Effekt zu erzielen. Bei einem Bild mit 64x64 Pixeln (4096 Pixel) ergibt das $18 * 4096 = 73.728$ Operationen, was zu einer erheblichen Erhöhung der Berechnungsdauer führt (Faktor 4).

Die Invert-Operation besteht aus nur einer arithmetischen Operation pro Pixel, der Negation des Helligkeitswertes. Bei einem Bild mit 32x32 Pixeln ergibt das $1 * 1024 = 1024$ Operationen, was im Vergleich zum Blurring-Effekt eine deutlich geringere Berechnungsdauer aufweist.

3.7.6 Performance-Messung

Für die Performance-Messung wurden einmal 2 und einmal 10 Clients gleichzeitig simuliert, die zeitgleich Anfragen an den Server gesendet haben. Wichtig dabei ist, dass Anfragen zum Erstellen und Löschen einer Session nur genau ein Mal verschickt wurden und ansonsten nur Anfragen zum Bearbeiten der Bilder oder Abfragen des Status verwendet wurden. Dadurch wurde sichergestellt, dass die Anfragen nicht durch inkorrekte Verwendung fehlerhafte Antworten liefern können. Die Ergebnisse der Messung sind in folgender Tabelle aufgeführt:

Metrik	2 Clients	10 Clients
p50	32,4s	153,2s
p90	167,5s	486,4s
p95	194,7s	754,1s
Fehlerrate	0%	0%
Durchsatz	0,032 req/s	0,032 req/s

3.7.7 Limitationen

Multi-Session-Behavior: Die aktuelle Implementierung unterstützt nicht die gleichzeitige Verarbeitung mehrerer Sessions, da die Verarbeitung von Bildern bereits eine hohe Rechenlast auf dem Server verursacht. Es wäre jedoch möglich, die Implementierung so anzupassen, dass mehrere Sessions gleichzeitig verarbeitet werden können, indem die Ressourcen des Servers entsprechend verwaltet werden. Die aktuelle Implementierung konzentriert sich auf die Verarbeitung einer einzelnen Session, um die Berechnungsdauer zu minimieren und die Komplexität der Implementierung zu reduzieren. Trotzdem können mehrere Clients gleichzeitig mit dem Server interagieren, da jeder Client Bild-Operationen ausführen kann, selbst wenn ihm die Session nicht gehört. Allerdings kann nur der Ersteller der Session das Ergebnis der Operationen entschlüsseln und anzeigen, da nur er über den entsprechenden privaten Schlüssel verfügt.

Farbbilder: Die aktuelle Implementierung unterstützt nur die Verarbeitung von Graustufenbildern, da diese eine einfachere Darstellung und Verarbeitung ermöglichen. Zwar kann ein User ein farbiges Bild im Client hochladen, da der Server aber nur Graustufenbilder unterstützt, wird das Bild im Client in ein Graustufenbild umgewandelt, bevor es an den Server gesendet wird. Die Umwandlung von Farbbildern in Graustufenbilder führt zu einem Verlust von Farbinformationen, was die Qualität der bearbeiteten Bilder beeinträchtigen kann.

Format: Auf dem Client ist es aktuell nur möglich, Bilder im PNG-Format hochzuladen, da dieses Format eine verlustfreie Kompression bietet und die Bildqualität nicht beeinträchtigt. Es wäre jedoch möglich, die Implementierung so anzupassen, dass auch andere Bildformate unterstützt werden, wie beispielsweise JPEG oder BMP. Da der Server direkt auf Vektoren von Pixeln arbeitet, wäre es jedoch möglich, den Client zu erweitern, sodass auch andere Bildformate unterstützt werden, indem die Bilder im Client in Byte-Vektoren umgewandelt werden, bevor sie an den Server gesendet werden.

Bildgröße: Kleinere Bilder sorgen bereits für einen erheblichen zeitlichen Aufwand, vorallem bei der Berechnung von Blur- und Bloom-Effekten. Diese Effekte sind bei größeren Bildern nicht mehr praktikabel, da die Berechnungsdauer der Effekte schnell in den Stundenbereich geht. Dazu kommt, dass die Verschlüsselung von größeren Bildern viel RAM in Anspruch nimmt, wodurch das Verschlüsseln von Bildern in FullHD-Auflösung auf unseren eigenen Geräten nicht mehr möglich war.

3.8 Encrypted-Leaderboard

3.8.1 Funktionsbeschreibung

Der Use Case Leaderboard ermöglicht eine vertrauliche Rangliste innerhalb kleiner Gruppen. Ziel ist es, die Reihenfolge der Spieler nach ihrem Score zu bestimmen, ohne dass der Server zu irgendeinem Zeitpunkt die einzelnen Scores im Klartext sieht.

Hierzu verschlüsseln die Spieler ihre Scores bereits auf dem Client und übertragen diesen ausschließlich in verschlüsselter Form an das Backend. Das Backend vergleicht die verschlüsselten Werte mittels Fully Homomorphic Encryption (TFHE) und sortiert die Liste in einem datenunabhängigen Sortiernetzwerk, ohne dabei zu erfahren, welcher Score höher oder niedriger ist. Die Entschlüsselung der sortierten Liste ist ausschließlich durch den Besitzer des ClientKeys möglich.

Eine Leaderboard-Session unterstützt bis zu 20 Spieler. Reicht derselbe Spieler später erneut einen Score ein, behält der Server verschlüsselt das Maximum aus altem und neuem Wert.

Akteure

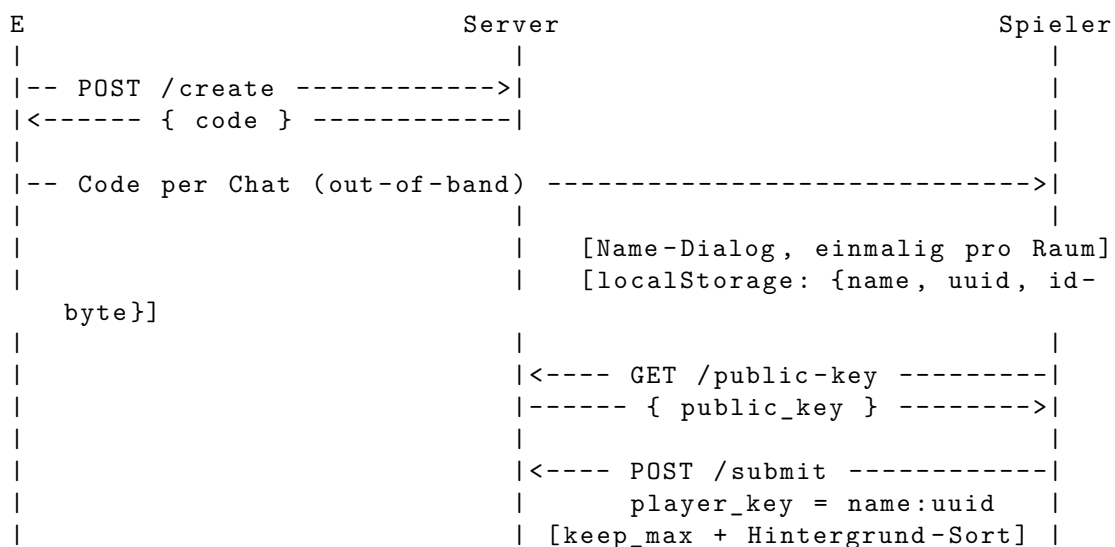
- **Ersteller (E):** Der Ersteller generiert lokal das TFHE-Schlüsselmateriale (ClientKey, ServerKey, PublicKey), legt den Raum an und teilt den 6-stelligen Raumcode extern an die Spieler. Der ClientKey verbleibt ausschließlich bei E, nur ServerKey und PublicKey wandern zum Server. E ist der einzige Akteur, der die sortierte Liste am Ende entschlüsseln kann.
- **Spieler (Client):** Ein Spieler tritt einem Raum bei, indem er den Raumcode eingibt. Beim ersten Beitritt wählt er einmalig einen Namen, der Client erzeugt im Hintergrund eine UUID und ein zufälliges ID-Byte und legt beides in `localStorage` ab. Der Spieler holt den PublicKey vom Server, verschlüsselt damit seinen Score und sein ID-Byte und sendet beides per Submit. Spieler sehen zu keinem Zeitpunkt die Score-Werte anderer Teilnehmer.
- **Backend (Server):** Der Server verwaltet pro Raum eine Session mit dekomprimiertem ServerKey, der Liste verschlüsselter Einträge (Score + ID als Ciphertext) und einer sortierten Sicht. Er führt homomorphe Vergleiche aus, sortiert die Liste blind und hat zu keinem Zeitpunkt Zugriff auf Klartext-Scores.

Lebenszyklus einer Session

1. **Raum anlegen:** Der Ersteller generiert das TFHE-Schlüsselpaar und ruft `POST /create` mit ServerKey und PublicKey auf. Der Server dekomprimiert den ServerKey einmalig im Hintergrund, legt eine Session an und gibt einen 6-stelligen Code zurück. Der ClientKey verbleibt ausschließlich beim Ersteller.
2. **Code teilen:** Der Ersteller gibt den Code out-of-band an seine Spieler weiter, also etwa per Chat oder mündlich. Es gibt keinen Einladungs-Endpoint, die Zulassung wird sozial geregelt.

3. **Namen wählen:** Beim ersten Beitritt zeigt der Spieler-Client einen einmaligen Name-Dialog (nur Buchstaben, 1 bis 20 Zeichen). Im Hintergrund werden eine UUID und ein zufälliges ID-Byte erzeugt. Beides wird in `localStorage` unter `lb_player_<code>` abgelegt. Bei einem späteren Beitritt in denselben Raum wird der Dialog übersprungen und die gespeicherte Identität wiederverwendet.
4. **Mitmachen:** Der Spieler ruft `GET /{code}/public-key` ab, verschlüsselt lokal seinen Score und sein ID-Byte und sendet beides per `POST /{code}/submit`. Als `player_key` geht das Klartext-Format `name:uuid` zum Backend, die UUID dient nur der Eindeutigkeit für die Wiedererkennung und wird nirgends im UI angezeigt. Reicht derselbe Spieler später erneut ein, behält der Server verschlüsselt das Maximum aus altem und neuem Score (`keep_max`).
5. **Sortieren:** Nach jedem Submit stößt der Server im Hintergrund eine Sortierung an (Batcher-Odd-Even-Mergesort). Pro Session läuft immer nur eine Sortierung gleichzeitig; weitere Submits während eines laufenden Sorts werden nach dessen Abschluss berücksichtigt.
6. **Ergebnis abrufen:** Der Ersteller ruft `GET /{code}/entries` ab. Die Response enthält zwei Listen: die sortierten Ciphertexte (Score und ID) und eine Klartext-Spielerliste mit `player_key` und `encrypted_id` pro Eintrag. Der Ersteller entschlüsselt jeden ID-Ciphertext lokal mit seinem ClientKey, baut daraus eine Tabelle `id-byte → name` und ordnet die Namen den sortierten Positionen zu. Optional fragt er über `POST /{code}/rank` die Position einer bestimmten ID ab und bekommt pro Listenplatz einen verschlüsselten Bool zurück.
7. **Schließen:** Es gibt keinen expliziten Schließen-Endpoint. Sessions werden vom Janitor nach 10 Minuten Inaktivität entfernt. Der Janitor ist ein Hintergrund-Task, der einmal pro Minute durch die Session-Map iteriert und alle Räume freigibt, an denen seit über 10 Minuten kein Request mehr eingegangen ist. Beim Entfernen wird der dekomprimierte ServerKey (mehrere hundert MB) freigegeben, andernfalls würde der Speicher volllaufen. Bei einem Server-Neustart gehen alle Sessions verloren; es gibt keine Persistenz.

Verhaltensdiagramm



```

|                                     |<---- POST /submit ... ----->|
|                                     |                                     |
|-- GET /entries ----->|                                     |
|<-- entries + Spielerliste --|                                     |
|                                     |                                     |
| E entschlüsselt jede ID, mappt Spielerliste -> Namen pro Rang

```

3.8.2 OpenAPI-Schnittstelle

Der Service stellt eine fachliche Leaderboard-API unter dem Pfad-Prefix `/leaderboard` bereit, mit der Räume erstellt, verschlüsselte Scores eingereicht und sortierte Ergebnisse abgefragt werden können. Die OpenAPI-Definition wird per `aide` aus dem Code generiert und ist unter `GET /openapi.json` sowie `GET /docs` (Swagger UI) verfügbar. Daneben existieren `/healthz`, `/readyz`, `/version` und `/metrics` aus den shared Crates.

Das Body-Limit liegt bei 2 GiB (`DefaultBodyLimit::max(2*1024^3)`), weil der komprimierte `ServerKey` je nach Parametern dreistellige Megabyte groß werden kann.

POST /create Legt einen neuen Leaderboard-Raum an. Der `ServerKey` wird einmalig dekomprimiert und in der Session gehalten, der `PublicKey` wird unverändert gespeichert und später an Spieler ausgeliefert.

Request:

```

{
  "server_key": "<base64(bincode(CompressedServerKey))>",
  "public_key": "<base64(bincode(CompactPublicKey))>"
}

```

Response 200:

```

{ "code": "847391" }

```

Fehlercodes:

- **400** – `server_key` oder `public_key` kein gültiges Base64 oder fehlgeschlagene `bincode`-Deserialisierung
- **500** – Panic im Dekomprimierungs-Thread

GET /{code}/public-key Gibt den `PublicKey` des Raums unverändert zurück, so wie er beim Anlegen hochgeladen wurde. Spieler nutzen ihn, um ihre Scores zu verschlüsseln.

Response 200:

```

{ "public_key": "<base64>" }

```

Fehlercodes:

- **404** – Raum nicht gefunden oder bereits vom Janitor entfernt

POST /{code}/submit Ein Spieler reicht seinen verschlüsselten Score ein. Bei einem Re-Submit desselben `player_key` führt der Server `keep_max` aus und behält verschlüsselt das Maximum aus altem und neuem Score.

Request:

```
{
  "player_key": "Alice:550e8400-e29b-41d4-a716-446655440000",
  "encrypted_score": "<base64(bincode(FheUint16))>",
  "encrypted_id": "<base64(bincode(FheUint8))>"
}
```

- `player_key` ist Klartext im Format `name:uuid`. Der Server nutzt den vollständigen String als Wiedererkennungsschlüssel, sodass ein erneutes Einreichen derselben Identität ein `keep_max` auslöst. Die UUID sorgt für Eindeutigkeit, falls mehrere Spieler denselben Namen wählen, und wird nirgends im UI angezeigt.
- `encrypted_score` ist ein `FheUint16` (Score zwischen 0 und 65535).
- `encrypted_id` ist ein `FheUint8`, ein zufälliges ID-Byte (0 bis 255), das der Client pro Raum einmalig erzeugt. Es wandert zusammen mit dem Score durch den FHE-Sort und dient dem Ersteller später als Klartext-Label, um die sortierte Liste auf Namen zu mappen.

Response 200: Leerer Body.

Fehlercodes:

- **400** – ungültiges Base64 oder fehlgeschlagene `bincode`-Deserialisierung
- **404** – Raum nicht gefunden
- **409** – Raum voll (`MAX_ENTRIES` = 20 erreicht und `player_key` ist neu)
- **500** – FHE-Panic

GET /{code}/entries Liefert die zuletzt fertig berechnete sortierte Reihenfolge sowie eine Klartext-Spielerliste der bekannten Teilnehmer. Ist noch kein Sort durchgelaufen, fällt `entries` auf die Einfügereihenfolge zurück. Die Spielerliste ist unabhängig vom Sort-Status immer vollständig (Einfügereihenfolge, mit `player_key` und `encrypted_id` pro Eintrag).

Response 200:

```
{
  "entries": [
    { "encrypted_score": "<base64(FheUint16)>",
      "encrypted_id": "<base64(FheUint8)>" },
  ],
  "roster": [
    { "player_key": "Alice:550e8400-...",
      "encrypted_id": "<base64(FheUint8)>" },
  ]
}
```

Der Ersteller entschlüsselt aus der Spielerliste pro Eintrag das ID-Byte und baut sich daraus eine byte → name-Tabelle. Mit dieser Tabelle bekommt jede sortierte Position in entries ihren Namen.

Fehlercodes:

- **404** – Raum nicht gefunden

POST /{code}/rank Fragt die Position einer verschlüsselten ID in der sortierten Liste ab. Pro Listenplatz kommt ein verschlüsselter Bool zurück (true heißt: die ID passt). Der Ersteller entschlüsselt jeden Bool lokal, und jede true-Position ist ein 1-basierter Rang. Mehrfachtreffer werden automatisch unterstützt.

Request:

```
{ "encrypted_id": "<base64(bincode(FheUint8))>" }
```

Response 200:

```
{ "matches": [ "<base64(FheBool)>", "<base64(FheBool)>" ] }
```

Fehlercodes:

- **400** – ungültiges Base64 oder fehlgeschlagene bincode-Deserialisierung
- **404** – Raum nicht gefunden
- **500** – FHE-Panic

3.8.3 Trust- und Threat-Model

Datum	Am Server klar	Am Server verschlüsselt	Am Client
Raumcode (6-stellig, im URL-Pfad)	X		X
ServerKey (notwendig für FHE-Ops)	X		X
PublicKey (wird an Spieler ausgeliefert)	X		X
ClientKey			X (nur bei E)
player_key (name:uuid, Wiedererkennungstoken)	X		X
UUID-Anteil von player_key	X		X (nie im UI sichtbar)
Anzahl Spieler im Raum	X		X
Submit-Zeitpunkt	X		X
Spieler-Score		X (FheUint16)	X (nur entschl. bei E)
Spieler-ID-Byte		X (FheUint8)	X (nur entschl. bei E)
/rank-Target-ID		X (FheUint8)	X
Sortierte Reihenfolge (verschlüsselt)		X	X
Sortierte Reihenfolge (entschlüsselt)			X (nur bei E)

Analyse: Beobachtbare Metadaten: TFHE schützt ausschließlich die Inhalte von Scores, IDs und der sortierten Liste. Für den Server bleiben weiterhin verschiedene Metadaten sichtbar:

- Anzahl und Aktivität paralleler Räume

- Anzahl der Spieler pro Raum
- Wer (per `player_key = name:uuid`) wann und wie oft postet
- Aus dem Timing der FHE-Operationen die Unterscheidung zwischen Erst-Submit (kein Vergleich) und Re-Submit (`keep_max`)
- Bei `/rank` der Zeitpunkt, zu dem E eine Rang-Abfrage stellt – jedoch nicht für welche Position
- Request-Timing und Submit-Pattern

Ein Server-Operator kann daraus Rückschlüsse auf Aktivitätsmuster, Gruppengrößen und das Verhalten einzelner Spieler ziehen. Der Klartext-Score bleibt dabei stets unsichtbar.

Restvertrauen in den Server: Der Server kann zwar keine Scores entschlüsseln, muss jedoch weiterhin als korrekter Ausführer des Protokolls vertrauenswürdig sein. Insbesondere wird angenommen, dass der Server:

- den richtigen PublicKey ausliefert und ihn nicht gegen einen unter seiner Kontrolle stehenden Schlüssel austauscht
- homomorphe Operationen korrekt ausführt und Ciphertexte nicht manipuliert
- die sortierte Liste vollständig und unverändert zurückgibt und keine Einträge ignoriert oder dupliziert
- Räume voneinander trennt
- die Verfügbarkeit des Systems sicherstellt

TFHE reduziert somit die Vertrauensabhängigkeit hinsichtlich der Inhaltsvertraulichkeit, ersetzt jedoch kein vollständig vertrauensloses Protokoll. Eine Verifizierbarkeit der serverseitigen Berechnung (etwa per ZKP) ist nicht umgesetzt.

Annahmen außerhalb von TFHE:

- Die Kommunikation zwischen Client und Server erfolgt über TLS. In Test-Umgebungen wird teilweise reines HTTP verwendet, was akzeptabel ist, da ohnehin nur Ciphertexte über die Leitung gehen.
- Das Frontend führt die Verschlüsselung und Entschlüsselung korrekt aus.
- Die verwendete TFHE-rs-Bibliothek (Version 1.6.1) wird als kryptographisch korrekt implementierte Black Box betrachtet.
- Der ClientKey verbleibt ausschließlich beim Raum-Ersteller.
- Der `player_key` ist absichtlich Klartext, damit der Server Re-Submits desselben Spielers erkennen kann.

Das System besitzt keine starke Benutzerauthentifizierung. Die Spieler-Zulassung erfolgt ausschließlich über die Kenntnis des Raumcodes, der out-of-band geteilt wird.

Schutzversprechen:

Durch den Einsatz von TFHE wird garantiert:

- Der Server kann individuelle Scores nicht im Klartext lesen.
- Der Server kann beim Vergleich zweier verschlüsselter Scores das Vorzeichen des Vergleichs nicht ableiten.
- Die sortierte Liste liegt auf dem Server ausschließlich verschlüsselt vor.
- Auch die `/rank`-Antwort liegt auf dem Server ausschließlich verschlüsselt vor.

Nicht garantiert werden:

- Vertraulichkeit der Klartext-Namen (`player_key`), da sie für die serverseitige Wiedererkennung notwendig sind
- Vertraulichkeit von Metadaten wie Anzahl Spieler, Submit-Zeitpunkten und Raum-Aktivität
- Schutz vor Traffic- und Timing-Analysen
- Schutz vor einem aktiv manipulierenden Operator (Key-Tausch, Eintrags-Manipulation, Antwort-Fälschung)
- Authentifizierung, Rate-Limiting oder Persistenz

„Der Server kennt den Klartext-Score nicht. Er sieht nur Ciphertexte, vergleicht zwei Ciphertexte miteinander, ohne das Vorzeichen des Vergleichs zu kennen, und gibt die sortierte Liste an E zurück. Nur E kann die Liste mit seinem ClientKey entschlüsseln.“

Diese Garantie hält gegen einen Operator, der das Protokoll korrekt ausführt, aber Metadaten beobachtet. Gegen einen aktiv manipulierenden Operator hält sie nicht.

Einordnung: Der Use Case implementiert ein TFHE-geschütztes Leaderboard mit Vertraulichkeit auf Inhaltsebene. Geschützt werden ausschließlich individuelle Scores, die zugehörigen ID-Bytes und die daraus abgeleitete sortierte Reihenfolge. Klartext-Namen, Metadaten und der allgemeine Kontrollfluss der Anwendung bleiben außerhalb des Schutzbereichs von TFHE.

3.8.4 FHE-Designentscheidungen

Verwendet wird TFHE-rs in Version 1.6.1 mit `ConfigBuilder::default()`, also die Standard-Parameter ohne eigenes Tuning. Für die hier gebrauchten Bitbreiten reicht das aus.

Verwendete TFHE-rs-Typen:

Für den Score wird `FheUint16` (Wertebereich 0 bis 65535) verwendet. Spielrelevante Scores wie Punkte oder Zeiten in Hundertstelsekunden lassen sich darauf vollständig abbilden. `FheUint8` (max 255) wäre für viele Quiz- und Spielszenarien zu eng, und `FheUint32` ist etwa doppelt so langsam pro Vergleich, ohne dass der erweiterte Wertebereich hier benötigt wird.

Für die Spieler-ID wird `FheUint8` verwendet. Bei `MAX_ENTRIES = 20` reichen die 0 bis 255 darstellbaren Werte mehr als aus. Die ID dient nur als Tag, der zusammen mit dem Score durchsortiert wird, damit der Ersteller nach dem Entschlüsseln weiß, wer auf welchem Platz steht.

Als Rückgabewert von `/rank` wird `FheBool` verwendet, jeweils einer pro Position der sortierten Liste. Das ist der minimale Ciphertext, da der Client nur „passt / passt nicht“ lesen muss.

Verwendete homomorphe Operationen:

Der Server führt drei verschiedene Operationen auf den verschlüsselten Werten aus. `FheUint16::lt` ist der einzige benötigte Vergleich – absteigend sortieren heißt: „tausche, wenn links kleiner ist“. `FheBool::if_then_else` schaltet Score und ID synchron um, ohne dass der Server weiß, welcher Branch gewinnt. Diese Operation wird in `keep_max` (zweimal für Score, zweimal für ID) und im Sort-Comparator (viermal pro Vergleich) eingesetzt. `FheUint8::eq` kommt ausschließlich in `rank_matches` zum Einsatz, dem Endpoint für die Positions-Abfrage.

Andere homomorphe Operationen wie Addition, Multiplikation oder Bit-Shifts werden nicht benötigt, dadurch bleibt die serverseitige Verarbeitung vollständig auf Vergleichen und datenabhängigem Auswählen beschränkt.

Verworfen Alternativen:

FheUint32 für Score: `FheUint32` hätte mehr Headroom geboten, wurde aber verworfen, da jede zusätzliche Bitbreite die FHE-Vergleichszeit etwa verdoppelt und 16 Bit für diesen Use Case ausreichen.

Klartext-Score mit ZKP: Ein Klartext-Score, kombiniert mit einem Zero-Knowledge-Proof für den erlaubten Wertebereich, wäre semantisch interessant. Dann sähe der Server jedoch den Score, und genau das soll vermieden werden.

Vollständig datenabhängiger Sort: Ein klassischer Sortier-Algorithmus mit datenabhängigen Branches ist in TFHE-rs prinzipiell nicht möglich, weil Verzweigungen auf Ciphertexten nicht entscheidbar sind. Die Lösung ist ein datenunabhängiges Sortiernetzwerk (siehe Komplexität), das in einer festen Sequenz von Compare-and-Swap-Schritten arbeitet.

Verschlüsselter player_key: Den `player_key` zu verschlüsseln wäre technisch möglich, aber unverhältnismäßig teuer. Der Server benötigt den Schlüssel, um beim Submit zu prüfen, ob bereits ein Eintrag desselben Spielers existiert, und gegebenenfalls `keep_max` auszuführen. Im Klartext geschieht das durch einen einfachen String-Vergleich in $O(1)$. Verschlüsselt müsste der Server pro Submit jeden existierenden Eintrag homomorph vergleichen (`eq`) und jede Score- und ID-Position bedingt überschreiben (`if_then_else`). Da er auf einem verschlüsselten Bool keine Verzweigung treffen kann, müsste er jeden Submit zugleich als neuen Eintrag anhängen und auf alle bestehenden Einträge bedingt anwenden. Damit stiegen die Submit-Kosten von $O(1)$ auf $O(n)$ FHE-Operationen, und die Listenlänge wäre nicht mehr sinnvoll beschränkbar. Der Privacy-Gewinn (Pseudonym statt Name) rechtfertigt diesen Aufwand nicht, da der Creator die Identität der Mitspieler ohnehin kennt.

Approximationen werden nicht eingesetzt. Alle Operationen liefern exakte Resultate, solange die `ServerKey`-Parameter korrekt installiert sind. Es gibt kein Rauschen, das akzeptiert werden müsste, weil TFHE-rs intern bootstrapt und der Output deterministisch ist.

3.8.5 Komplexität der eigenen Algorithmen

- n : Anzahl der Spieler im Raum (maximal 20)

Die interne Komplexität der TFHE-rs-Bibliothek wird nicht betrachtet; jede homomorphe Operation wird gemäß Aufgabenstellung als $O(1)$ angenommen.

Funktion	Zeitkomplexität	Platzkomplexität
<code>create_session</code>	$O(1)$	$O(1)$
<code>get_public_key</code>	$O(1)$	$O(1)$
<code>submit_score (keep_max)</code>	$O(1)$	$O(1)$
<code>sort_by_score_desc</code>	$O(n \cdot \log^2 n)$	$O(n)$
<code>get_entries</code>	$O(n)$	$O(n)$
<code>query_rank (rank_matches)</code>	$O(n)$	$O(n)$

Create_session – $O(1)$, $O(1)$: Beim Anlegen eines Raums wird der ServerKey einmalig im Hintergrund dekomprimiert und in der Session abgelegt. Die übrigen Operationen (Code-Generierung, Initialisierung leerer Listen, Einfügen in die Session-Map) besitzen konstante Laufzeit. Der dekomprimierte ServerKey belegt mehrere hundert MB RAM, ist aber unabhängig von n .

Get_public_key – $O(1)$, $O(1)$: Die Session wird per HashMap-Lookup gefunden, der PublicKey-String wird unverändert zurückgegeben. Es erfolgt weder eine FHE-Operation noch eine Iteration über Einträge.

Submit_score – $O(1)$, $O(1)$: Reicht ein Spieler einen neuen Score ein, obwohl bereits ein alter vorliegt, behält der Server verschlüsselt den höheren. Das sind unabhängig von der Belegung des Raums stets dieselben fünf FHE-Operationen (ein `lt` plus vier `if_then_else`), also $O(1)$. Ist der Spieler neu, entfällt der Vergleich ganz und der Eintrag wird direkt angehängt. Der nachgelagerte Sort läuft asynchron im Hintergrund und blockiert die Antwort nicht.

Sort_by_score_desc – $O(n \cdot \log^2 n)$, $O(n)$: Auf verschlüsselten Zahlen kann der Server die Größenrelation nicht einsehen und auf dieser Grundlage entscheiden. Die Vergleiche müssen blind erfolgen, in einer festen Reihenfolge, die unabhängig vom Inhalt funktioniert. Dies leistet ein Sortiernetzwerk: eine vorab festgelegte Folge von Compare-and-Swap-Operationen der Form „Vergleiche zwei Positionen und tausche sie, falls links kleiner ist“. Verwendet wird Batcher's Odd-Even-Mergesort, da seine Vergleiche in Schichten organisiert sind, deren Paare disjunkt sind. Eine vollständige Schicht kann dadurch parallel auf mehreren CPU-Kernen ausgeführt werden. Der Aufwand beträgt $O(n \cdot \log^2 n)$ Vergleiche insgesamt bei $O(\log^2 n)$ Schichten in der Tiefe. Für $n = 20$ ergeben sich rund 120 Vergleiche in etwa 15 Schichten. Die Korrektheit ist im Test `batcher_layers_form_a_valid_sorting_network` per 0/1-Prinzip nachgewiesen. Der Platzbedarf ist linear in n , da die Liste der Ciphertexte vorgehalten wird.

Der aufwendige Sortierschritt ist der maßgebliche Grund für die feste Obergrenze `MAX_ENTRIES` = 20.

Get_entries – $O(n)$, $O(n)$: Für die Antwort wird die sortierte Liste (oder als Fallback die Einfügereihenfolge) sowie die Spielerliste ausgegeben. Jeder der n Einträge wird einmal in das DTO kopiert. Es erfolgt keine FHE-Operation.

Query_rank – $O(n)$, $O(n)$: Für jede der n Positionen der sortierten Liste wird genau ein `eq`-Vergleich zwischen der Ziel-ID und der Listen-ID ausgeführt. Das Ergebnis ist eine Liste von n verschlüsselten Booleans. Es werden weder Schleifen über Scores noch zusätzliche Comparator-Operationen benötigt.

3.8.6 Performance-Messung

Mess-Setup: Die Tests laufen auf einem NetCup-Server (AMD EPYC 9645, 8 Cores) gegen Image v0.1.19, TFHE-rs 1.6.1 mit `ConfigBuilder::default()`, `FheUint16` für den Score

und FheUint8 für die ID. Die Last erzeugt k6 mit vor-generierten Ciphertexten gegen <http://159.195.145.100/leaderboard>. Die Skripte liegen unter loadtest/k6/.

Gemessen werden die FHE-Endpunkte: POST /create (ServerKey-Decompress) und POST /{code}/submit (keep_max plus Background-Sort). Bewusst ausgeklammert sind GET /entries, GET /{code}/public-key und POST /rank, weil sie kein FHE machen und nur Grundrauschen liefern.

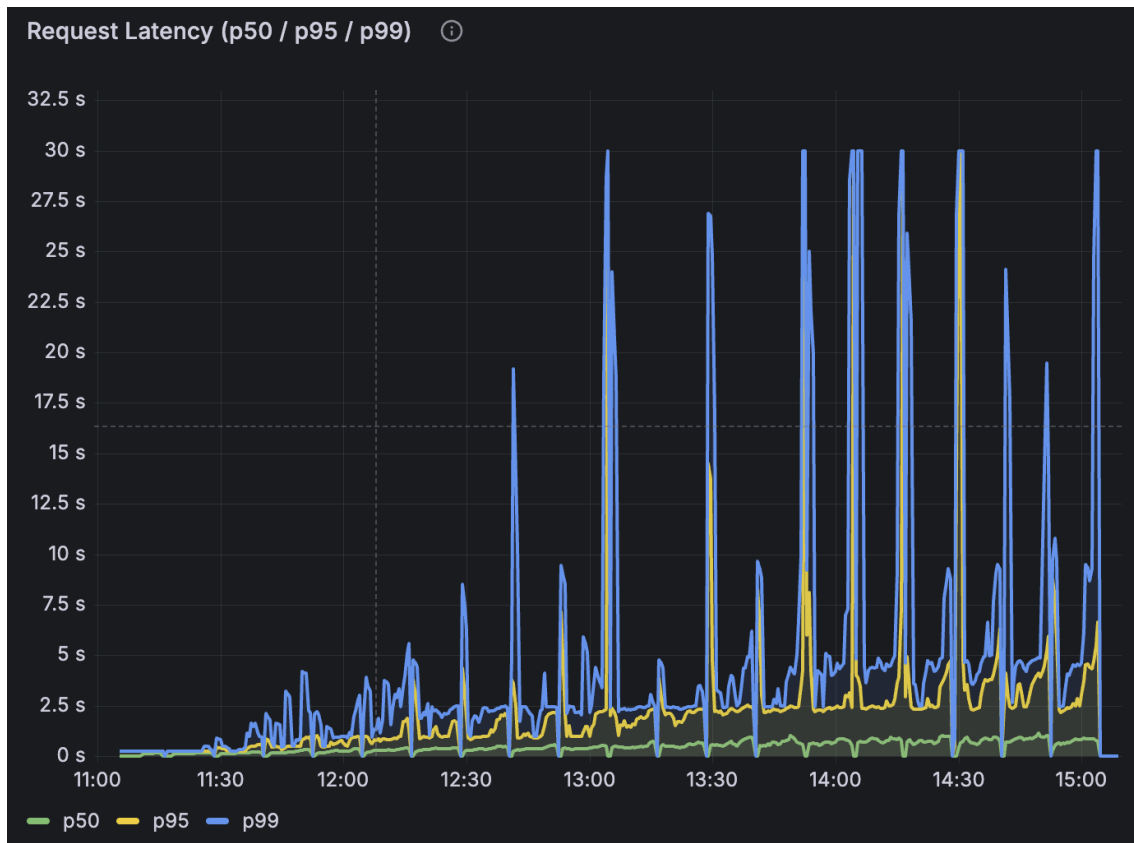
Test 1 – Room-Growth (29.05.2026) Jede Minute wird ein neuer Raum mit einem Spieler und einem Submit angelegt, mit einem Keepalive alle 9 Minuten. Skript: 01_room_growth.js.

Metrik	Wert
p50	328 ms
p95	6,41 s
Fehlerrate	5,20 %
Kipp-Punkt	Raum 59 (502 Bad Gateway)
Sessions vor Kipp	57 stabil

Die CPU-Last bleibt während des gesamten Tests nahezu konstant niedrig. Jeder Raum erzeugt nur ein create und alle 9 Minuten einen Keepalive-Submit, also keine nennenswerte FHE-Dauerlast. Dieser Test misst damit ausschließlich die RAM-Obergrenze durch Session-Akkumulation, nicht die Rechenleistung.

Test 2 – Room-Fill (30.05.2026) Ein Raum, in dem die Spielerzahl in 20 Runden von 1 auf 20 wächst. Pro Runde gibt es 10 Tempo-Stufen (Sleep 10 s herunter auf 1 s) und danach 2 Minuten Pause. Skript: 02_room_fill.js.

Metrik	Wert
p50 (Submit)	612 ms
p95	2,39 s
Fehlerrate	0,02 %
Durchsatz Ø	1,75 req/s über 4 h
Kipp-Punkt	kein Crash, 4 h durchgelaufen
Erste p95-Sprünge	ab Runde 7 (= 7 Spieler)
6 Timeouts	Runde 18 (Sek 12 317), sofortige Recovery



Test 2 – p50/p95/p99 über die Zeit

Im Latenz-Verlauf zeigt sich:

- p50 (grün) steigt gleichmäßig mit jeder zusätzlichen Spielerzahl.
- p95 (gelb) zeigt ab Runde 7 erste deutliche Sprünge, die mit jeder weiteren Runde größer werden.
- p99 (blau) hat Spitzen bis 30s ab Runde 12, die sich am Ende häufen.

Die p95- und p99-Spitzen treten systematisch zu Beginn jeder Runde auf. Nach der zweiminütigen Pause setzen alle aktiven Spieler nahezu gleichzeitig ihren ersten Submit ab, wodurch sich die FHE-Queue und der Hintergrund-Sort kurz aufstauen. Innerhalb weniger Sekunden desynchronisieren sich die Spieler und die Latenz stabilisiert sich auf einem niedrigeren Niveau.

Test 3 – Happy-Flow (30.05.2026) Ein Spieler durchläuft 20-mal sequentiell den Ablauf public-key → 5× submit → entries → rank (8 Requests pro Iteration). Skript: 03_happy_flow.js.

Metrik	Wert
Fehlerrate	0,00 % (160/160 Checks)
Flow-Summe p50	4,6 s
Flow-Summe p95	5,9 s

Throughput-Grenze

- **Parallele Sessions:** 57 stabil. Ab Raum 59 fällt der Service mit 502 aus.
- **Submit unter Dauerlast (4 h, bis 20 Spieler):** p95 gesamt 2,39s, erste p95-Sprünge ab Runde 7 (siehe Test-2-Grafik), p50 steigt gleichmäßig mit der Spielerzahl, p99-Spitzen bis 30s.
- **RAM pro aktiver Session:** etwa 350 bis 400 MB für den dekomprimierten ServerKey (komprimiert über die Leitung 80 MB).

Lastkurve

Im Grafana-Dashboard leaderboard-perf lässt sich der jeweilige Run über den Filter testid einblenden. Dort sind die per-Stufe-Latenzen und die Ressourcen über die Zeit ablesbar.

Reproduzierbarkeit

```
cargo run --release -p encrypted-leaderboard --features loadtest \
  --bin gen_corpus -- --out services/08-encrypted-leaderboard/loadtest/
  corpus
```

```
kubectrl port-forward -n monitoring svc/prometheus-operated 9090:9090 &
cd services/08-encrypted-leaderboard/loadtest/k6
```

```
K6_PROMETHEUS_RW_SERVER_URL=http://localhost:9090/api/v1/write \
k6 run --out experimental-prometheus-rw 01_room_growth.js \
-e BASE_URL=http://159.195.145.100/leaderboard
```

```
K6_PROMETHEUS_RW_SERVER_URL=http://localhost:9090/api/v1/write \
k6 run --out experimental-prometheus-rw 02_room_fill.js \
-e BASE_URL=http://159.195.145.100/leaderboard
```

```
K6_PROMETHEUS_RW_SERVER_URL=http://localhost:9090/api/v1/write \
k6 run --out experimental-prometheus-rw 03_happy_flow.js \
-e BASE_URL=http://159.195.145.100/leaderboard
```

3.8.7 Limitationen

- Maximal 20 Spieler pro Raum (MAX_ENTRIES), bedingt durch die FHE-Sort-Komplexität $O(n \cdot \log^2 n)$. Größere Räume sind technisch möglich, würden aber die Latenz pro Submit überproportional steigen lassen.
- Maximal etwa 57 parallele Räume pro Service-Instanz, begrenzt durch den RAM-Verbrauch der dekomprimierten ServerKeys (siehe Test 1). Ab Raum 59 fällt die Instanz mit 502 Bad Gateway aus.
- Sessions werden nach 10 Minuten Inaktivität vom Janitor entfernt. Es gibt keine Persistenz, ein Server-Neustart entfernt sämtliche aktiven Sessions inklusive aller eingereichten Scores.

- Keine Authentifizierung und kein Rate-Limit. Wer den Raumcode kennt, kann submitten. Die Spieler-Zulassung ist sozial geregelt, dadurch dass der Ersteller den Code nur an seine Gruppe weitergibt.
- Keine Float-Werte, keine Division und keine datenabhängigen Branches in TFHE-rs. Score-Updates müssen sich auf `max()` reduzieren lassen, Nachkommastellen werden über eine Skalierung auf Ganzzahlen abgebildet.
- Der `player_key` muss im Klartext zum Server gehen, damit die serverseitige Wiedererkennung funktioniert. Das verrät, wer wann postet, lässt aber keinen Rückschluss auf den Score zu.
- Verliert der Ersteller seinen ClientKey, ist die sortierte Liste nicht mehr lesbar. Eine Recovery ist nicht möglich, da der Server keinen Klartext-Score kennt.

3.9 Encrypted-Program-Execution

3.9.1 Funktionsbeschreibung

Der Use-Case “Encrypted Program Execution” bietet Usern die Möglichkeit, ein Programm auf einem Server auszuführen, ohne dass diesem dafür Einsicht in Algorithmen oder Berechnungen gewährt wird.

Dafür läuft auf dem Server ein CPU-Simulator, auf dem Programme isoliert und vollständig verschlüsselt ausgeführt werden.

Der Simulator basiert dabei auf einer Von-Neumann-Akkumulator-Architektur und verfügt über

- ein Akkumulator-Register A
- ein weiteres Register B, das bei vielen Instruktionen einen der beiden Operanden enthält
- 16-Bit Instruktionen mit 8-Bit Opcode und 8-Bit Operanden
- eine Carry-Flag, u.a. integriert durch einen JMC-Befehl
- native Unterstützung von höheren Zahlenbereichen durch ADDC und SUBC Befehlen
- 8-Bit Wortbreite
- einem RAM mit 8-Bit Adressraum
- geteiltem Adressraum für Programmcode und Programmspeicher

?? bietet eine Übersicht über das Instruction-Set. Besonders hervorzuheben ist dabei, dass auf jeden Opcode ein Operand folgt, auch bei Operationen, bei denen dieser keine Bedeutung hat. Dieses redundante Byte kann als Don't-Care angesehen, und frei verwendet werden. Bemerkenswert ist auch das explizite Fehlen eines HALT-Befehls, wobei dies technisch begründet ist, und im Unterabschnitt Trust- und Threat-Model weiter erklärt wird. Nach Sprüngen wird der Programmcounter nicht inkrementiert, ein Assembler muss Sprungadressen dementsprechend bei der Assemblierung nicht verändern. Nach allen weiteren Instruktionen wird der Programmcounter um zwei inkrementiert:

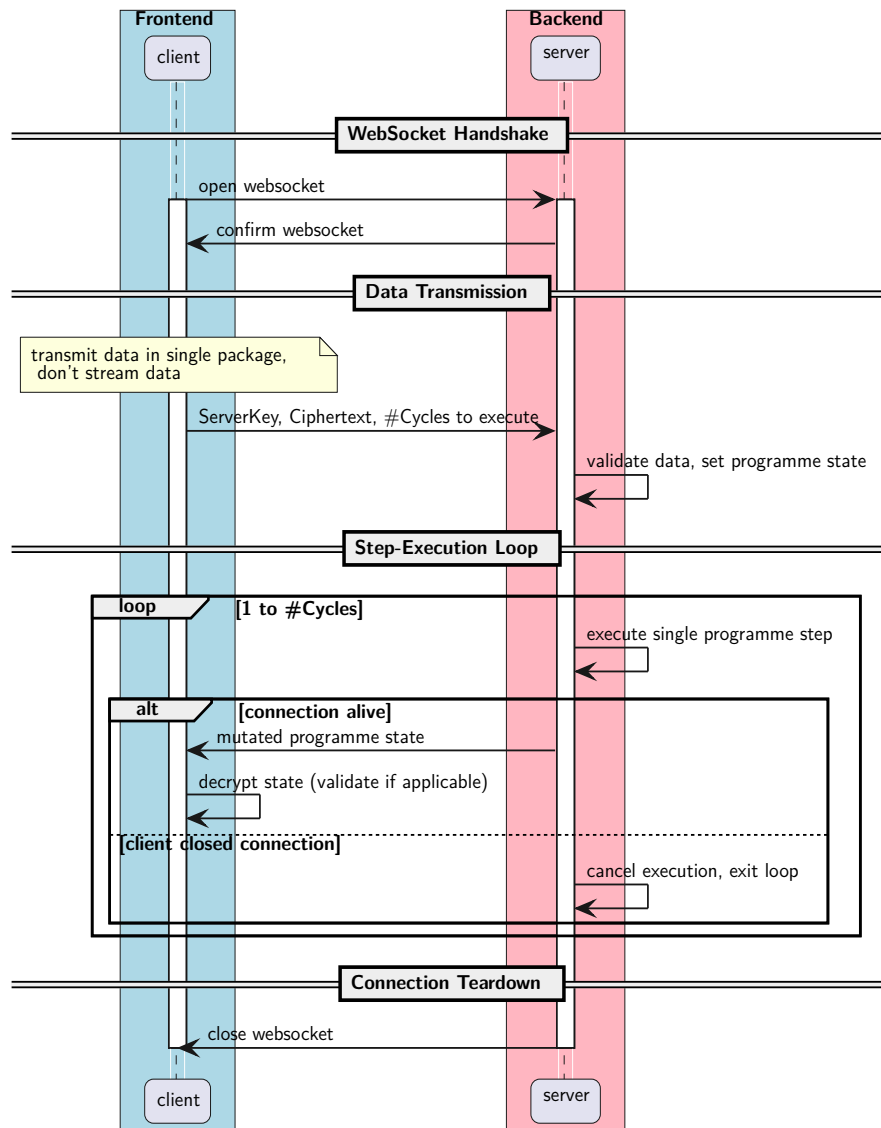
$$PC \leftarrow PC + 2$$

Client-Server-Interaktion:

Die Kommunikation mit dem Server findet über eine REST-Schnittstelle oder einen WebSocket statt. Das folgende Sequenzdiagramm illustriert den Ablauf der Ausführung eines Programms. Zuvor muss der Client ein Schlüsselpaar (ClientKey, ServerKey) generieren, Zellenwerte und Länge des RAMs (beinhaltet Programm), und Werte für A und B Register sowie für die Carry-Flag bestimmen und verschlüsseln.

Tabelle 3: Simulator Instruction-Set-Architecture

Opcode	Mnemonic	Operand	Carry	Description
0000_0000	<i>NOP</i>	<i>ignored</i>	–	<i>No operation</i>
0000_0001	<i>LDA(direct)</i>	<i>source_ADR</i>	–	$A \leftarrow M[\text{source_ADR}]$
0000_0010	<i>LDA(immediate)</i>	<i>data</i>	–	$A \leftarrow \text{data}$
0000_0011	<i>LDR</i>	<i>target_ADR</i>	–	$M[\text{target_ADR}] \leftarrow A$
0000_0100	<i>SWP</i>	<i>ignored</i>	–	$A \leftarrow B, B \leftarrow A$
0000_0101	<i>JMP(direct)</i>	<i>ADR</i>	–	$SP \leftarrow \text{ADR} - 1$
0000_0110	<i>JMZ(direct)</i>	<i>ADR</i>	–	if $A = 0$ then $SP \leftarrow \text{ADR}$
0000_0111	<i>JMC(direct)</i>	<i>ADR</i>	–	if carry then $SP \leftarrow \text{ADR}$
0000_1000	<i>DJNZ(direct)</i>	<i>ADR</i>	–	$A \leftarrow A - 1$; if $A \neq 0$ then $SP \leftarrow \text{ADR}$
0000_1001	<i>ADD</i>	<i>ignored</i>	*	$A \leftarrow A + B$
0000_1010	<i>ADDC</i>	<i>ignored</i>	*	$A \leftarrow A + B + \text{carry}$
0000_1011	<i>ADD(immediate)</i>	<i>data</i>	*	$A \leftarrow A + \text{data}$
0000_1100	<i>ADDC(immediate)</i>	<i>data</i>	*	$A \leftarrow A + \text{data} + \text{carry}$
0000_1101	<i>SUB</i>	<i>ignored</i>	*	$A \leftarrow A - B$
0000_1110	<i>SUBC</i>	<i>ignored</i>	*	$A \leftarrow A - B - \text{carry}$
0000_1111	<i>SUB(immediate)</i>	<i>data</i>	*	$A \leftarrow A - \text{data}$
0001_0000	<i>SUBC(immediate)</i>	<i>data</i>	*	$A \leftarrow A - \text{data} - \text{carry}$
0001_0001	<i>MUL</i>	<i>ignored</i>	0	$A \parallel B \leftarrow A \times B$
0001_0010	<i>MUL(immediate)</i>	<i>data</i>	0	$A \parallel B \leftarrow A \times \text{data}$
0001_0011	<i>DIV</i>	<i>ignored</i>	0	$A \leftarrow \lfloor A/B \rfloor$
0001_0100	<i>DIV(immediate)</i>	<i>data</i>	0	$A \leftarrow \lfloor A/\text{data} \rfloor$
0001_0101	<i>SLA</i>	<i>ignored</i>	$A[7]$	$A \leftarrow A \ll 1$
0001_0110	<i>SRA</i>	<i>ignored</i>	$A[0]$	$A \leftarrow A \gg 1$
0001_0111	<i>RLA</i>	<i>ignored</i>	$A[7]$	$A \leftarrow (A \ll 1) \vee \text{carry}$
0001_1000	<i>RLC</i>	<i>ignored</i>	$A[7]$	$A \leftarrow (A \ll 1) \vee A[7]$
0001_1001	<i>RRA</i>	<i>ignored</i>	$A[0]$	$A \leftarrow (A \gg 1) \vee (\text{carry} \ll 7)$
0001_1010	<i>RRC</i>	<i>ignored</i>	$A[0]$	$A \leftarrow (A \gg 1) \vee (A[0] \ll 7)$
0001_1011	<i>CCA</i>	<i>ignored</i>	–	$A \leftarrow \overline{A}$
0001_1100	<i>AND</i>	<i>ignored</i>	–	$A \leftarrow A \wedge B$
0001_1101	<i>OR</i>	<i>ignored</i>	–	$A \leftarrow A \vee B$
0001_1110	<i>XOR</i>	<i>ignored</i>	–	$A \leftarrow A \oplus B$
0001_1111	<i>DEC</i>	<i>ignored</i>	*	$A \leftarrow A - 1$
0010_0000	<i>INC</i>	<i>ignored</i>	*	$A \leftarrow A + 1$



Bei längeren und komplexeren Programmen sind die regelmäßigen Mitteilungen des aktuellen Programmzustandes, die durch die Verwendung von WebSockets ermöglicht werden, besonders vorteilhaft, da so eventuelle Fehler im Programm frühestmöglich erkannt, und die Ausführung abgebrochen werden kann. Ebenso kann der Server die Ausführung ohne einen expliziten Befehl des Clients abbrechen, falls die Verbindung ungeordnet beendet wurde, um so Ressourcen zu schonen.

Das Schließen des WebSockets beendet die Session aus Sicht des Servers. Aus Sicht des Clients kann die Session jedoch verlängert werden, indem ein neuer WebSocket geöffnet, und der übermittelte Programmendzustand dem Server als Startzustand des Programms einer neuen Programmausführung übermittelt wird. So kann ein Programm über mehrere Sessions hinweg ausgeführt werden.

3.9.2 OpenAPI-Schnittstelle

GET /execute-stream

Etabliert den WebSocket. Nach Verbindungsaufbau kann genau ein Programm ausgeführt werden, nachdem die angegebene Anzahl an Programmschritten ausgeführt wurde, wird der

WebSocket geordnet geschlossen. Es wird maximal ein Programm gleichzeitig ausgeführt, parallel können weitere WebSockets geöffnet werden, sendet ein weiterer Client eine Request-Execution-Nachricht während bereits ein Programm ausgeführt wird, schließt der Server den WebSocket des zweiten Clients und meldet eine Überlastung (Close-Code 1013)

Response 101

Request Execute Program Message

Programmzustand, Server-Key und Anzahl auszuführender Schritte zur unmittelbaren Ausführung. Dies ist die erste Nachricht, die über den WebSocket gesendet wird. Der Client sendet diese Nachricht genau einmal zu Beginn der Kommunikation, für nachfolgende Programme muss ein neuer WebSocket geöffnet werden. Weitere Request-Execute-Program-Nachrichten haben keinen Effekt und bewirken keine Antwort vom Server.

```
{
  "cycles": "number",
  "a": "string", <Base64–Enkodierter TFHE–rs FheUint8>
  "b": "string", <Base64–Enkodierter TFHE–rs FheUint8>
  "pc": "string", <Base64–Enkodierter TFHE–rs FheUint8>
  "carry": "string", <Base64–Enkodierter TFHE–rs FheBool>
  "memory": [
    "string", <Base64–Enkodierter TFHE–rs FheUint8>
  ],
  "server_key": "string"
  <Base64–Enkodierter TFHE–rs compressed Server Key>
}
```

Error Response

Falls die Nachricht unvollständig oder fehlerhaft ist, sendet der Server eine Error-Response und schließt den WebSocket geordnet.

```
{
  "message": "string"
}
```

Program State Response

Nach jedem ausgeführtem Zyklus teilt der Server dem Client den mutierten Programmzustand mit. Diese Nachricht wird also bei n auszuführenden Zyklen n -Mal gesendet. Nach der n -ten Nachricht schließt der Server den WebSocket geordnet.

```
{
  "a": "string", <Base64–Enkodierter TFHE–rs FheUint8>
  "b": "string", <Base64–Enkodierter TFHE–rs FheUint8>
  "pc": "string", <Base64–Enkodierter TFHE–rs FheUint8>
  "carry": "string", <Base64–Enkodierter TFHE–rs FheBool>
  "memory": [
    "string", <Base64–Enkodierter TFHE–rs FheUint8>
  ]
}
```

Close Codes

Folgende Close-Codes werden vom Server gesendet:

- 1000 bei geordnetem Verbindungsabbau durch Client oder nach Ende der Programm-

ausführung

- 1003 bei fehlerhaftem Request-Execute-Payload
- 1011 bei Fehler während der Simulation
- 1013 falls bereits ein Programm ausgeführt wird

3.9.3 Trust- und Threat-Model

Eine Übersicht über den Wissensstand des Servers zu Eingaben und Komponenten bietet ??.

Tabelle 4: Übersicht Trust-Modell

Datum	am Server klar	am Server verschlüsselt	nur am Client
Programmzustand			
Programm		X	
Programmcounter		X	
Registerwerte		X	
Carry-Flag		X	
RAM		X	
Simulator-Spezifikation			
Instructionset	X		
Wortbreite Simulator	X		
Architektur (Akkumulator)	X		
Programmausführungsmetadaten			
Anzahl Zyklen	X		
Größe RAM	X		
Kommunikationsmetadaten	X		

Folgende Mechanismen gewähren die Unwissenheit des Servers über das ausgeführte Programm:

- Die Instruktionen sind vollhomomorph verschlüsselt, der Server kann keine ausgeführten Algorithmen erkennen.
- Werte von RAM-Zellen und Registern sind vollhomomorph verschlüsselt, der Server kennt weder Operanden noch Ergebnisse.
- Der Programmcounter ist vollhomomorph verschlüsselt, der Server kann den Programmfluss (Sprünge, Schleifen, ...) nicht nachvollziehen.
- Der RAM-Zugriff ist vollhomomorph verschlüsselt, der Server kennt weder Ursprungs- noch Zieladressen von Lese- oder Schreibbefehlen.

Der Server führt somit in jedem Zyklus jede Instruktion aus, und selektiert schließlich für A und B Register, Programmcounter, jede RAM-Zeile, und Carry Flag das Ergebnis, vollhomomorph verschlüsselt.

Die fehlenden Informationen über auszuführende Instruktionen bedingen das oben genannte Fehlen eines HALT Befehls. Dies in Kombination mit der Unkenntnis des Programmcounters, begründet warum die konkrete Angabe der Anzahl an auszuführenden Zyklen nötig ist.

Folgende Informationen liegen dem Server vor:

- Metadaten über die Kommunikation: IP-Adresse, Zeitpunkt, Frequenz, Latenz, etc.
- technische Spezifikationen des Simulators: Wortbreite, Instruction-Set, Akkumulator-Architektur, Register (Anzahl, Größe),...
- Größe des RAMs für ein Programm
- Anzahl der auszuführenden Schritte des Programms

Die Kommunikationsmetadaten sind in diesem Kontext nicht explizit schützenswert, da nicht die Identität der User, sondern deren Algorithmen der Geheimhaltung obliegen. Diese Metadaten lassen keine Schlüsse auf schützenswerte Programme und Algorithmen zu. Ein frühzeitiges Abbrechen der Ausführung lässt einen Fehler im ausgeführten Programm vermuten, Informationen über den konkreten Inhalt des Programms, oder die Art des Fehlers werden damit nicht preisgegeben.

Die durch Klartextdaten erlangten Informationen über ein Programm lassen sich folgendermaßen quantifizieren: die Anzahl möglicher Instruktionsabfolgen ist

$$instruction_sequences = number_of_instructions^{executed_cycles}$$

mit den 32 Instruktionen des Simulators ergibt sich

$$instruction_sequences = 32^{executed_cycles}$$

Beispiel: 10 Zyklen

$$instruction_sequences = 32^{10} > 10^{15} \text{ (1 Brd.)}$$

Auch der limitierte Adressraum von $2^8 = 256$ Adressen ermöglicht mit 8-Bit Wortbreite einen Zustandsraum von maximal

$$2^{256 \cdot 8} = 2^{2048}$$

möglichen Zuständen. Falls ein kleinerer RAM mit bspw. 10 Adressen gewählt wird, ergibt sich ein Zustandsraum von

$$2^{10 \cdot 8} = 2^{80} > 10^{24} \text{ (1 Quadrillion)}$$

Dabei schränkt die Größe des RAMs die möglichen Instruktionsabfolgen nicht weiter ein, da ein Programm selbstverändernde Instruktionen beinhalten kann.

Somit lässt sich zwar demonstrieren, dass der Server auch bei kleinen Programmen keine Abschätzungen über den möglichen Inhalt oder Zustand des Programmes machen kann, insbesondere die Anzahl der Zyklen lässt jedoch Rückschlüsse auf die Komplexität ausgeführter Algorithmen zu. Sollte dies ein relevantes Risiko für User darstellen, können Programmausführungen künstlich verlängert werden: auch wenn das Ergebnis nach n Zyklen bereit ist, kann der Server bspw. angewiesen werden, das Programm für $2n$ Zyklen auszuführen. Die WebSocket-Kommunikation erlaubt dem User dabei, das Ergebnis bereits nach den tatsächlich nötigen n Takten auszuwerten.

Die Software ermöglicht somit Usern, schützenswerte Programme und Algorithmen, die einem Geheimhaltungsgebot unterliegen, auf einem Server einer nicht vertrauenswürdigen dritten Partei, etwa eines Cloud-Providers, auszuführen. Ferner auch bei bekannten Programmen und Algorithmen schützenswerte Ergebnisse auf Basis von schützenswerten Daten berechnet werden.

Diese Sicherheit ist bedingt durch:

- Kryptographische Sicherheit des TFHE-Verschlüsselungsschemas
- Kryptographisch sichere Implementierung durch TFHE-rs
- Geheimhaltung von Programmen und Algorithmen von Usern über den Entwicklungsprozess hinweg
- Vertrauen in die Entwicklungs-Toolchain (git-Server, Assemblierer, etc.)
- Vertrauen in eine Client-Anwendung, die die Klartextdaten verschlüsseln und an den Server sendet
- Kryptographisch sichere Implementierung des TFHE-Verschlüsselungsschemas einer Client-Anwendung
- Beschränkung von Zugriff auf alle Instanzen von unverschlüsselten Programmen, Daten und Rechenergebnissen auf Client-Seite

Sollten eine, oder mehrere dieser Bedingungen ausfallen, wird die Geheimhaltung schützenswerter Daten kompromittiert. Ferner müssen Schlüssel mit ausreichend großer LWE-Dimension gewählt werden, sodass sie für die Zeitspanne, für die Geheimtexte sicher bleiben sollen, nicht anfällig für Angriffe via bspw. Lattice-Reduction oder Dual-Attacks sind.

Der Simulator arbeitet auf 8-Bit positiven Ganzzahlen (256 Zustände) und Booleans (2 Zustände). Ein Server-Provider kann Werte wie Operanten, Instruktionen, Register, etc. während der Ausführung eines Programmes manipulieren. Durch den begrenzten Zustandsraum (insbesondere bei Booleans) können so Ergebnisse manipuliert werden, die User nicht durch einfache Plausibilitätschecks erkennen können. Auch wenn Server-Provider Ergebnisse nicht arbiträr wählen können, können sie quasi-unerkennbare Fehler in Programme einfließen lassen, und so Ergebnisse und anschließende Interpretation von Usern manipulieren.

Dieses Problem kann durch ein Trusted-Execution-Environment auf Hardwareseite gelöst werden, dabei muss Vertrauen in Hardware-Hersteller gegeben sein. Dank Hardware-Attestation-Keys kann die Existenz einer solchen Umgebung verifiziert werden, Vertrauen in Server-Provider ist nicht nötig.

Ebenfalls kann eine kryptographische Softwarelösung durch Zero-Knowledge-Proofs eingesetzt werden, dies ist jedoch nicht implementiert.

3.9.4 FHE-Designentscheidungen

Die Software verwendet ausschließlich TFHE-verschlüsselte Booleans und 8-Bit positive Ganzzahlen. Die darauf ausgeführten Operationen sind beschrieben in ???. Dabei werden Bitshift und -rotationen exklusiv im Kontext von Instruktionen ausgeführt, bspw.:

$$SLA: A \leftarrow A \ll 1$$

Shifts und Rotationen über Klartextwerte ist in TFHE-rs nicht rechenintensiv, dies steht im Kontrast zu Shift und Rotationen über verschlüsselten Werte, die sehr teuer sind.

Da in jedem Zyklus des Programmdurchlaufes immer alle Operationen ausgeführt werden müssen, sinkt die Performance mit jeder zusätzlichen Instruktion im Instruction-Set. Für jede Instruktion muss Gleichheit zum aktuellen Opcode geprüft werden, sie muss berechnet werden, sie muss in den CMUX-Tree aufgenommen werden, der den nächsten Wert des A-Registers bestimmt, evtl. müssen auch die CMUX-Ketten von Programmcounter, B-Register, Carry-Flag oder RAM-Zugriff angepasst werden.

Die Kosten vieler dieser ausgeführten Operationen skaliert mit der Größe der Operanden, somit wurde eine Wortbreite von 8-Bit für gewählt um angemessene Performance zu gewähren. Verwendete Datentypen sind somit `FheUInt8`, und `FheBool`. Mit den SUBC und ADDC Instruktionen unterstützt der Simulator dennoch nativ das Rechnen in höheren Zahlenbereichen.

Tabelle 5: Komplexität verwendeter Operationen

Operation	Kosten
Addition & Subtraktion	Gering
Bitweise Logik	Gering-Moderat
Bitshift & -rotationen	Gering
CMUX	Moderat (Effizient in Verbindung mit Bootstrapping)
Gleichheit	Moderat
Multiplikation & Division	Hoch

3.9.5 Komplexität der eigenen Algorithmen

Fundamental skaliert die Laufzeit einer Programmausführung linear mit der Anzahl der auszuführenden Zyklen: $\mathcal{O}(N_{cycles})$. Da bei der Ausführung aufgrund der Unwissenheit keine Bedingungen evaluiert werden können, und die Ausführung somit zwangsweise branchless geschieht, gibt es keinen Unterschied zwischen Worst-Case, Best-Case und Average-Case.

Interne Algorithmen haben folgende Laufzeitkomplexitäten':

- RAM Lese- und Schreibzugriffe werden in Batches parallel ausgeführt: $(\mathcal{O}(N_{adr_space}))$
- Ergebnisse von Instruktionen werden parallel berechnet: $(\mathcal{O}(N_{instructions}))$
- Der nächste Wert des A-Registers wird in einer Baumstruktur bestimmt: $(\mathcal{O}(\log(N_{instructions_A})))$
- Der nächste Wert des B-Registers wird sequenziell bestimmt: $(\mathcal{O}(N_{instructions_B}))$
- Der nächste Wert des Programmcounters wird sequenziell bestimmt: $(\mathcal{O}(N_{instructions_{pc}}))$
- Der nächste Wert der Carry-Flag wird sequenziell bestimmt: $(\mathcal{O}(N_{instructions_{CF}}))$

Wobei $(N_{instructions_A})$ die Menge an Instruktionen, die das A-Register verändern bezeichne, äquivalent für B-Register, Programmzähler und Carry-Flag.

Der verwendete Platz ist lediglich abhängig von der Größe des RAMs und der Bitbreite. Die Bitbreite ist dabei Konstant, weswegen die Platzkomplexität $\mathcal{O}(N_{adr_space})$ beträgt.

3.9.6 Performance-Messung

Da die Kommunikation über WebSockets geschieht, ist eine klassische Latenzmessung nicht Anwendbar. Ferner blockiert der Server die nebenläufige Ausführung mehrerer Programme, weswegen auch ein Lasttest mit mehreren Clients nicht sinnvoll ist. Stattdessen wurde ein Programm mit Das folgende Programm wurde als Benchmark gewählt:

```
; Addition zweier 64-Bit Zahlen.
; Ergebnis liegt nach 40 Takten vor, 104Byte RAM.
; $80d bis $87d = A, $88d bis $95d = B
; Ergebnis = $96d bis $103d
; Byte 0 | ; Byte 4
LDA $95d | LDA $91d
SWP      | SWP
LDA $87d | LDA $83d
ADD      | ADDC
LDR $103d| LDR $99d
; Byte 1 | ; Byte 5
LDA $94d | LDA $90d
SWP      | SWP
LDA $86d | LDA $82d
ADDC     | ADDC
LDR $102d| LDR $98d
; Byte 2 | ; Byte 6
LDA $93d | LDA $89d
SWP      | SWP
LDA $85d | LDA $81d
ADDC     | ADDC
LDR $101d| LDR $97d
; Byte 3 | ; Byte 7
LDA $92d | LDA $88d
SWP      | SWP
LDA $84d | LDA $80d
ADDC     | ADDC
LDR $100d| LDR $96d
```

Da das Programm selbst keinen Effekt auf die vom Server ausgeführten Berechnungen hat, können diese Messungen mit einem beliebigen Programm mit 104-Byte RAM, welches 40 Zyklen ausgeführt wird reproduziert werden.

Performance auf Netcup RS 1000 G9.5, limitiert auf 6 Kerne:

Werte sind Intervalle zwischen zwei Program-Execution-State-Response des Servers, das erste Intervall ist zwischen der Program-Execution-Request und der ersten Antwort. Dabei liegt die CPU-Auslastung durchschnittlich bei 5.32 Kernen, es werden durchschnittlich 420MiB RAM (Max. 605MiB) verwendet. Die Software ist nutzt für die Ausführung eines Zyklus' durchgängig das Maximum der verfügbaren Kerne, eine Messung mit mehr als sechs Kernen war in der Testumgebung nicht möglich. Stattdessen wurde eine weitere Messung auf einem AMD Ryzen 9 9950X3D mit 16 Kernen, 32GB DDR5 RAM @5600MHz durchgeführt. Ergebnisse sind in ?? aufgeführt.

Das Maximum ist bei beiden Messungen ein deutlicher Ausreißer, in beiden Fällen ist dies die erste Antwortzeit. Zu erklären ist dies dadurch, dass vor dem ersten Zyklus der ServerKey

Metrik	Zeit NetCup [s]	Zeit 9950X3D[s]
Gesamtdauer	1,080.397	459.901
Minimum	25.821	9.871
Maximum	40.681	33.014
p_{50}	28.127	10.560
p_{95}	30.010	12.669
$\emptyset$	28.432	11.433

Tabelle 6: Vergleich Antwortzeiten

dekomprimiert, und sämtliche Werte von Bas64 in TFHE-rs Formate gebracht werden müssen.

3.9.7 Limitationen

Fachlich stellen die Limitierte Wortbreite von 8-Bit, der 8-Bit-Adressraum, sowie das limitierte Instructionset die größten Limitationen dar. Das Instructionset ist zwar Turing-Complete, und bietet viele Funktionen, die über dieses Minimum hinaus gehen an, der Unterschied zu modernen Befehlssatzarchitekturen ist jedoch offensichtlich. Diese Limitationen sind allesamt durch die Performance von Operationen auf vollhomomorph verschlüsselten Daten zurückzuführen.

Fazit

Die Software ermöglicht die verschlüsselte Ausführung eines Programmes, ohne das eine nicht vertrauenswürdige dritte Partei, die das Programm ausführt Informationen über Inhalt oder berechnetes erlangen kann. Limitiert ist dieser Anwendungsfall dadurch, dass die vollhomomorph verschlüsselte Ausführung deutlich langsamer ist, als die unverschlüsselte Ausführung. So dauert eine Addition auf einem modernen Consumer-Grade-CISC-Prozessor meist einen Takt (mit 4GHz Takt $\hat{=}$ $0.25ns$), inklusive Pipelining, sodass vier Additionen von 64-Bit-Zahlen gleichzeitig ausgeführt werden können, und das Ergebnis der vierten Addition nach lediglich $0.4375ns$ bereit ist.

Um auf dem Simulator zwei 64-Bit-Zahlen zu addieren, müssen acht 8-Bit Additionen durchgeführt werden, zzgl. 24 Transportbefehle, und 8 SWAP-Befehlen für insgesamt 40 Instruktionen pro 64-Bit Addition,. Wie oben beschrieben benötigt der NetCup-Server hierfür 18.97 Minuten. Die vollhomomorph verschlüsselte Ausführung ist somit um einen Faktor von ca. $2.6 * 10^{12}$ langsamer, als die lokale Ausführung auf Consumer-Grade-Hardware.

Somit ergeben sich zwei Limitationen:

1. Das Instructionset ist für Berechnungen im Kontext schützenswerter Algorithmen, Daten, oder Programme evtl. nicht ausreichend
2. Die drastischen Performanceeinbußen bedeuten, dass die Ausführung des Programmes auf dem Endgerät des Nutzers um Größenordnungen schneller wäre, als sie verschlüsselt auf dem Server auszuführen.

Hierbei ist der letzte Punkt besonders relevant, da die lokale Ausführung des Programmes ebenso die Anforderung erfüllt, dass unauthorisierte dritte Parteien keine Informationen über die oben genannten schützenswerten Daten erlangen können.